Subsequence-Centric Methods for Enhancing Time Series Analysis: Piecewise Similarity, Covariate Augmentation, and Genomic Encoding

Coleman Yu

Abstract

Time series data are ubiquitous, appearing inherently in diverse fields or as transformations of non-temporal data such as biological sequences (strings). Many data mining tasks are defined on them, such as searching (query-by-content), motif (nearest-neighbor pair) finding, transformation (feature extraction), and classification. Given the prosperity of time series analysis, the development of robust and more flexible analytical frameworks is essential.

We first provide a one-sentence description of this thesis. This thesis develops three model-agnostic, subsequence-centric methods to advance time series analysis—by proposing efficient piecewise scaling for more flexible similarity computation, generating subsequence-derived covariates to improve forecasting accuracy, and encoding domain-specific biological subsequences into multivariate time series for classification—demonstrating how targeted exploitation of local patterns can benefit diverse downstream tasks from general time series modeling to bioinformatics. We then explain the three methods part by part.

The first part addresses the limitations of current similarity measures. While Dynamic Time Warping (DTW) and Uniform Scaling (US) are prevailing measures for handling local distortions and global scaling, respectively, and some studies have demonstrated that combining both DTW and US is necessary to obtain meaningful results. However, the current approaches fail when different phases of a process are expressed at varying speeds. They assume a single scaling factor for the entire sequence. We introduce Piecewise Scaling Distance (PSD), the first framework that achieves invariance to multiple local scaling factors, and incorporates efficient speed-up techniques. PSD provides a more flexible, expressive, and interpretable measure of similarity, as demonstrated by its ability to segment and identify different scaling factors within the two compared time series during the computation of the similarity measure.

The second part exploits a data mining primitive, namely Matrix Profile, to enhance time series forecasting. In brief, Matrix Profile encodes the information about the nearest neighbor for each subsequence of a time series. We argue that

historical subsequence occurrences contain predictive power. By identifying the k-nearest neighbors of subsequences in the past using Matrix Profile, we extract historical occurrence information as covariates for a forecaster. Our results demonstrate that these computationally efficient, data-driven covariates significantly improve forecasting accuracy across various domains.

The last part demonstrates the utility of time series analysis in bioinformatics through the prediction of Human Dicer Cleavage Sites. Current methods for RNA sequence analysis often rely on complex feature engineering or computationally expensive deep learning models. We propose MTSCCleav, a method that encodes 1-D RNA sequences and base-pair probabilities in predicted secondary structures into multivariate time series. To the best of our knowledge, we are the first to make use of the base-pair probabilities in predicted secondary structures. This approach allows the application of well-established time series tools to biological problems. Experiments show that MTSCCleav achieves state-of-the-art accuracy with a 3.7x to 28.8x speedup, while perturbation analysis confirms that pre-miRNA center regions are critical for cleavage-site prediction, consistent with the existing literature.

Collectively, this thesis advances the field of time series analysis by demonstrating that targeted, subsequence-level insights—through piecewise scaling, covariate augmentation, and encoding—provide a unified and powerful perspective for addressing fundamental challenges from general data mining to bioinformatics.

Acknowledgements

To begin, I want to express my deepest gratitude to my PhD supervisor at Kyoto University, Prof. Tatsuya Akutsu. I thank him for his kindness and patience. He is a brilliant researcher who can explain complex concepts simply. In a seminar, he showed that simply using clear notation makes presentations much easier to follow. More importantly, he helps students when they are in "valleys" in their lives. When a car is off track, it needs a push—so do people. I am equally indebted to my MPhil supervisor at The Hong Kong University of Science and Technology (HKUST), Prof. Raymond Chi-Wing Wong. He has continued to help and advise me even after I graduated from HKUST.

Akutsu-sensei taught me the importance of reading good materials. In the weekly seminars, he organized reading seminars, assigning some of his favorite books for us to read and discuss. His favorite book is "Probability and Computing" by Mitzenmacher and Upfal. Additionally, through the journal club he organized, students were asked to present the core ideas from their ten selected papers published in leading venues, such as Nature, Science, Cell, and PNAS. Reading beyond my fields helped me identify research gaps others missed. I think some great ideas are born when we merge the ideas from different fields. He also emphasized the importance of mathematics and proof. It is what computer scientists can stand out in the field of bioinformatics. I think it becomes especially true now, since advances in AI make it possible for almost everyone to code (i.e., Vibe coding). He advised me to use the existing tools, libraries, and studies to leverage my own research. He also taught me that a good researcher must master both oral and written presentation; otherwise, others may misinterpret your talent and the effort you put in. When we are reading a paper, he urges us to identify the novelty and analyze the pros and cons of the paper.

Raymond taught me the power of focus and "First Principles". I was always surprised that he did not use reference management software like Zotero, preferring to annotate hard copies and even type .bib files manually. He told me that when he starts to do research, he will print out the papers and get focused on the

stack of papers (hard copies) in front of him. I am not saying it is beneficial not to use tools, but I want to emphasize the power of focus that underlies his work routine. He taught me that every good research starts with a set of good papers. He also emphasized that there is no right or wrong in research, only what you choose to do about it. His research receipt works as follows. When you are tackling a problem, you first review the relevant existing studies to understand the state of the art (SOTA) for it. If the existing work addresses your particular problem, you can apply and adapt it to your problem setting. But most of the time, since your problem must be a particular version of a general problem, the existing general solution should not work well for it. It means that you have some room to improve it. And this is the research gap!. It reminds me of when we deal with the NP-complete problem. As Kleinberg and Tardos's Algorithm Design (Chapter 10) suggests, an NP-complete problem (assuming $P \neq NP$) does not allow us to have an algorithm that possesses all three of the following desired properties simultaneously: Efficiency, Correctness, and Generalization. Hence, it is sometimes preferable to address a specific instance of the problem rather than the general one. He also emphasized the theory. He consistently noted that you need to add theory to the paper to strengthen it. He always mentioned that you need three motivations (why you are doing it) and three contributions (what you have done). Sometimes, I ask why he can write so fast, and he replies, "If you know what you are doing, then you write fast.". He also taught me that when you don't know some of the ideas, you just go back to the original idea. Doing a "deep first search" can help you reach the first principle, a concept that Elon Musk, one of the greatest entrepreneurs of our time, always emphasized.

Without these two supervisors, I would not have made it this far. I could not have finished this program.

I would like to take this opportunity to share some of my thoughts on my PhD journey. I hope the readers may learn something from what I have gone through. My PhD journey was, to quote Dickens, "the best of times and the worst of times", and to quote Churchill, "This was their finest hour". For the best parts, it allows me to explore both in my daily life and in my research. I tried many things, walked many roads, drank many colas and alcohol, and met many people. This helped me see problems from new perspectives. Regarding the worst parts, I am reminded of a quote from one of my favorite movies, "Les Choristes": "Fond de l'étang". It literally means "Bottom of the Pond". At times, I felt like a frog at the well's bottom, trying hard to get out. Research is fun but also hard. Research is about exploring something new. It is about publishing (so others can learn from it).

When I am stuck, the best solution is to aim for a reachable, well-defined goal. The goal should be clear, with obvious rewards and requirements. Also, make sure the effort of your actions can be accumulated. Like the frog, do not jump randomly, but aim to move to stable platforms towards escape. Then, each jump matters for your progress. Two of the materials helped me; they are "Eat the Frog!" and "THE PH.D. GRIND".

I would like to thank my thesis committee members, Prof. Tatsuya Kawahara and Prof. Hisashi Kashima. I appreciate their willingness to serve as committee members, to provide me with valuable advice. I would like to acknowledge Tamura-sensei and Mori-sensei (a former member) of the Akutsu laboratory. Tamura-sensei taught me to focus on the input-output relationship when encountering new algorithms/methods, and Mori-sensei introduced me to the fascinating application of time-series analysis to gene expression data, especially in trajectory inferences (i.e., pseudotime). I would like to thank the many communities that made my life in Japan colorful. Thank you to the people of the dormitories where I lived at the Uji and Yoshida campuses. I am grateful to my Japanese language teachers. I also cherish the friends I met through Akutsu's laboratory and the extracurricular activities, including Kendo, Table Tennis, Naginata, Karate, and Aikido. I also met many interesting people from various international student events.

I am honored to receive the Asian Future Leaders Scholarship Program (AFLSP) scholarship to support my studies in Japan. I have made a great choice by enrolling in Kyoto University Design School, which has provided me with many valuable interdisciplinary experiences and opportunities to meet people outside my field. In this program, I have conducted fieldwork in Okinawa, Hong Kong, and Bali, a rare opportunity for researchers in computer science. I would also like to thank the Institute for Chemical Research (ICR) for the Research Assistant position in the Akutsu laboratory.

Finally, I would like to express my deepest gratitude to my family and my friends who have stayed by my side with their countless acts of support; I have nothing to return to them but my love and time.

A special acknowledgment goes to my niece. As you grow up, I hope you explore the world and fulfill your eagerness for knowledge. Find materials that interest you. One day, you might find this thesis online and, I hope, find it worth reading and inspiring.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Human beings generate a large amount of data nowadays. We are producing more new data in one single day today than in the first twenty-one centuries of AD combined. We are drowning in information but thirsty for knowledge. It is natural for us to develop computational methods to accelerate the process of "harvesting" knowledge from information.

Computational tasks typically aim to uncover the underlying functions that govern input-output relationships using either algorithmic (No "learning" from the data, e.g., the minimum-cost flow problem (MCFP)) or machine-learning approaches ("Learning" occurs during the training phase). Machine learning is broadly categorized into supervised learning (e.g., classification) and unsupervised learning (e.g., clustering). They fundamentally rely on how we define the objects to focus on the features that we are interested in and the similarity between objects.

Time series data are ubiquitous across domains like finance, healthcare, and engineering. Many critical data mining tasks are defined on them, including searching (query-by-content), motif finding, and classification. Given the prosperity of time series analysis, developing robust and flexible analytical frameworks is essential.

We first introduce the one-sentence description to set up the overall theme of this thesis. This thesis develops three model-agnostic, subsequence-centric methods to advance time series analysis—by proposing efficient piecewise scaling for more flexible similarity computation, generating subsequence-derived covariates to improve forecasting accuracy, and encoding domain-specific biological subsequences into multivariate time series for classification—demonstrating how targeted exploitation of local patterns can benefit diverse downstream tasks from bioinformatics to general time series modeling.

1.1 Contributions

Here, we review the three studies.

The first study is about Piecewise Similarity. Prevailing measures like Dynamic Time Warping (DTW) and Uniform Scaling (US) assume a single, global scaling factor for the entire sequence, failing when different phases of a process are expressed at varying speeds. We introduce Piecewise Scaling Distance (PSD), the first framework to achieve invariance to multiple local scaling factors while incorporating efficient speed-up techniques. PSD provides a more flexible and interpretable measure of similarity by automatically segmenting and identifying different scaling factors within compared series. Experiments show it outperforms traditional measures like ED and DTW.

The second study is about covariate augmentation. Standard sliding-window forecasting models are often limited by making predictions based only on observations within a narrow lookback window. We leverage the Matrix Profile to identify the left (past) nearest neighbors of subsequences. Information from these historical occurrences—such as immediate subsequences and similarity scores—is extracted as new covariates for a forecaster. These data-driven covariates significantly improve forecasting accuracy across various domains by providing a "lookahead" of future trajectories based on similar historical patterns.

The third study is about encoding. Predicting Human Dicer Cleavage Sites typically relies on complex feature engineering or computationally expensive deep learning models, which often lack generalizability. We propose MTSCCleav, which frames cleavage site prediction as a multivariate time series classification problem. It encodes 1-D RNA sequences and, for the first time, base-pair probabilities from secondary structures into multivariate time series. This approach achieves state-of-the-art accuracy with a 3.7x to 28.8x speedup compared to current methods. Perturbation analysis confirms that pre-miRNA center regions are critical for prediction, aligning with existing biological literature.

1.2 Organization

The remainder of this thesis is organized as follows: Chapter 2 reviews preliminaries in time series data mining. In particular, we focus on (1) distance measures, (2) subsequence analysis using matrix profile, and (3) transformations for feature extraction. Chapters 3, 4, and 5 detail the three main studies on Piecewise

Similarity, Covariate Augmentation, and Encoding, respectively. Finally, Chapter 6 provides conclusions and directions for future work.

Chapter 2

Preliminaries and Related Work

In this chapter, we provide a background on time series analysis. A more detailed background will be presented in the respective chapters if necessary.

Since many tasks require measuring the similarity between two time series, we introduce several popular distance measures in Section 2.2, which complements Chapter 3. They are developed for achieving different kinds of invariances. Note that there is no single best distance measure that can handle all types of data. In particular, we introduce "CID" and "Prefix and Suffix Invariant Dynamic Time Warping (ψ-DTW)". CID can be regarded not as a single distance measure but as a framework for achieving complexity-invariance with respect to other distance measures used as the base measure. ψ-DTW proposes the idea that we need to ignore some subsequences during comparison for better results, which highlights the situation that whole sequence comparison performs poorly.

Then, we introduce the analysis of time series using their subsequences in Section 2.3. In detail, we focus on a powerful time-series data-mining primitive, namely Matrix Profile, which complements Chapter 4. Matrix profile can be considered as a tool for annotating a given time series, enabling many interesting data mining tasks.

Finally, we focus on time series classification (TSC) in Section 2.4. In particular, we introduce methods for transforming time series into tabular representations, thereby enabling the use of other methods that were not originally designed for time series, which complements Chapter 4. A popular downstream method, namely the Ridge Classifier, would be introduced. Due to the prosperity of deep learning methods recently, we also provide a brief survey of deep learning methods in time series classification.

We first introduce the basic definitions of time series and subsequences. After that, some tasks defined on time series are also introduced.

2.1 Basic Definitions

We begin by defining a time series. A general form of a time series T is an ordered pair of n real-valued variables, $T = (b_1, c_1), (b_2, c_2), \ldots, (b_n, c_n)$, where b_i is the behavioral attribute and c_i is the contextual attribute, where $1 \leq i \leq n$. c_i refers to the time stamp at which the measurement b_i is taken. Since the measurements are always taken in a uniform manner, c_i is simply incrementing from 1 to n uniformly. Hence, we can represent a time series more concisely as $T = t_1, t_2, \ldots, t_n$, where $t_i = b_i$.

Definition 1 (Time Series)**.** A time series $T = t_1, t_2, ..., t_n$ is a sequence of real-valued numbers with length $= n$.

We may be interested not only in the entire time series but also in a segment of it, called a subsequence.

Definition 2 (Subsequence)**.** A subsequence $T(i : j)$ of a time series T is a shorter time series that starts from position i and ends at position j with length $= j - i + 1$. Both ends are inclusive. Formally, $T(i : j) = t_i, t_{i+1}, ..., t_j$, $1 \leq i \leq j \leq n$.

We call $T(1 : m)$ as the prefix of length m of T, m-prefix in short.

Many problems can be formulated on time-series data. Several of them are listed below.

- **Classification**: Learning from a set of time series with assigned label. Assigning a predefined label/category to a new time series. It will be further discussed in Chapter 5.
- **Clustering**: Grouping a set of unlabeled time series based on their similarity. In bioinformatics, this is often applied to time-course gene expression data to group genes with similar expression patterns. The resulting clusters can be biologically validated using external tools like functional enrichment analysis (e.g., Gene Ontology enrichment or GSEA), which evaluates whether the genes within a cluster are significantly associated with shared biological functions or pathways (i.e., enriched).
- **Searching**: Given a query, retrieve the most similar time series (or top-k) from the dataset. This task is also called "query-by-content". It will be further discussed in Chapter 3.

- **Segmentation**: Partitioning/Segmenting a long time series into several contiguous subsequences, which are called regimes or states. It will be further discussed in Chapter 3.

- **Forecasting**: Given historical time series data (i.e., training set), predicting future values (horizon), which can be one-step ahead or multi-step ahead. In addition, there are two main approaches, depending on whether the predicted results are used in the next round of prediction. They are called "Iterative methods" and "Direct methods" [2]. One goal is to predict a horizon h. We can either predict the complete horizon with length h in a single batch, as in "Direct methods". Or at each iteration, we can predict a shorter horizon $h' < h$ and use the predicted results to predict the next h' steps and finally predict all the h steps. Since predicted results are used in iterative methods, small errors at each iteration accumulate and can lead to large errors over iterations. In contrast, direct methods produce the final result in a single batch (i.e., one iteration only). However, the models would be more complex. It will be further discussed in Chapter 4.

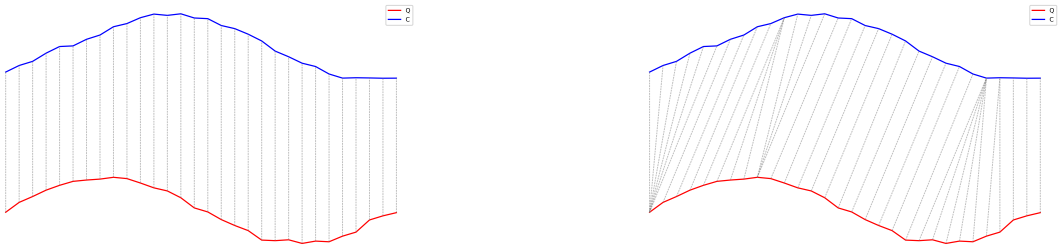
2.2 Distance Measures

To quantify the similarity between two time series, we need to define a distance measure, also known as a similarity measure, between them. A smaller value means that the two time series are more similar. Most of the distance measures require whole-sequence comparisons, whereas others allow us to ignore parts of the data.

Many distance measures have been proposed in the literature. Among them, the most established measures are undoubtedly Euclidean Distance (ED) and Dynamic Time Warping (DTW). They are representatives of the two board classes of distance measures, namely "lock-step" and "elastic".

2.2.1 Euclidean Distance (ED)

ED is a lock-step distance measure. Given two time series Q and C with the same length n, it compares the time point q_i of Q with the time point c_i of C at the same time (index). Note that, traditionally, lockstep distance measures require the two time series to have the same length because of the one-to-one alignment, as shown in Figure 2.1. However, in the setting of query by content, where it is always the case that $|Q| < |C|$, we can still apply a lock-step distance measure by either comparing Q with $C(1 : |Q|)$ or padding Q using its last element to lengthen



(a) ED (b) DTW

Figure 2.1: Alignments.

it to the same length of C. Minkowski distance is a generalization of Euclidean distance. Minkowski distance is the L_p-norm of the difference between the two time series X and Y, defined as:

$$\mathrm{D}(X,Y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}} \tag{2.1}$$

When $p = 2$, it corresponds to the Euclidean distance. When $p = 1$, it corresponds to the Manhattan distance. When $p = \infty$, it corresponds to the Chebyshev distance. In our studies, we focus on the Euclidean distance. Other lock-step distance measures include Pearson correlation distance. It accounts for the linear association between the two time series using the Pearson correlation coefficient. To note, there is another measure called Edit Distance for strings. And Edit Distance is also sometimes abbreviated as ED in string processing or bioinformatics. In this study, we focus on time series analysis and use ED to denote Euclidean distance rather than edit distance.

2.2.2 Dynamic Time Warping (DTW)

Dynamic Time Warping is an elastic measure [3]. In contrast to lock-step distance measures, elastic distance measures allow one-to-many point matching, as shown in Figure 2.1. The one-to-many point matching allows the elastic distance measures to warp in the time axis (i.e., temporally) such that it can handle the local temporal distortions. While it will be detailed in Chapter 3, it is briefly explained here. In short, it minimizes the cumulative distance between two time series, subject to constraints, by finding an optimal warping path W^* in a cost matrix, where W is the set of all possible paths. The constraints typically are (1) Boundary conditions, (2) Continuity, and (3) Monotonicity. This warping path provides feasibility for

the point alignments to go beyond the 1-1 point alignment, as in the case of a lock-step distance measure, Note that we can reduce pathological warping and accelerate computation by introducing a warping window. DTW with a warping window constraint is called constrained DTW (cDTW). Two famous windows are the Sakoe-Chiba band [3] and the Itakura parallelogram [4]. In the study, we focus on the Sakoe-Chiba band.

2.2.3 Derivative Dynamic Time Warping (DDTW)

The Derivative Dynamic Time Warping (DDTW) is a variant of DTW [5]. Instead of comparing original raw values, it compares two time series using their first-order derivatives, but with an approximation. In DTW, a point on a rising trend may be mapped to a point on a falling trend. It goes against our intuition. It can be solved by comparing their first-order derivatives, which encodes the slope information. The derivative T' of a time series T is computed approximately as follows.

$$t'_i = \frac{(t_i - t_{i-1}) + \frac{t_{i+1} - t_{i-1}}{2}}{2} \tag{2.2}$$

This estimate is simply the average of "the slope of the line through t_i and t_{i-1} (i.e., its left neighbor)" and "the slope of the line through t_{i-1} (i.e., its left neighbor) and t_{i+1} (i.e., its right neighbor)". The $1/2$ term in the second item of the numerator comes from the fact that the separation in time of the t_{i-1} and t_{i+1} is 2. Note that the estimate is not defined for the first and last elements of the time series in the above equation. In these boundary cases, we use the estimates of the second and penultimate (i.e., the second-to-last thing) as the estimates for the first and last elements, respectively.

2.2.4 Weighted Dynamic Time Warping (WDTW)

The Weighted Dynamic Time Warping (WDTW) is a variant of DTW [6]. It is a penalty-based DTW designed to prevent pathological paths. Recall that a warping window (e.g, Sakoe-Chiba band) is enforced on the cost matrix of DTW, such that some paths are excluded. Only the paths that reside entirely in the warping window are feasible. This constraint may be too strict. WDTW uses a softer way for the same purpose. Instead of using a window to forbid the alignment of x_i and y_j that are far away in time. WDTW weights the cost of such alignment by multiplying it by a modified logistic weight function (MLWF) $\omega(k)$, defined as

follows.

$$\omega(k) = \frac{\omega_{\max}}{1 + \exp\left(-g \cdot (k - m_c)\right)} \tag{2.3}$$

Where:

- $k = |i - j|$. It is the phase difference (i.e., distance on the time axis from the diagonal. The diagonal refers to the line where $i = j$.)
- $\omega_{\max}$ is the desired upper bound for the weight parameter, which is suggested to be set to 1.
- m_c is the midpoint of a sequence. $m_c = m/2$.
- g is a constant that controls the level of penalization. It controls the curvature (slope) of the function.

Intuitively, if x_i and y_j are far apart temporally, it will have a larger weight to discourage their alignment and vice versa.

2.2.5 Weighted Derivative Dynamic Time Warping (WD-DTW)

[6] also proposed the weighted version of DDTW. In brief, a weight is applied to the local cost function when computing DTW on the first derivative.

2.2.6 Shape Dynamic Time Warping (shapeDTW)

The Shape Dynamic Time Warping (shapeDTW) is a variant of DTW [7]. The main modification to the original DTW is the way the local distance between points is computed. Recall that DTW compares single scalar points. shapeDTW compares local descriptors. The local descriptors are constructed using a sliding window on the original series, such that for each point x, a L-subsequence with x as the center is extracted to compute the higher-level feature of x. L is the user-given length of the subsequence to consider. By default, it is set to 15 in Aeon, which is a time series library in Python. There are several ways to construct such a descriptor. For example, a raw subsequence (i.e., a set of neighbor points surrounding the point of interest), Piecewise aggregate approximation (PAA) [8, 9], slope, derivative, HOG-1D [10].descriptor.

Then, the distance between descriptors is calculated rather than between raw values. When a raw subsequence is chosen to construct the local descriptors, a

common metric used for comparing two local descriptors is the Euclidean distance. In the evaluation, a raw subsequence is chosen to construct the local descriptors, which is the default setting in Aeon.

2.2.7 Amercing Dynamic Time Warping (ADTW)

The Amercing Dynamic Time Warping (WDTW) is a variant of DTW [11]. It is also designed to constrain the amount of warping, as in cDTW and WDTW. While cDTW imposes a hard window and WDTW uses multiplicative weights (i.e, MLWF), ADTW introduces an additive penalty for non-diagonal alignment. The word "Amercing" means "fining". The non-diagonal alignments are required to pay the fines. Unlike WDTW, which uses a multiplicative weight based on the position of the alignment, ADTW applies an additive penalty ω based on the action of warping. The non-diagonal alignments are penalized. Formally, the recursive relation for ADTW is defined as:

$$\begin{aligned} D(i,j) &= \mathrm{d}(q_i, c_j) \\ &\quad + \min \begin{cases} D(i-1, j-1), \\ D(i-1, j) + \omega, \\ D(i, j-1) + \omega \end{cases} \end{aligned} \tag{2.4}$$

ADTW penalizes the last two alignment actions. ω is a user-given hyperparameter. It should be a non-negative scalar constant. In practice, it is defined through cross-validation, which determines the optimal ω by training on a subset of data or heuristic search, which searches values in a user-given range.

To note, ADTW generalizes ED and DTW. If $\omega = 0$, no need to pay the fine for the non-diagonal alignment, which reduces to DTW. If $\omega \to \infty$, the non-diagonal alignment becomes prohibitive, and it reduces to ED.

2.2.8 Complexity-Invariant Distance (CID)

Real-world data often exhibit intrinsic variations that must be "calibrated" or "penalized" during measurement to prevent the distance from being artificially affected and match our intuition. We use an interesting distance measure, namely Complexity-Invariant Distance (CID) [12], to demonstrate this. Ordinary Euclidean distance often misclassifies complex time series, treating them as more similar to simple time series than to other complex sequences. This occurs because simple

time series often present an average/dull pattern that can minimize the point-to-point Euclidean distance when compared to a complex sequence. For example, a single mismatch between a peak and a valley in two complex time series can result in a large error term. In contrast, the average pattern in a simple time series would not lead to a large error term even if the alignment is not good. To fix this discrepancy, CID introduces a penalty for complexity differences. CID estimates the complexity of a sequence based on the physical intuition of "stretching" the time series until it forms a straight line. A complex time series would result in a longer stretching time series because of more peaks and valleys. A simple time series would result in a shorter one. The Complexity Estimate (CE) for a sequence Q of length n is defined as:

$$CE(Q) = \sqrt{\sum_{i=1}^{n-1}(q_i - q_{i+1})^2} \tag{2.5}$$

The Correction Factor (CF) between two sequences, Q and C, is defined as follows:

$$CF(Q, C) = \frac{\max(CE(Q), CE(C))}{\min(CE(Q), CE(C))} \tag{2.6}$$

The final CID is calculated by multiplying the base distance measure by this correction factor. Here, we use ED for the base distance measure:

$$CID(Q, C) = ED(Q, C) \times CF(Q, C) \tag{2.7}$$

To note, if Q and C have the same complexity $CE(\cdot)$, the calibration factor would be one. Hence, CID would degenerate to the original base distance measure.

There are other cases where we need to calibrate the time series during measurement, such as uniform scaling. It is further discussed in Chapter 3.

2.2.9 Prefix and Suffix Invariant Dynamic Time Warping

Sometimes, we cannot consider the whole sequence; we need to ignore some data to get a meaningful result.

The endpoint constraints of DTW force the alignment of the first and last observations of the compared sequences. Sometimes, there are unrelated head and tail noise in the time series, and they need to be ignored during the distance computation.

Prefix and Suffix Invariant DTW (ψ-DTW) [13] addresses this by relaxing the endpoint constraint, allowing the algorithm to dynamically ignore a defined amount r of leading and trailing values in one or both time series.

Given two time series x and y of lengths n and m respectively, and a user-defined relaxation factor r, ψ-DTW alters the initialization of the traditional DTW cost matrix:

$$D(i,j) = \begin{cases} \infty, & \text{if } (i = 0 \text{ and } j > r) \text{ or } (j = 0 \text{ and } i > r) \\ 0, & \text{if } (i = 0 \text{ and } j \leq r) \text{ or } (j = 0 \text{ and } i \leq r) \end{cases} \tag{2.8}$$

The above equation defines the base cases of DTW. The remaining part is the same as ordinary DTW. This study shows that a single time series in the dataset may not only contain the desired action. Hence, some data in the prefix or suffix should be ignored. We further argue that a time series may contain several actions in a row, instead of a single action in Chapter 3.

2.3 Subsequence analysis using matrix profile

Matrix Profile [14] is a data mining primitive that can solve many tasks by computing the nearest neighbor of each m-subsequence in T_A. To note, those nearest neighbors could be found within the same time series (i.e., T_A) or from other time series (i.e., T_B). The former case is called self-join or AA-join. The latter case is called AB-join.

First, we introduce the Distance profile, which is used to compute the Matrix Profile. Besides, we focus on AA-join situation first.

Definition 3 (Distance profile)**.** A *distance profile* $D_i = d_{i,1}, d_{i,2}, \ldots, d_{i,n-m+1}$ of a T is a vector of the distances between a given subsequence $T_{i,m}$ and each subsequences $T_{j,m}$ in T, where $d_{i,j}$ is the distance between $T_{i,m}$ and $T_{j,m}$, $1 \leq i, j \leq n - m + 1$.

Definition 4 (Matrix profile)**.** A *matrix profile* $P = \min(D_1), \min(D_2), \ldots,$ $\min(D_{n-m+1})$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T.

Another closed related time series of P, namely the Matrix Profile Index I, stores the location of the corresponding nearest neighbor.

Definition 5 (Matrix profile index)**.** A *matrix profile index* $I = I_1, I_2, \ldots, I_{n-m+1}$ of T is a vector of integers, where $I_i = j$ if $d_{i,j} = \min D_i$.

With P and I, we know the location and similarity of the nearest neighbor of each m-subsequence in T. It can be computed in $\mathcal{O}(n^2)$ [15, 16].

2.3.1 Time series motifs

Time series motifs are the most similar pairs of subsequences within a dataset [17]. They find applications in music and machine log data. With the matrix profile, they can be identified by locating the two lowest values from P. Then, the two subsequences can be extracted from the corresponding locations, informed by I. It can be extended to find the top-k motifs. Additionally, we can identify other members of a motif using the following idea. Let's denote the two subsequences of the motif as A and B in a time series T. First, compute the mean time series M from A and B by averaging. Set a threshold which equals $\theta = r \times \text{ED}(A, B)$, where $r > 1$ is user-given and $\text{ED}(A, B)$ is obtained from P. Then, we compute a distance profile of M on T, which encodes the distances between M and each subsequence with length $|M|$ in T. Any part of the distance profile that is smaller than θ points to a member of M in T. Now, we have a motif with more than 2 members. To distinguish, we call it a motif family.

2.3.2 Discords

Discords refers to outliers. In time series, they refer to the most unusual subsequences. Given the matrix profile, the highest peak (greater than a user-defined threshold) corresponds to a discord. The idea is that if a subsequence's nearest neighbor is far away from it (i.e., a peak), no other subsequence is similar to it, so it is an outlier.

2.3.3 Time series chains

It is a special kind of pattern that captures evolutionary behavior [18]. Note that not any two members of the chain are similar. But a member should be similar to some member in the chain. Besides, a member is regarded as the initial pattern, and a member is regarded as the result pattern. Time series chains capture this continuously evolving directionality. It is defined on left/right version of the distance profile, matrix profile, and matrix profile index. We define the left version as follows. The right version can be defined similarly.

Definition 6 (Left distance profile)**.** A *left distance profile* $D_i^L = d_{i,1}, d_{i,2}, \ldots, d_{i,i-\lceil m/4 \rceil - 1}$ of T is a vector of Euclidean distances between a given subsequence

$T_{i,m}$ and each subsequence that appears before $T_{i,m}$. To note, $i - \lceil m/4 \rceil - 1$ is the index location of the last eligible subsequence before $T_{i,m}$ because of the exclusion zone.

Definition 7 (Left matrix profile)**.** A *left matrix profile* $P^L = \min(D_1^L), \min(D_2^L), \ldots, \min(D_{n-m+1}^L)$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T before it.

Definition 8 (Left matrix profile index)**.** A *left matrix profile index* I^L is a vector of integers $I_1^L, I_2^L, \ldots, I_{n-m+1}^L$ of T, where $I_i^L = j$ if $d_{i,j} = \min D_i^L$.

The left (right) matrix profile index defines a directional link between two similar subsequences. By analyzing these directional links, a sequence of evolving patterns over time can be identified. The time series chain captures degradation or evolving trends.

2.3.4 Segmentation

Semantic segmentation aims to partition a whole time series into contiguous subsequences [19]. Each subsequence is internally consistent. It is called a regime. The main idea is that a subsequence in a regime should be similar to other subsequences in the same regime. Recall that I indicates the index of the nearest neighbor of a subsequence. Intuitively, these indices refer to similarity arcs between subsequences and their corresponding nearest neighbor. Then, for each time point, we can calculate how many arcs have passed through this point. A time point where few arcs cross indicates that it is a boundary of a regime, as the subsequences of its left are not similar to those on its right.

2.3.5 MPdist

The aforementioned distance measures require the main body (even for ψ-DTW) of two time series to be aligned. MPdist moves the whole-sequence-based similarity to subsequence-based similarity.

MPdist [20] (Matrix Profile distance) is a distance measure that evaluates similarity based on shared subsequences, regardless of their positions [20]. Traditional measures, such as ED or DTW, compute similarity by aligning and explaining *all* data points within two time series. However, some parts of the time series may be irrelevant. MPdist considers two time series to be similar if they share a large set of similar subsequences. Thus, some irrelevant subsequence can then be intentionally ignored.

Its computation relies on using a data mining primitive, namely the Matrix Profile. The algorithm first extracts all subsequences from two compared series A and B using a sliding window of length m. Then, an ABBA similarity join J_{ABBA} is created, which consists of each subsequence in A with its nearest neighbor in B and vice versa. We have another vector, namely the Join Matrix Profile P_{ABBA} to store the corresponding z-normalized ED of the pairs stored in J_{ABBA}. To note, $|P_{ABBA}| = |J_{ABBA}|$. The algorithm then reports the k^{th} smallest value in the sorted (ascendingly) P_{ABBA} as the final result, where k is suggested to be set to 5% of the whole length. If the minimum distance in P_{ABBA} is used, the distance measure is overly generous and provides no discrimination power. If the maximum distance in P_{ABBA} is used, it is sensitive to every single subsequence, including the irrelevant ones. It has a nice property that when $m = n$, where $n = |A| = |B|$, MPdist degenerates to ED. It can be computed in $\mathcal{O}(\text{Number of subsequences} \times n)$. To note, it does not satisfy the triangle inequality, so it is not a distance metric.

2.3.6 Shapelets

Shapelets [21] are subsequences that are maximally representative of a class. They are the characteristics of a time series. One advantage is its interpretability. Shapelets offer domain experts a tangible, visual explanation for classification. For example, a ECG time series is classified as abnormal because it contains a certain shape of subsequence, which is the inverted T-wave. The classifier needs to know only whether the candidate time series contains this shaplet, not where it is. So the classifier is also robust to noise.

There are several methods for creating shapelets [21, 22]. Here, we present how to use the matrix profile to extract the candidates for shapelets. Recall that a normal matrix profile is calculated from a single Time series T_A. It finds the nearest neighbor of each m-subsequence both in the same time series. It can be extended to find the nearest neighbor in T_B of each m-subsequence in T_A. The resulting matrix profile is called "AB-join" P_{AB}, while the original matrix profile is called "self-join" or "AA-join" P_{AA}.

For each m-subsequences in T_A, an AB-join compares it with all of the m-subsequences in T_B. P_{AB} is the corresponding distance vector. To note, AB-join is not the same as BA-join, because the latter one refers to the operation of annotating every subsequence in T_B with its nearest subsequence in T_A.

Matrix profiles can be used to identify candidate shapelets by identifying subsequences in T_A that have a similar nearest neighbor in T_A but not in T_B, and vice versa.

In other words, we can compute $P_{\text{diff}} = P_{AB} - P_{AA}$ and the resulting P_{diff} are good indicator of shapelet candidates because they are well conserved in their own class A but cannot find a similar match in the other class B. In this simple case, one class only contains a time series A, and the other class only contains a time series B.

2.4 Time series classification

2.4.1 Transformations for feature extraction

Here, we introduce three transformation methods, including hctsa [23], catch22 [24], and ROCKET [25] (and its variants [26, 27, 28]). They fall into two categories of transformation: curated and non-curated. The transformations of hctsa and catch22 are curated to incorporate the domain knowledge. Hence, the features generated are interpretable. The transformations of ROCKET and its variants are non-curated. They simply generate a large set of features quickly and rely on a downstream classifier, such as a Ridge Classifier, to learn which features are useful for classification. They output feature vectors that capture the characteristics of the original time series, which are eventually input to the downsteam methods, such as classifiers, for the final results.

hctsa

hctsa [23] computes over 7,700 features from time series by leveraging past domain knowledge. Recall that we can summarize a time series by measuring its statistical values, such as the mean. This study uses a set of curated algorithms that incorporate domain knowledge across domains to select useful and interpretable features. Each feature encodes a different scientific analysis method. The methods range from simple variance to nonlinear dynamics and entropy. The result can be treated as a feature matrix with a row for every time series and a column for every feature. With the matrix, downstream tasks such as classification or regression can be applied. This exhaustive approach enables researchers to identify which features are useful to the downstream tools, such as XGBoost, which can tell which features are the most discriminative by plotting the feature importance graph. One obvious limitation of this approach is its computational cost.

catch22

catch22 [24] is a follow-up study of hctsa. It aims to find a small set of time-series features that exhibit strong classification performance across a given collection of time-series problems and are minimally redundant. It filters 22 highly predictive, mathematically independent, and fast-to-compute features from the large curated hctsa feature set. Thus, it provides a lightweight, canonical subset of features that retains good interpretability. Note that, with much fewer features, this yields a small tabular representation of the time series. Hence, it can be used with more computationally expensive methods. As a result, it provides a good balance between speed, accuracy, and domain explainability.

ROCKET

The main difference between ROCKET [25] (and its variants [26, 27, 28]) and the aforementioned approaches is that the transformation is based on thousands of (random) convolutional kernels—filters with randomly assigned lengths, weights, and dilations —rather than on curated methods. To note, in MINIROCKET [26], the convolutional kernels are created deterministically for faster speed. By sliding these random filters across the time series, ROCKET extracts the summary statistics, such as the proportion of positive values. The main idea is that by a sufficiently large and diverse set of random kernels, some local shapes can be captured. ROCKET has good predictive accuracy, and it's fast because the features are simply extracted by computationally cheap kernels. However, this has a main disadvantage: the resulting features are difficult to interpret.

2.4.2 Ridge Classifier

The classifier that is usually chosen to work with ROCKET and its variants is a ridge classifier. The main advantage of it is speed. ROCKET and its variants generate a large number of features, making a fast classifier desirable.

A ridge classifier is a wrapper that uses a ridge regression model as a routine to perform classification. It first maps the categorical labels of targets into continuous numbers, does the regression, and finally thresholds the numerical results from the regressor to obtain the classification result.

Given a training dataset $D = \{(x_i, y_i)\}_{i=1}^{n}$ with n instances, where $x_i \in \mathbb{R}^P$ is the feature vector with P dimensions and $y_i \in \{+1, -1\}$ is its label, it minimize the following optimization function.

$$min_w \left(\sum_{i=1}^{n} (x_i^T w - y_i)^2 + \lambda \|w\|_2^2 \right) \tag{2.9}$$

Where $\|w\|_2^2 = \sum_{j=1}^{p} w_j^2$ is the L_2 norm of the weight vector and $\lambda > 0$ control the penalty. We explain the equation in brief. There are two terms inside the bracket. The first term is simply the sum of the residual, same as the one in the least squares method. The second term is called the L_2 penalty and is used to introduce bias in the fit to avoid overfitting. Hence, λ serves as a regularization hyperparameter between the trade-off between bias and variance.

Since the above optimization function in a ridge classifier has a closed-form solution, it can be solved using linear algebra rather than iterative optimization, as in logistic regression. Besides, the generated features by the random kernels in ROCKET and its variants are highly correlated. Ridge regularization, also known as the L_2 norm, can handle this case. Ridge regression shrinks regression coefficients toward zero by adding an L2 penalty. It reduces model complexity and helps with multicollinearity.

2.4.3 Deep learning for Time Series Classification

Recently, deep learning has made advancements in many different aspects in data science such as computer vision. This thesis is not focused on deep learning methods. But for completeness, we briefly review some of the major deep learning architecture used for time series classification (TSC) here, with special focus on two recently introduced architecture, namely InceptionTime and TimesNet. For a comprehensive review, we direct the readers to [29].

One of the earlier work apply deep learning on TSC is [30]. They used a multi-scale convolutional neural network (MCNN), which archive similar performance to the state-of-the-art TSC classifier namely COTE, in the year of 2016. After that, many specific architectures, mainly based on Convolutional Neural Networks (CNNs) has been proposed.

Multi Layer Perceptron (MLP), Fully Convolutional Networks (FCN) and Residual Network (ResNet) have also been proposed [31]. These models have a shorter training and deploying time compared to MCNN and the ensemble-based classifiers, which combines the predictions of multiple weak learners.

Temporal CNN (tempCNN) has been proposed for TSC and applied in the area of remote sensing [32]. It is inspired by the success of the Convolution Neural

Networks (CNN) for two dimensional image recognition. It adopts one dimensional CNNs.

One of the advantages of deep learning methods is transfer learning [33]. The model can be pre-trained on one dataset and then continue to train on a new dataset. It is particularly useful when the new dataset does not have enough labeled data.

InceptionTime

InceptionTime [34] is an ensemble of deep Convolutional Neural Network (CNN) models. Its aim is to overcome the high computational costs of traditional methods such as HIVE-COTE while maintaining high accuracy. It scales linearly with the training set size and the time series length.

It adopts an ensemble strategy. Since the model's accuracy depends on the initial weights which are initialized randomly, it is an ensemble of five different Inception networks with same architecture but are initialized with different weights randomly. It helps to reduce variance and improve the stability and accuracy of the results.

Each network contains two different residual blocks, where each block is comprised of three inception modules. After all these inception blocks, a Global Average Pooling (GAP) layer is used to average the output multivariate time series. Finally, a fully-connected softmax layer is used.

We explain the structure of its core building block, inception module. Its first component is the bottleneck layer. It is a 1×1 convolution that is used to reduce the dimensionality of the input to decrease the computational cost and achieve generalization The second component is parallel convolutions. It applies different 1D convolutional filters of varying lengths simultaneously to capture patterns at multiple resolutions. It helps to identify local and global shape patterns. Then, a parallel max-pooling layer, followed by a bottleneck layer is added to reduce the dimensionality. Finally, the outputs of parallel convolutions and max-pooling are concatenated to output the final output. To note, to mitigate the vanishing gradient problem, shortcut linear connections (i.e., Residual Blocks) are used at every third inception module. It is the second fastest state-of-the-art TSC classifier after ROCKET [35].

TimesNet

TimesNet [36] is designed as a general purpose tool that it outperforms previous models in Long-term Forecasting, Short-term Forecasting, Anomaly Detection, Imputation, and Classification. The main idea is based on the observations that real-world time series are multi-periodic. It breaks down the complex pattern due to multi-periodic into two types of simpler patterns, namely Intraperiod-variation (short-term fluctations within a single period) and Interperiod-variation (long-term trends accross different periods).

The core building block is a novel component called TimesBlocks. It first use Fast Fourier Transform (FFT) to identify the top-k most significant frequencies (periods) in the data. For each identifed period, it expands 1D data into 2D matrix and so the standard 2D convolutional kernels can capture these two variations. This allow the models to benefit from advanced computer vision techniques that is designed to deal with 2D matrix. Finally, the extracted 2D features are reshaped back to 1D and combined using a weighted sum based on the amplitudes of the original frequencies.

By expanding 1D sequence to 2D matirx, it disentangles intricate periodic structures and hence improves the model's ability to capture complex temporal patterns.

Chapter 3

Scaling with Multiple Scaling Factors in Time Series Searching

Time series data are ubiquitous across many different fields. Many data mining tasks, such as classification, clustering, and motif finding, have been defined for time series data. They use similarity measures as core subroutines, making it crucial to design them to align with our intuitions. Dynamic Time Warping (DTW) is arguably the most prevailing distance measure. However, studies have shown that for certain data, another distance measure, namely Uniform Scaling (US), is equally crucial as DTW. DTW handles the local distortion, while US handles the global scaling. In addition, studies have demonstrated that combining DTW and US is necessary in some cases. Surprisingly, all existing studies assume a single scaling factor across the entire time series. A time series could consist of phases. Since each phase expresses at its own rate, using a single scaling factor is insufficient when comparing two time series that share similar phases with different expression rates. We introduce the first framework that accounts for multiple scaling factors, Piecewise Scaling Distance (PSD). It employs a distance measure as its base distance measure. Because the direct implementation is slow, we propose a constrained version that enforces constraints based on the allowed segment lengths derived from the given scaling factor bound. It also prevents pathological results. Two additional speedup techniques have been proposed, which achieve 8.40X to 178.72X speedup. We also demonstrate the use of a lower bound when DTW is employed as the base distance measure to accelerate the corresponding PSD computation. Moreover, we show that PSD segmentation results can improve the accuracy of other distance measures.

3.1 Background

To study the mechanism of a process, we take measurements. They are usually taken continuously by the sensors. It results in continuous values at discrete timestamps. For example, smartphones collect users' GPS data; ECG monitors measure patients' heart rate. The continuous measurements compose a time series. It is not hard to see why time series data are ubiquitous across many different fields. In GPS data, each data point consists of the user's latitude and longitude information. They are multivariate time series. In ECG data, each data point represents the amplitude of the patient's cardiac electrical activity. They are univariate time series. We focus on univariate time series.

Many data mining tasks are defined on time series data. For example, given a set of time series, we can perform clustering based on pairwise similarities between them. A classifier can be trained when categorical labels are available. Alternatively, given a single long time series, we identify recurring patterns for motif finding. In contrast, we identify abnormal subsequences for anomaly detection. Almost all tasks can be reduced to arguing the similarity between two time series. A good distance (similarity) measure can determine the success or failure of the algorithms built on it. The choice of an appropriate distance measure is particularly evident in classification. Studies show that a simple nearest-neighbor (1-NN) classifier with an appropriate distance measure is difficult to beat and can compete with more complex methods [37].

A time series is treated as a whole rather than as simply a collection of data points. The relationships between them are important. They constitute trends and shapes. Hence, the similarity measure is approximate-based rather than exact match-based [38]. Besides, different invariances should be allowed.

Dynamic Time Warping (DTW) is one of, if not the most common, similarity measures. DTW provides invariance to time distortion by aligning and measuring the similarity between the two series that are misaligned in time. However, it assumes that they are expressed at a similar global rate. Its performance is limited when global rates differ significantly. We often see this behavior in domains such as speech recognition, motion analysis, patient biomedical signals, and sensor data in the manufacturing industry.

Uniform Scaling (US) can achieve global scaling invariance by scaling the two series to the same length via interpolation, such as nearest-neighbor interpolation, before comparison, as shown in Figures 3.1, 3.2. It is reported that in some domains, such as gestures [39, 40] and music performance [41], the scaling is about

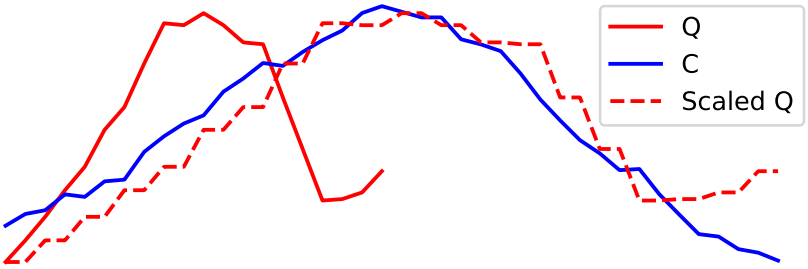


Figure 3.1: Applying nearest neighbor interpolation on Q results in scaled Q, which can better reflect the similarity between Q and C.

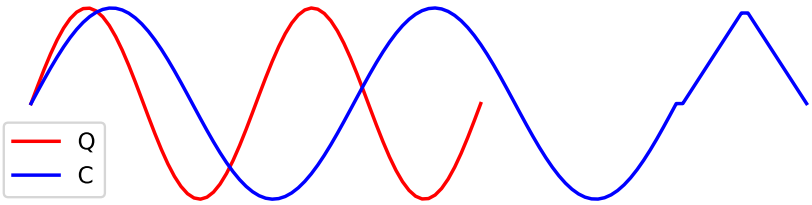


Figure 3.2: Q and C are in different rates. A stretching version of Q is similar to a prefix of C, but not the whole C.

10-15% (i.e., scaling factors: 1.1 to 1.15). The scaling factors are relatively small, since the nature of the music and the gait will change with significant scaling factors. However, in some other domains, we may encounter larger scaling factors. In bioinformatics, gene expression time series data could differ by a factor of 1.41 [42, 12].

DTW and US are used to achieve different kinds of invariance. DTW handles the local distortion, while US handles the global scaling. Furthermore, some studies show that for some data, the combination of US and DTW, namely USDTW, better reflects the similarity [43, 44, 45]. US is first applied to transform the two time series into the same length to eliminate the effect resulting from the different rates. Then, DTW, rather than ED, is applied to address the local distortion. USDTW is computationally more expensive than DTW because it involves the computation of the DTW between Q and different lengths of the prefixes of C, as shown in Figure 3.2. The different lengths correspond to different scaling factors.

The time series community heavily relies on the UCR Time Series Archive for benchmarking. Undoubtedly, it is a valuable resource. However, improvements made by a new idea, as verified by the datasets in the archive, are necessary but not sufficient to warrant the idea's success, since the datasets have been carefully contrived so they proxy real-world problems poorly [13, 20]. For example, in the

GunPoint dataset, the actor's actions (Gun/Point) were carefully prompted by a metronome that produced an audible cue every five seconds, resulting in a set of perfect five-second time series, each of which refers to a single action [13]. But in reality, several actions are monitored continuously rather than one at a time. Besides, the durations of the actions are similar but not identical. For example, in a department store, an announcement, such as today's closing time, would be repeated several times continuously, with each repetition varying slightly in duration. It is also common for the sampling strategy to change over time [46].

Existing studies mostly assume perfect segmentation. The time series in a dataset would have the same length, and each represents a single target action. [13] points out that in a more realistic setting, due to the improper segmentation in the data preparation phase, the time series could contain not just the target action but also other actions that were performed just before/after the target action. It proposed that some data on the prefix and suffix should be ignored by relaxing the endpoint constraints of the DTW formulation. But this study still assumes that there is only one target action in the time series. It cannot handle cases where the time series consists of multiple actions expressed at different rates. To achieve invariance for this kind of scaling effect resulting from multiple rates, rather than using a single scaling factor, it is beneficial to identify these different phases and use the appropriate scaling factors for these segments (pieces). We refer to this as piecewise scaling (PS).

Figure 3.3 shows the intuition of PS. $C(1:k)$ and Q share the same set of segments, but each has a different scaling. Multiple scaling factors must be used. It motivates us to design a new framework that considers applying a scaling factor on each of the phases as defined by dashed lines in the figure, during the comparison of two time series. To the best of our knowledge, no studies exist that address multiple scaling factors, but only one scaling factor [47, 43, 44, 45]. The proposed algorithm relaxes the assumption underlying US and USDTW, namely that only a single scaling factor is used to achieve x-axis scaling invariance.

Problem statement: Given two time series Q and C and a user-given number of pieces P, we would like to design an algorithm that minimizes the sum of the distances of the P aligned pairs of segments of Q and C, as illustrated in Figure 3.3. Within each pair, the segments are first rescaled to the same length, and the similarity between the rescaled segments is measured using a user-chosen base distance measure.

Note that the proposed algorithm is agnostic to the user-chosen base distance measure. The segmentation results themselves review the phase structure of the

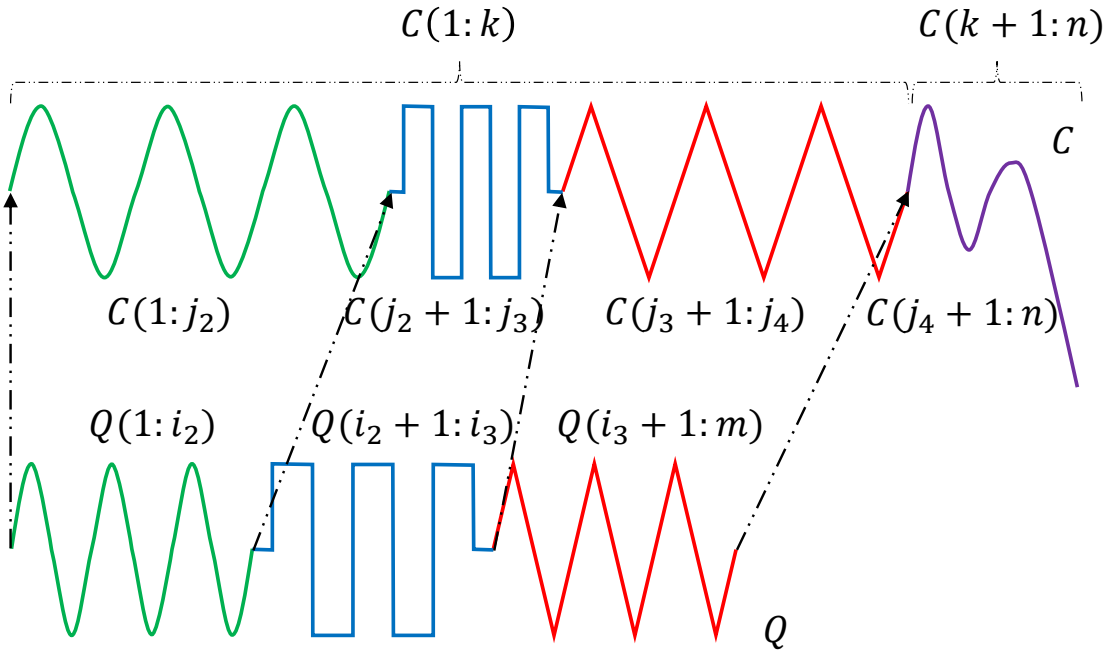


Figure 3.3: Intuition of piecewise scaling (PS).

time series. Moreover, they can be leveraged for downstream usage with other distance measures. First, we align the pairs according to the segmentation results. Then, we use the other distance measure M to compute the internal distance for each scaled pair, where M can achieve certain desired invariance within each pair. Finally, the sum of the distances is used to evaluate the similarity.

Our contributions are as follows:

- We demonstrate the necessity of piecewise scaling (PS) invariance and the first framework (via dynamic programming) to achieve it. It is agnostic to the base distance measure used. In particular, we focus on two instantiations of PSD using ED and DTW. They are called PSED and PSDTW, respectively.
- Similar to the local constraints in DTW, we propose a constrained version of PSD based on the allowed segment lengths to improve the efficiency and to prevent pathological results.
- Two other speedup techniques have been proposed, namely parallel computing and early abandoning. Besides, we demonstrate the use of a lower bound to further speed up the computation of PSDTW.
- We demonstrate that the segmentation results returned by PSD can improve the accuracy of other distance measures.

The rest of this paper is structured as follows. We present related work in Section 4.2 and preliminaries in Section 4.3. Section 5.2 introduces PSD, its constrained version, and speedup techniques. It is experimentally demonstrated in Section 3.5 for the problem of querying. In Section 5.5, we conclude this study with some future work.

3.2 Related Work

This study focuses on distance measures of time series. For the overall review of time series, we direct the readers to [38, 48] for a more comprehensive understanding of this field.

For many tasks, having appropriate distance measures to achieve different invariances that align with our intuition for the domains we work with is essential. For example, in polarimetric radar research, researchers employ roll-invariant features to maintain detection accuracy despite distortions introduced by sensor effects, such as varying grazing angles [49]. It improves the acquisition of fully polarimetric information, especially in a joint-domain processing framework [50, 51].

One well-known distance measure is Dynamic Time Warping (DTW). It is initially designed for speech analysis [3]. However, DTW is computationally expensive. Lower bounds are used to speed up time series similarity search by admissibly pruning unpromising candidates. One of the popular exact lower bounds of DTW is $\mathrm{LB}_{\mathrm{Keogh}}$. [52] improves the scalability of DTW for searching subsequences by introducing a subsequence search suite based on their four novel ideas, namely the UCR suite. For an overall review of lower bounds, we refer readers to [53, 54]. There is an approximate algorithm that approximates DTW with high accuracy while drastically cutting down the time and space requirements [55].

Uniform Scaling (US) has been shown to be a critical invariance in domains such as motion capture. [47] demonstrated that DTW is insufficient for handling global scaling effects, and that identifying DTW is not the solution to achieve this kind of invariance. [56] shows the importance of US in motif discovery. The authors show that meaningful motifs often suffer from a global scaling effect, causing standard motif finding algorithms to miss them completely.

To the best of our knowledge, three studies analyze the combination of US and DTW, namely USDTW. It was first proposed by [43]. It extended $\mathrm{LB}_{\mathrm{Keogh}}$ to bound the USDTW. However, the extended $\mathrm{LB}_{\mathrm{Keogh}}$ is still too loose with invariance to large amounts of uniform scaling. [44] and its follow-up study [45]

proposed a new lower bound [1], namely LB_{Shen}, which has been shown to be tighter than LB_{Keogh} on USDTW.

Recently, some DTW variants have been proposed. [57] proposes a variation called Angle-distance Penalized Metric DTW (APMDTW). It makes use of directional information to cope with the cases where subtle variations and trajectory orientation carry semantic significance in the similarity measure by introducing a piecewise logarithmic transformation and incorporating a parameterized angle-distance penalty into the cost function. [58] addresses the efficiency problem of using DTW in subsequence matching in a data stream. It explores the use of piecewise approximation methods to improve subsequence matching. In detail, they propose a method to segment a data stream adaptively and extract Chebyshev features to approximate the original data. They show that this method achieves a comparable accuracy of the original DTW with up to a three-order-of-magnitude speedup. [59] also addresses the efficiency problem by proposing an abstract-adaptive PAR-DTW. It computes DTW in a piecewise abstract representation (PAR) space. The dimension of the original data has been reduced. Hence, it accelerates DTW at the cost of losing precision. Instead of using globally uniform pointwise measures, it learns pointwise measures for each pair of abstract clusters to put more weight on the more discriminative features to achieve precision comparable to the original DTW while reducing computational complexity.

To our surprise, despite a fruitful discussion of DTW, US, and USDTW, and different variations of DTW for better accuracy or efficiency, no study has proposed a distance measure or framework capable of handling scaling effects across multiple scaling factors. This is precisely what we will address in this study.

3.3 Preliminaries

We refer to time as the contextual attribute because it provides the context for the measurements to be made. We refer to the measurements as the behavioral attributes. Time series are multivariate when more than one behavioral attribute is present. Otherwise, it is called univariate. We focus on the univariate case.

Definition 9 (Time Series)**.** A time series $T = t_1, t_2, ..., t_n$ is a sequence of real-valued numbers with length $= n$.

When two time series are involved in the discussion, we denote them as Q (Query) and C (Candidate), with lengths m and n, respectively. Since Q is the

[1] It is denoted as LB_{New} in the original study. We rename it to prevent any confusion.

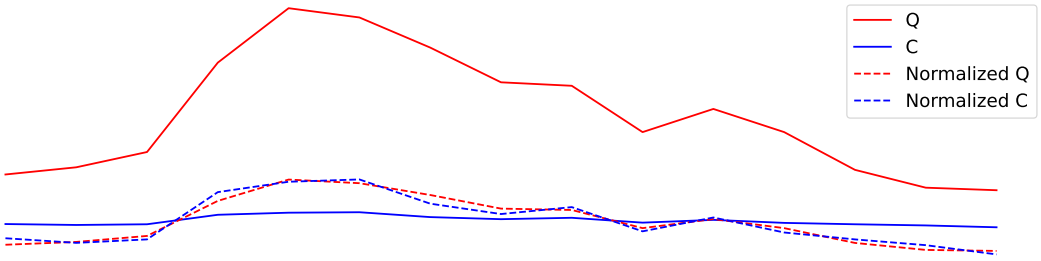


Figure 3.4: Z-normalization. Resulting time series would have mean = 0 and std = 1.

query sequence, it is not longer than C (i.e., $m \leq n$). The requirement of "$m \leq n$" is a natural setting. In "Query by Content", a user is going to search for a candidate in the database from a user-input query in which the query may only contain partial information of the target candidate. For example, a user often wants to find a song or tune that is lingering in their head by humming a part but not whole of the tune [60]. We are interested in a segment or subsequence of a time series.

Definition 10 (Subsequence)**.** A subsequence $T(i:j)$ of a time series T is a shorter time series that starts from position i and ends at position j with length $= j - i + 1$. Both ends are inclusive. Formally, $T(i:j) = t_i, t_{i+1}, ..., t_j$, $1 \leq i \leq j \leq n$.

We call $T(1:j)$ the prefix of T of length j.

Before comparison, we need to standardize or normalize them. A common way is Z-Normalization, which is $T = (T - \text{mean}(T))/\,\text{std}(T)$, as shown in Figure 3.4. The most fundamental distance measure is the Euclidean Distance (ED).

3.3.1 Euclidean Distance (ED)

Given two points on a plane, it is intuitive to define the distance between them as the length of the line segment between them. This idea extends to the case of n-dimensions in time series with length n.

Definition 11 (Euclidean Distance (ED))**.** Given two series Q and C both with length n, the Euclidean Distance between them is defined as:

$$\text{ED}(Q, C) = \sqrt{\sum_{i=1}^{n} (q_i - c_i)^2} \tag{3.1}$$

A square root is usually involved in the computation of distance measures. It is a monotonic function. Since it does not change the relative ranking of the results,

we can omit the square root operation for simplicity and optimization. ED aligns the entries between two series in a one-to-one manner. Two similar series will have a large distance under ED if they are not aligned well in the time dimension. ED cannot handle the local distortion along the time axis because warping in the alignment is not allowed. A standard distance measure that provides warping invariance is called Dynamic Time Warping (DTW).

3.3.2 Dynamic Time Warping (DTW)

Dynamic Time Warping (DTW) is an algorithm that measures similarity between two time series while accounting for the local distortion. DTW aligns the two series by nonlinearly warping the time axis to minimize the final cumulative distance.

Given two time series, Q and C, we first construct an m by n distance matrix M. The origin (1, 1) is set at the bottom-left element of M. The (i, j) element of M contains the distance $\mathrm{d}(q_i, c_j)$ between points q_i and c_j. The local distance $\mathrm{d}(\cdot, \cdot)$ is usually calculated by $(q_i - c_j)^2$. Each (i, j) element refers to an alignment or mapping of the two points. A contiguous set of such elements forms a warping path W, which represents the non-linear alignment of Q and C. $W = w_1, w_2, ..., w_K$ in which $w_k = (i, j)_k$ represents the mapping between q_i in Q and c_j in C, where $\max(m, n) \leq K \leq m+n-1$. "$\max(m, n) \leq K$" because the alignment of two series must include every point in both Q and C. "$K \leq m + n - 1$" because the longest warping path is either the "concatenation of the bottom row and the rightmost column" (with the bottom-right cell being overlapped) or the "concatenation of the leftmost column and the top row" (with the top-left cell being overlapped).

The warping path W is typically subject to the following three constraints.

- Boundary conditions: $w_1 = (1, 1)$ and $w_K = (m, n)$. The first (last) point of Q must map to that of C.
- Continuity: Given $w_k = (i, j)$ and $w_{k-1} = (i', j')$, $i - i' \leq 1$ and $j - j' \leq 1$. An entry in the warping path W is adjacent to its one-step previous entry.
- Monotonicity: Given $w_k = (i, j)$ and $w_{k-1} = (i', j')$, $i - i' \geq 0$ and $j - j' \geq 0$. The warping path W does not go back.

We denote W^* as a set of all allowed possible paths.

Definition 12 (Dynamic Time Warping (DTW) [3])**.** DTW returns the minimum warping cost:

$$\mathrm{DTW}(Q, C) = \min_{W \in W^*} \sum_{k=1}^{|W|} w_k \tag{3.2}$$

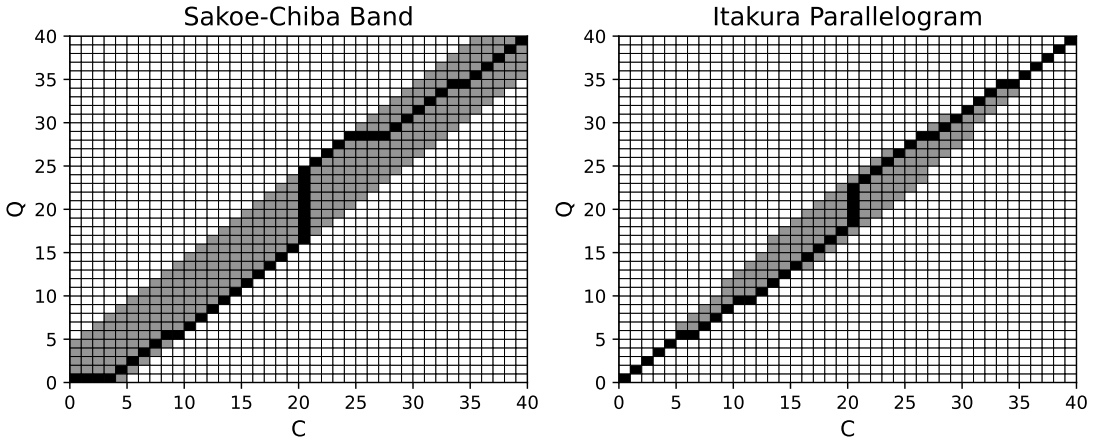


Figure 3.5: Visualization of the accumulated cost matrix D with local constraints. The black cells form the warping path.

To find the minimum warping cost and its corresponding warping path, we can use dynamic programming (DP) to evaluate the following recurrence.

$$\begin{aligned} D(i,j) = \mathrm{d}(q_i, c_j) \\ + \min \begin{cases} D(i-1, j-1), \\ D(i-1, j), \\ D(i, j-1) \end{cases} \end{aligned} \tag{3.3}$$

It can be solved by building the accumulated cost matrix D, where the y-axis refers to Q, and the x-axis refers to C. The base cases, which are the first row and the first column, are defined as $D(1,j) = \sum_{k=1}^{j} \mathrm{d}(q_1, c_k), j \in [1,n]$ and $D(i,1) = \sum_{k=1}^{i} d(q_k, c_1), i \in [1,m]$. After we have initialized the base cases, we can fill up D starting from the bottom up. There are $m \times n$ entries in D. It takes $\mathcal{O}(mn)$ to fill it up. Once D is built, we can find the path corresponding to this minimum cost by simple backtracking from the end cell $D(m,n)$ to the origin cell $D(1,1)$.

Some constraints have been proposed to prevent pathological warping paths with the aim of accuracy and efficiency. Two of the most popular are Sakoe-Chiba Band [3] and Itakura Parallelogram [4]. They only allow the warping paths to pass through the allowed region, as shown in Figure 3.5, by setting the cells outside this region in D to have ∞ accumulated distance cost. [61] suggested that these constraints can be viewed as constraints on the warping path entry $w_k = (i,j)_k$. It

represents these constraints as inequalities applying to the indices i and j locally, without depending on the main diagonal in D. In the Sakoe-Chiba Band, the constraints can be represented as $j - r \leq i \leq j + r$, where r is an integer. It is sometimes specified as a fraction (percentage) of the longer time series length to ensure length invariance. For clarity, we assume r is an integer unless specified otherwise. This means that q_i can only align with c_j if their indices differ by at most r. Since r defines the maximum allowed difference between the mapping indices, $|n - m| \leq r$, to ensure that the end points of Q and C can map. In the Itakura Parallelogram, r is a function of i rather than a constant value.

ED can be seen as a special case of DTW where the warping path is fixed to be diagonal. q_i aligns to c_i for every i (i.e., $r = 0$). DTW minimizes over all possible warping paths, and the warping path of ED is one of them. Because of the band, we only need to fill up the cells within the band. The complexity is $\mathcal{O}(\#\_of\_cells\_inside)$. In the case of the Sakoe-Chiba Band, the band has a constant width $w = 2r+1$. The complexity becomes $\mathcal{O}(w \times length\_of\_diagonal) = \mathcal{O}(r \times n)$. The constrained DTW is denoted as DTW_r.

3.3.3 Uniform Scaling (US)

In US, we compare the whole sequence of Q to a prefix of C, as shown in Figure 3.2. The two compared sequences are scaled to the same length via interpolation before ED is applied. A common interpolation method is nearest neighbor interpolation.

Definition 13 (Nearest Neighbor Interpolation)**.** Given a time series T of length n and an integer L , Nearest Neighbor Interpolation scales T into T^L as follows:

$$T_j^L = T_{\lceil n(j/L) \rceil)} \quad where\ 1 \leq j \leq L \tag{3.4}$$

We can scale up or down a given series using Equation 3.4, as shown in Example 1.

Example 1. Given a series $T = 1, 2, \cdots, 6$ with length $n = 6$. Let $T(1:4)$ be the prefix of T of length $k = 4$ (i.e., $T(1:4) = 1, 2, 3, 4$). Given an integer $L = 8$, we compute $T(1:4)^8$ as follows.

$$\begin{aligned} T(1:4)^8 &= T_{\lceil 4(1/8) \rceil}, T_{\lceil 4(2/8) \rceil}, T_{\lceil 4(3/8) \rceil}, \ldots, T_{\lceil 4(8/8) \rceil} \\ &= T_1, T_1, T_2, \ldots, T_4 \\ &= 1, 1, 2, \ldots, 4. \end{aligned}$$

When $L > k$, T is said to be stretched. When $L < k$, T is said to be shrunk.

Definition 14 (Uniform Scaling (US) [47])**.** Given two series Q and C, of length m and n respectively, and a scaling factor bound l, where $l \geq 1$. Let $C(1:k)$ be the prefix of C, where $\lceil m/l \rceil \leq k \leq \min(\lfloor lm \rfloor, n)$, and $C(1:k)^L$ be a rescaled version of $C(1:k)$ with length L, where $L = \min(\lfloor lm \rfloor, n)$. L is called the alignment factor. $\min(\lfloor lm \rfloor, n)$ is the largest alignment factor.

$$\mathrm{US}(Q, C, l, L) = \min_{k=\lceil m/l \rceil}^{\min(\lfloor lm \rfloor, n)} \mathrm{ED}(Q^L, C(1:k)^L) \tag{3.5}$$

L is set as the largest alignment factor [45] to ensure all the points in Q and $C(1:k)$ are preserved during interpolation because of up-sampling, and the scaled version of all the prefixes is going to have the same length (i.e., L), for fair comparison, since comparison between two longer time series generally results in a larger distance measure.

Through Equation 3.5, we find the minimum value and the corresponding argument (i.e., k) by checking the minimum value of the ED function from $\lceil m/l \rceil$ to $\min(\lfloor lm \rfloor, n)$. The scaling factor is defined by the argument minimum value of k. The smaller k is, the more we need to "stretch" $C(1:k)$ for Q to compare with $C(1:k)$, which is m/k times. The tail part of C (i.e., $C(k+1:n)$) is ignored, as shown in Figure 3.2.

Consider a time series database D comprising a set of candidate instances, and a single query series Q. The search task aims to retrieve the most similar instance (or top-k similar instances) from D to Q. We maintain Q as a fixed reference and extract only the prefixes from each instance in D for comparison. Furthermore, to simplify notation, we apply scaling exclusively to the instances in D, leaving Q unscaled.

3.3.4 Uniform Scaling & Dynamic Time Warping (USDTW)

Uniform Scaling & Dynamic Time Warping (USDTW) measures the similarity by applying scaling with an appropriate scaling factor on C and then applying DTW.

Definition 15 (Uniform Scaling & Dynamic Time Warping (USDTW) [45])**.** With the same notations defined in Definition 14,

$$\begin{aligned}&\mathrm{USDTW}_r(Q, C, l, L) = \\ &\qquad \min_{k=\lceil m/l \rceil}^{\min(\lfloor lm \rfloor, n)} \mathrm{DTW}_r(Q^L, C(1:k)^L)\end{aligned} \tag{3.6}$$

where r is the DTW constraint parameter.

We replace the ED function in Equation 3.5 by DTW function to form Equation 3.6.

As mentioned in Section 4.1, there may exist more than one scaling factor. Hence, both the existing distance measures, US and USDTW, which are designed to handle only one scaling factor, are insufficient. This motivates us to design a new framework of distance measures.

3.3.5 Lower Bounds for DTW and USDTW

We first introduce the concept of lower bounds and explain how they can benefit searching. DTW is computationally more expensive than ED. DTW has quadratic complexity. This complexity poses a challenge for similarity search. The most common approach is to compute a lower bound $\mathrm{LB}(Q, C)$ of the real distance, which is computationally cheap and tight. We can use this lower bound to filter out unpromising candidates and perform the expensive distance computation only on the small set of promising candidates. In searching, we would keep track of the best-so-far distance bsf between the query Q and the testing candidates. When testing a new candidate C, we first compute the $\mathrm{LB}(Q, C)$. We only compute the actual DTW when $\mathrm{LB}(Q, C) \leq \mathrm{bsf}$.

We introduce some common lower bounds.

Kim Lower Bound [62]: $\mathrm{LB}_{\mathrm{Kim}}$ is a simple and fast lower bound of DTW. The complexity is $\mathcal{O}(n)$. It uses the four features in Q and C. We denote t_{-1} as the last point and $t_{\max}$ ($t_{\min}$) as the maximum (minimum) point in time series T.

$$\mathrm{LB}_{\mathrm{Kim}} = \max \begin{cases} \mathrm{d}(q_1, c_1) \\ \mathrm{d}(q_{-1}, c_{-1}) \\ \mathrm{d}(q_{\max}, c_{\max}) \\ \mathrm{d}(q_{\min}, c_{\min)} \end{cases} \tag{3.7}$$

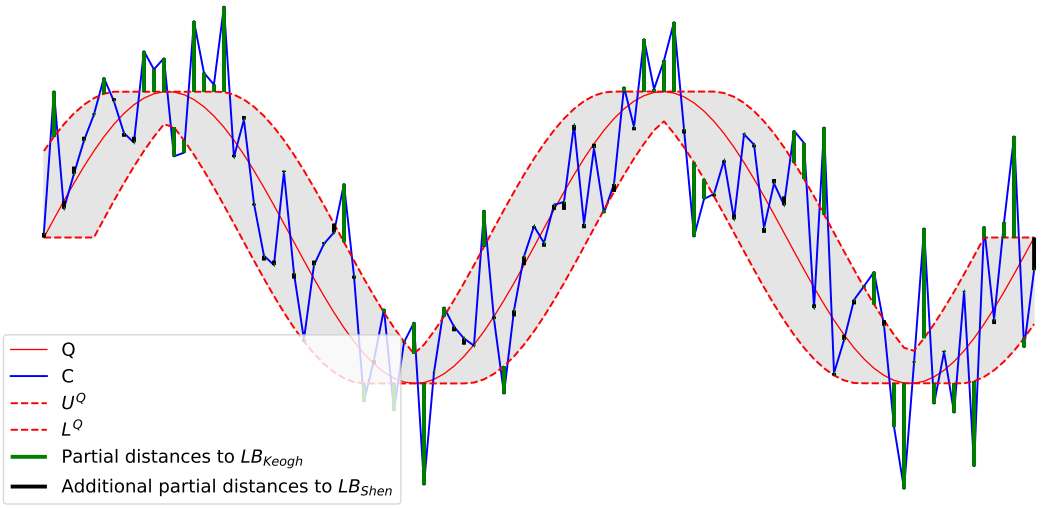


Figure 3.6: Visualization of $\mathrm{LB}_{\mathrm{Keogh}}$ and $\mathrm{LB}_{\mathrm{Shen}}$.

$\mathrm{d}(\cdot,\cdot)$ refers to the local distance used in the point alignment. The first two lines come from the boundary condition in DTW. The alignment between the first pair of points and the last pair of points must contribute to the accumulated sum of DTW. Each point in Q must align with some point in C, and vice versa. Each alignment contributes a local distance to the final sum. The minimum possible local distance for the alignment of $q_{\max}$ would be the one aligned with $c_{\max}$. The same applies to the last line in the equation.

There is a simplification of $\mathrm{LB}_{\mathrm{Kim}}$. Only the first and last pair are used in the lower bound computation. In the normalized time series, the third and fourth rows in Equation 3.7 should have small values [52]. Ignoring them can improve the complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$. The simplified lower bound is $\mathrm{LB}_{\mathrm{KimFL}} = \mathrm{d}(q_1, c_1) + \mathrm{d}(q_{-1}, c_{-1})$. "FL" stands for First-Last.

Keogh Lower Bound [61]: $\mathrm{LB}_{\mathrm{Keogh}}$ builds the upper U and lower envelopes L of one of the time series out of the two compared series. Usually, the envelopes are constructed for Q rather than C, since we compare the same Q against many candidates. Otherwise, we need to build the envelopes for each candidate instead [52].

To the best of our knowledge, they are the first to interpret the constraint as a restriction on the indices of the warping path $w_k = (i, j)_k$ such that $j - r \leq i \leq j + r$, where r defines the allowable deviation of alignment between q_i and c_j. For ease of exposition, we focus on the most used constraint in the literature, which is the Sakoe-Chiba Band [63]. Two sequences are constructed for Q, namely the upper

and lower envelopes (U^Q and L^Q) of Q as shown in Figure 3.6. For each q_i, we would assign a bounding envelope of q_i based on its index i as follows.

$$\begin{aligned} U_i^Q &= \max(q_{\max(1,i-r)} : q_{\min(i+r,m)}) \\ L_i^Q &= \min(q_{\max(1,i-r)} : q_{\min(i+r,m)}) \end{aligned} \tag{3.8}$$

$\max(1, \cdot)$ and $\min(\cdot, m)$ are used for handling the boundary cases. U^Q and L^Q together form a bounding envelope that encloses the original Q, it is the grey region in the figure. For each c_j, either (c_j, U_j^Q) or (c_j, L_j^Q) corresponds to the possible alignment that contributes the minimum distances if c_j falls outside the envelope. Herein, the lower bound is the sum of these distances, as shown in the following equation.

$$\mathrm{LB}_{\mathrm{Keogh}}(Q, C) = \sum_{j=1}^{n} \begin{cases} d(c_j, U_j^Q) & \text{if } c_j > U_j^Q \\ d(c_j, L_j^Q) & \text{if } c_j < L_j^Q \\ 0 & \text{otherwise} \end{cases} \tag{3.9}$$

Visually, these distances are the distances between c_j outside the envelope and the vertically corresponding points on the envelope. The distances are the green bars in the figure. Equation 3.9 returns the sum of the green bars.

Shen Lower Bound [44, 45]: It leverages the boundary and continuity conditions to create a lower bound of DTW. It can be used to lower-bound the USDTW with slight modifications. The intuition is to find the minimum possible alignment of each c_j with a point in Q, subject to the local constraint r from DTW and the global constraint l from US.

We will first introduce $\mathrm{LB}_{\mathrm{Shen}}$ for DTW. Given the candidate sequence C with length n, for each c_j, we create its possible reach $\mathbb{q}_j$ in Q under DTW as in Equation 3.10. The elements $\mathbb{q}_j$ form an indexed collection $\mathbb{Q} = (\mathbb{q}_1, \mathbb{q}_2, \ldots, \mathbb{q}_{\min(\lfloor lm \rfloor, n)})$.

$$\mathbb{q}_j = (q_{\max(1,j-r)}, \ldots, q_{\min(j+r,m)}) \tag{3.10}$$

We define $\delta(x, Y) = \min_{y \in Y} d(x, y)$. The possible minimum cost contributed by the alignment of c_j and a point in Q to the final sum would be $\delta(c_j, \mathbb{q}_j)$. The lower bound $\mathrm{LB}_{\mathrm{Shen}}$ is defined as:

$$\mathrm{LB}_{\mathrm{Shen}}(Q, C) = \mathrm{d}(c_1, q_1) + \sum_{j=2}^{n-1} \delta(c_j, \mathbb{q}_j) + \mathrm{d}(c_n, q_m) \tag{3.11}$$

We direct the reader to [44] for the formal proof, while an intuition of the proof is presented here. There are three items on the right-hand side of Equation 3.11. The first and the last items are from the boundary condition. The continuity requirement ensures that each c_i is involved in at least one alignment. The middle item returns the possible minimum distances contributed by each c_j, where $2 \leq j \leq n-1$. To note, it is obvious that LB_{Shen} in Equation 3.11 is tighter when we use the first pair of points and the last pair of points instead of doing the middle summation all from $j = 1$ to n. It is because the distance contributed by the first pair (last pair) must be greater than $\delta(c_1, q_1)$ ($\delta(c_n, q_n)$).

It is proven that it is tighter than LB_{Keogh} [44]. It is shown in Figure 3.6 visually. The black bars refer to the additional partial distances on top of LB_{Keogh}. We will give an intuitive proof here. Both LB_{Keogh} and LB_{Shen} compute the lower bounds by summing the local distances resulting from the alignment of each c_j, which is guaranteed not to be greater than the partial distance contributed by the real alignment, which we only know until we compute the exact distance measure. In LB_{Keogh}, if this c_j falls outside the envelope, this local distance is the vertical distance between c_j and the envelope. If c_j falls inside, this local distance would be 0. For those points outside the envelope, the local distances for c_j of LB_{Keogh} and LB_{Shen} are the same. But LB_{Shen} aims to return the minimum possible partial distance for each c_j, even within the envelope. Hence, $\text{LB}_{\text{Keogh}} \leq \text{LB}_{\text{Shen}}$.

In order to compute Equation 3.11 efficiently, we first sort sequences q_j and the resulting sorted sequences are denoted as $\tilde{q}_j$. The sorted sequences allow us to do a binary search when we are calculating $\delta(c_j, \tilde{q}_j)$ in contrast to $\delta(c_j, q_j)$. The sorting only needs to be done once because we are testing the same Q with different candidates.

It can be extended to the USDTW case [44, 45]. The possible reach now is not only defined by r, but also by the scaling factor bound l.

$$q_j = (q_{\max(1,\lceil j/l \rceil - r)}, \ldots, q_{\min(\lfloor jl \rfloor + r, m)}) \tag{3.12}$$

[44] proves the following theorem to allow us to consider the lower bound between each prefix of C and Q without the scaling up of each prefix of C (i.e., $C(1:k)^L$) and that of Q (i.e., Q^L).

Theorem 16. *For any* $\lceil m/l \rceil \leq k \leq \min(\lfloor lm \rfloor, n)$, $\text{DTW}_r(Q^{\min(\lfloor lm \rfloor, n)}, C(1:k)^{\min(\lfloor lm \rfloor, n)})$ *is always lower bounded by* $\sum_{j=1}^{k} \delta(c_j, q_j)$.

Recall that USDTW calculates DTW distances between each scaled prefix of C to the scaled Q, and outputs the minimum DTW distance, as in Equation 3.6. The

incremental nature of the lower bound frees us to calculate the lower bound for each DTW distance from scratch. This theorem allows us to first calculate the lower bound of the shortest prefix $C(1:\lceil m/l \rceil)$ of C and Q, and then incrementally calculate the lower bound of the longer prefix with length from $\lceil m/l \rceil + 1$ to $\min(\lfloor lm \rfloor, n)$ by adding on each $\delta(c_j, \mathbb{q}_j)$. To note, it also means that if $\mathrm{LB}_{\mathrm{Shen}}(Q, C(1:k))$ is larger than a value, namely bsf, $\mathrm{LB}_{\mathrm{Shen}}(Q, C(1:k'))$, where $k' > k$, would also be larger than bsf.

The above analysis can also apply to the case of US distance. We only need to define the corresponding reaches as follows:

$$\mathbb{q}_j = (q_{\max(1,\lceil j/l \rceil)}, \ldots, q_{\min(\lfloor jl \rfloor, m)}) \tag{3.13}$$

3.4 Piecewise Scaling Distance

The motivation stems from the limitations of US and USDTW. They assume that the relationship between Q and C is governed by only a single, global scaling factor. This assumption fails when applied to multi-rate data, where different phases of the time series express at different rates.

Note that existing studies [47, 52] can find all scaling factors, defined by the chosen prefix of C, ranging from $\lceil m/l \rceil$ to $min(\lfloor lm \rfloor, n)$. However, they can use only one of them in the scaling. Consider the following illustrative example in ASCII text [52], where character repetition represents the duration of spoken phonemes, and space indicates a pause:

- "time series 20 25" and "time series 20 25". Here, the Hamming distance (the discrete analogue of ED) fails due to misalignment in the underlined locations, but the string edit distance (the discrete analogue of DTW) can resolve it.

- "time sseerriiss 222000222555".This sequence exhibits three distinct phases with scaling factors of 1, 2, and 3, respectively. The corresponding invariance cannot be achieved by DTW or US. US would enforce a single compromised global scaling factor instead.

In query-by-content scenarios, such as query-by-humming or gesture retrieval, the query is generated by a human. Humans do not maintain a consistent rate for each phase. For example, people rush through a familiar sequence (e.g., a sentence or a motion) but slow down for a new or complex sequence. It is commonly

observed in a piece of music performed by a beginner, where the tempo is not kept uniform.

We introduce a novel distance measure framework, termed Piecewise Scaling Distance (PSD). It addresses the local scaling effect within each phase by employing a scaling factor to each phase, instead of using a single scaling factor for the whole time series. It releases the basic constraint or assumption made in US and USDTW. PSD employs an existing distance measure, such as ED or DTW, to quantify the similarity of aligned segment pairs. While the PSD framework is agnostic to the underlying distance measure, this study focuses on its two fundamental instantiations:

- PSED, which employs ED to compute the similarity of aligned segment pairs.
- PSDTW, which employs DTW to compute the similarity of aligned segment pairs.

In the formulation that follows, we use PSDTW as the running example. PSDTW generalizes PSED. The PSED formulation can be trivially derived from it.

3.4.1 Piecewise Scaling & Dynamic Time Warping (PSDTW)

Problem formulation: To simplify the discussion, we focus on the comparison between Q and the entire sequence C, rather than a prefix of C. Note that this formulation can be generalized to prefix matching as in Definition 14 and Definition 15.

Given two sequences Q and C, where Q is not longer than C (i.e., $|Q| = m \leq |C| = n$) and the number of segments or pieces P allowed, our goal is to segmentalize both Q and C into P contiguous segments automatically in a way that minimize the total sum of DTW distance of aligned segment pairs with interpolation.

Definition 17 (Piecewise Scaling & Dynamic Time Warping (PSDTW))**.** With the same notations defined in Definition 15,

$$\begin{aligned}&\mathrm{PSDTW}_r(Q, C, l, L, P) = \\ &\min_{\substack{i_1<i_2<\cdots<i_{P+1} \\ j_1<j_2<\cdots<j_{P+1} \\ \frac{1}{l}\le\frac{i_{p+1}-i_p}{j_{p+1}-j_p}\le l}} \sum_{p=1}^{P} \mathrm{DTW}_r(Q(i_p+1:i_{p+1})^L, \\ &\qquad\qquad C(j_p+1:j_{p+1})^L)\end{aligned} \tag{3.14}$$

where $i_1 = 0$, $i_{P+1} = m$, $j_1 = 0$, $j_{P+1} = n$. The setting of L will be discussed later. Essentially, L needs to be at least the longest length of all considered segments to preserve all the points by up-sampling.

We refer to Figure 3.3 to clarify Equation 3.14. Given two sequences Q and C, with the aid of different colors and the dashed lines, we observe that Q consists of three segments while C consists of four segments, with the first three segments of C similar to those of Q. They form three segment pairs. The scaling factor used in each segment pair is determined from the length of the two subsequences involved, i.e., $(i_{p+1} - i_p)/(j_{p+1} - j_p)$. These three parts have different scaling factors. For example, the first (second) part in C is the stretched (compressed) version of that in Q.

Equation 3.14 can be formulated in a recurrence relation as follows. Let $D[i, j, p]$ be the minimum cost to align the first i points in Q (i.e, $Q(1 : i)$) with the first j points in C (i.e., $C(1 : j)$) using exactly p segments:

$$\begin{aligned}D[i,j,p] = \min_{\substack{i'<i \\ j'<j \\ \frac{1}{l}\le\frac{i-i'}{j-j'}\le l}} \Big\{ &D[i', j', p-1] \\ &+ \mathrm{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L) \Big\}\end{aligned} \tag{3.15}$$

Our goal is $D[m, n, P]$. Equation 3.15 can be solved exactly by dynamic programming (DP). The base case is $D[0, 0, 0] = 0$. It refers to the zero cost to align the first 0 point (i.e., the empty prefix) of Q with that of C. Other cells in D are first initialized with ∞. They are calculated using a bottom-up approach via Equation 3.15.

Algorithm 1 Naive PSDTW

Input: Query series Q, Candidate series C, DTW constraint parameter r (in a fraction), Number of pieces P, Alignment factor L

Output: Distance Matrix D of size $(m+1) \times (n+1) \times (P+1)$

1: Initialize D with ∞
2: $D[0,0,0] \leftarrow 0$
3: **for** $p \leftarrow 1$ **to** P **do**
4: **for** $i \leftarrow 1$ **to** m **do**
5: **for** $j \leftarrow 1$ **to** n **do**
6: $D[i,j,p] \leftarrow \min_{i'<i, j'<j} \{D[i', j', p-1] + \mathrm{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L)\}$
7: **return** $D[m,n,P]$

Algorithm 2 Line 6 in Algorithm 1

1: **for** $i' \leftarrow 0$ **to** $i-1$ **do**
2: **for** $j' \leftarrow 0$ **to** $j-1$ **do**
3: $dist_{\text{prev}} \leftarrow D[i', j', p-1]$
4: $dist_{\text{seg}} \leftarrow \mathrm{DTW}_r(Q(i'+1:i)^L, C(j'+1:j)^L)$
5: $D[i,j,p] \leftarrow \min(D[i,j,p],\ dist_{\text{prev}} + dist_{\text{seg}})$

Naive PSDTW

For ease of explanation, we first introduce a straightforward implementation in DP that does not consider the scaling-factor bound l, as shown in Algorithm 1. Line 6 is achieved by looping all the previous indices of the current i and j as in Algorithm 2.

We explain the lines in Algorithm 2. Line 3 retrieves the accumulated distance cost from the beginning up to the endpoints (i', j'), and saves it as $dist_{\text{prev}}$. Line 4 considers the current aligned segment pair, which consists of $Q(i'+1:i)$ and $C(j'+1, j)$, and they are interpolated to the length L. The DTW distance of this pair is calculated and saved as $dist_{\text{seg}}$.

We now analyze the time complexity of Algorithm 1. There are Pmn entries in D. The *min* operator in line 6 takes $\mathcal{O}(mn)$. Hence, the time complexity of Algorithm 1 is $\mathcal{O}(Pm^2n^2) = \mathcal{O}(Pn^4)$, multiplied by the running time of the DTW. It is slow, which prevents us from using it in practice. To note, we use r in a fraction instead of a fixed integer here. This allows the DTW deviation tolerance to scale adaptively with pieces of varying lengths.

3.4.2 Speedup Techniques

Length constraints of the segment

Algorithm 3 Initialization of PSDTW

1: $L^Q_{\text{gmin}} \leftarrow \lceil (m/P)/\sqrt{l} \rceil, \quad L^Q_{\text{gmax}} \leftarrow \lfloor (m/P)\sqrt{l} \rfloor$ ▷ "g" refers to "global".
2: $L^C_{\text{gmin}} \leftarrow \lceil (n/P)/\sqrt{l} \rceil, \quad L^C_{\text{gmax}} \leftarrow \lfloor (n/P)\sqrt{l} \rfloor$
3: $L = \max(L^Q_{\text{gmax}}, L^C_{\text{gmax}})$
4: Initialize D of size $(m+1) \times (n+1) \times (P+1)$ with ∞
5: $D[0,0,0] \leftarrow 0$

Table 3.1: Notation for the algorithms.

Symbol	Description
Q	Query series with length m
C	Candidate series with length n
l	Scaling factor bound
P	Number of pieces
L	Alignment factor
r	DTW constraint parameter (in a fraction)
r'	DTW constraint parameter (in an integer)
D	Distance Matrix of size $(m+1)\times$ $(n+1) \times (P+1)$
L^Q_{gmin} (L^C_{gmin})	Global minimum length of a segment in Q (C)
L^Q_{gmax} (L^C_{gmax})	Global maximum length of a segment in Q (C)
L^C_{min}	Local minimum length of a segment in C
L^C_{max}	Local maximum length of a segment in C
L^Q (L^C)	Length of the considered segment in Q (C)
Q' (C')	Reversed considered segment in Q (C)

A way to reduce complexity and to prevent pathological segment pairs is to limit the possible segment lengths that are considered by constraining the minimum and maximum lengths of segments. It is similar to the constrained DTW, in which we limit the search space of the warping path as in Figure 3.5. It is shown in Algorithm 4. The notations of the algorithms are listed in Table 3.1. Lines 11–14, 25–26, and 29–44 show the other speedup techniques, which will be explained later.

We initialize in Algorithm 3. For a given number of segments P, the expected length of each segment in Q and C would be m/P and n/P, respectively. For Q, we set the minimum possible segment length L^Q_{gmin} to be $\lceil (m/P)/\sqrt{l} \rceil$ and the maximum possible length L^Q_{gmax} to be $\lfloor (m/P)\sqrt{l} \rfloor$ in line 1 such that the scaling ratio of any two segments in Q would be bounded by l. It allows some deviation in the length of the segments from their expected length. Similarly, we compute

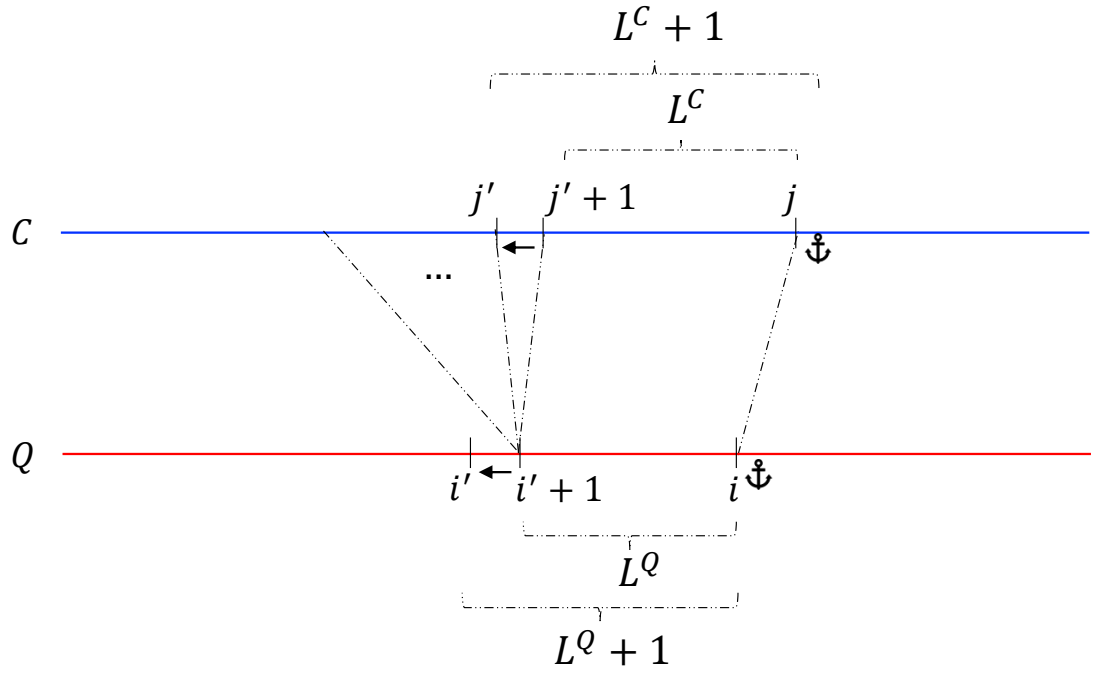


Figure 3.7: L^Q and L^C are the lengths of the considered segments of Q and C, respectively. The anchor signs indicate that the segment ends are fixed. The lengths increase as the starting indices decrease. The same Q segment of length L^Q is first compared with a set of lengthening C segments, where the starting index is decreased by 1 at each iteration. After that, the length of the Q segment is increased by 1.

the segment constraints for C in line 2. We set the maximum segment length be the alignment factor L in line 3. We set the base case in line 5.

We fill the table D in Algorithm 4. In line 3, given p pieces in Q, the ending index of the last piece (i.e., the p^{th} piece) will range from $(p \cdot L^Q_{\text{gmin}})$, given all the p pieces are in minimum length L^Q_{gmin}, to $\min(p \cdot L^Q_{\text{gmax}}, m)$, given all the p pieces are in maximum length L^Q_{gmax}.

In line 4, we enumerate for all the allowed lengths L^Q. For the segment with length L^Q in Q, the length L^C of the corresponding aligned segment in C will have a range from L^C_{min} to L^C_{max}, as defined in lines 9–10, such that the ratio of L^Q and L^C would be bounded by l. Because L^Q and L^C increase in line 4 and line 17, and the i and j are fixed in line 3 and line 15, respectively, the i' and j' decrease correspondingly.

It is visualized in Figure 3.7. A segment in Q with ending index i and starting index $i' + 1$ would be compared to a set of segments in C, all of which have a fixed end at j and a decreasing starting index, starting at $j' + 1$. For example, the

starting index of the first segment being compared is $j' + 1$, which has length L^C. The starting index of the second segment is j', which has length $L^C + 1$.

In line 23, we terminate the current iteration if there are no previously valid segment pairs (i.e., $dist_{\text{prev}} = \infty$).

In line 27, if the accumulated cost $dist_{\text{prev}}$ exceeds the current best so far, we stop the current iteration as the resulting $(dist_{\text{prev}} + dist_{\text{seg}})$ is guaranteed to be greater than the best-so-far, which is stored in $D[i, j, p]$.

To note, to be consistent with the results of using the lower bounding speedup technique, which will be introduced later, we compute the DTW in the reverse manner in line 45. Due to the nearest neighbor interpolation, $\text{DTW}_r(X^L, Y^L)$ may not equal to $\text{DTW}_r(\text{rev}(X)^L, \text{rev}(Y)^L)$, where X and Y are time series.

In line 47, we also save the 2-tuple (i', j'), which serves as the cutting points between segments, to obtain the segmentation result.

Parallel computing

We observe that the recurrence relation for the computation of $D[i, j, p]$ at p depends exclusively on the results of D computed at $p - 1$, as shown in Equation 3.15. It allows us to parallelize the loops over indices i and j on lines 3 and 15. In the following experimental section, we distribute the i-loop iterations (i.e., line 3 in Algorithm 4) across available threads only because there are already sufficient iterations from the i-loop to fill the available threads. There are $\left(\min(p \cdot L^Q_{\text{gmax}}, m) - (p \cdot L^Q_{\text{gmin}}) + 1\right)$ iterations from the i-loop. The parallel execution of the i-loop is implemented by `prange` in `Numba` in Python.

Early Abandoning

To accelerate the nearest neighbor search (or top-k search) for a query Q on a candidate set, we employ an early abandoning strategy. It prunes the search branch within the PSD computation of a specific candidate C as soon as the result corresponding to this search branch is determined to be suboptimal. We maintain a best-so-far variable, bsf, which represents the minimum final distance among the candidates processed with Q so far. bsf serves as an upper-bound threshold. To note, bsf is different to the best so far value stored in $D[i, j, p]$. During the evaluation of a new candidate C, we monitor the accumulated partial distance, $dist_{\text{prev}}$. If $dist_{\text{prev}}$ exceeds bsf, the corresponding final distance is guaranteed to exceed bsf. In such cases, the current search branch is immediately terminated. The implementation of this pruning mechanism is detailed in lines 25–26 in Algorithm 4.

In the case of top-k search, we need to maintain the top-k final distances and use the k-th final distance as bsf.

Lower bounding

Recall that DTW is quadratic in complexity. When we use DTW as the base distance measure in PSD, the computation of the DTW of the interpolated segment can be sped up by computing $\mathrm{LB}_{\mathrm{Shen}}$ for the lower bounding. From Figure 3.7, we observe that the same segment of Q, with length L^Q, is compared to a set of lengthening segments of C, with a fixed end at j. The length of these segments is from $L^C_{\min}$ to $L^C_{\max}$, as indicated in line 17 in Algorithm 4.

They share the same suffix with length $L^C_{\min}$. It encourages us to view both Q and C reversely. The reversed segments are denoted as Q' and C', as in lines 8 and 21 in Algorithm 4. In this reversed view, they share the same prefix with length $L^C_{\min}$. Hence, we construct an indexed collection $\tilde{\mathbb{Q}}$ for Q' with the maximum length of the segments being compared in C, which is $L^C_{\max}$, as in lines 12–14.

In lines 29–37, we compute the lower bound between Q' and the current segment of C from the sketch if it is not yet initialized. Note that, since we may skip some iterations in lines 23–28, the initialization may not be executed at $L = L^C_{\min}$. We add the minimum possible contribution of each c' to the distance contributed by the first alignment, which is $lb = (c'_1 - q'_1)$. For the lower bound of the subsequent segments, we compute them incrementally in line 34. Because the last point of segment C' must map to the last point of Q', we further tighten the lower bound by using the last alignment in line 42 instead. Note that, because some iterations may be skipped, not every lower bound of the subsequent segments has been computed incrementally. According to Theorem 16, we break the for-loop both in lines 30 and 17 by checking whether the current lower bound is greater than $D[i, j, p]$ in lines 35 and 40.

Furthermore, we can reduce the computational overhead of constructing the sorted reaches $\tilde{\mathbb{q}}_k$ (lines 11–14). As illustrated in Figure 3.7, the segment of Q involved in the comparison grows incrementally. Since the reversed versions of these segments share a common prefix, the sorted reaches $\tilde{\mathbb{q}}_k$ computed for a segment Q' of length L^Q can be reused to compute those for the subsequent segments.

This optimization is detailed in Algorithm 5. It is important to note that because r' is a function of L^Q, and the construction of $\tilde{\mathbb{q}}_k$ depends on r', reuse is limited to cases where r' remains constant. We construct reaches $\tilde{\mathbb{q}}_k$ from the sketch in lines 13–15 and keep track of the r' used for construction in line 16. If r'

remains constant, we can use previously computed reaches $\tilde{\mathrm{q}}_k$ to compute the new set of reaches $\tilde{\mathrm{q}}_k$. Since the ending index of reaches depends on $\min(\lfloor kl \rfloor + r', L^Q)$, we need to check whether we have a new ending index when L^Q increases. If the ending index has been changed, we need to add the new data points $q'_{e_{\text{new}}}$ to the sorted sequence $\tilde{\mathrm{q}}_k$, as in line 8.

$L^C_{\max}$ depends of L^Q. If $L^C_{\max}$ increase because L^Q increase, we construct the new reach $\tilde{\mathrm{q}}_k$ as in lines 10–11.

3.4.3 Guided Distance

For faster computation, one would want to use a distance measure with linear complexity, such as ED, as the distance base measure for PSD. While PSED is effective for identifying phase-scaling changes, certain applications require capturing complex properties within those segments that ED cannot handle. There are two ways to address it. One approach is to use an alternative distance measure that captures these complex properties as the base distance for computing the PSD, such as DTW. But PSDTW is slower than PSED. The other approach is to use the segmentation result returned by PSED. To address this, we propose a two-stage framework in which PSED-derived segmentation guides the application of the target distance measure M. Let $\mathcal{P}^* = \{(i_1, j_1), (i_2, j_2), \ldots, (i_{P+1}, j_{P+1})\}$ be the set of optimal cut points on Q and C obtained by computing the PSED. We utilize $\mathcal{P}^*$ to partition both series into P aligned pairs of segments. The final distance, denoted as M^{PSED}, is calculated by summing the distances of these scaled pairs using a target distance measure M:

$$M^{\text{PSED}}(Q, C) = \sum_{p=1}^{P} M(Q_p^L, C_p^L) \tag{3.16}$$

, where $Q_p = Q(i_p + 1 : i_{p+1})$ and $C_p = C(j_p + 1 : j_{p+1})$.

3.5 Experiments

We evaluate the performance of our proposed distance measure framework, PSD, via its two instantiations: PSED and PSDTW. Specifically, we focus on a query retrieval task in which the query exhibits piecewise-scaled distortion. Our objective is to verify whether PSD achieves invariance to piecewise-scaled distortion, thereby allowing it to correctly retrieve the most similar candidate.

We outline the computation environment used for experiments. The programs were implemented in Python. The Python interpreter version is 3.12.3. We utilized the `aeon` [64] library of version 1.2.0 to obtain the baseline distance measures. All experiments were conducted on a workstation running Almalinux 9, equipped with an Intel Xeon Gold 6326 CPU and 256GB of RAM.

The code and data are available at `https://github.com/colemanyu/k-scaling-factor-dtw`. The data was downloaded from `https://www.timeseriesclassification.com`.

We detail the experimental setup in Section 3.5.1 and present the results in Section 3.5.2.

3.5.1 Experimental Setup

We utilize the GunPoint dataset, originally released in 2003 [65], as the running example. Widely regarded as the "iris" dataset of the time series community [66], it has appeared in over one thousand publications. Beyond its ubiquity, the dataset addresses a critical distinction: misinterpreting the act of aiming a gun as merely pointing a finger could have life-threatening consequences.

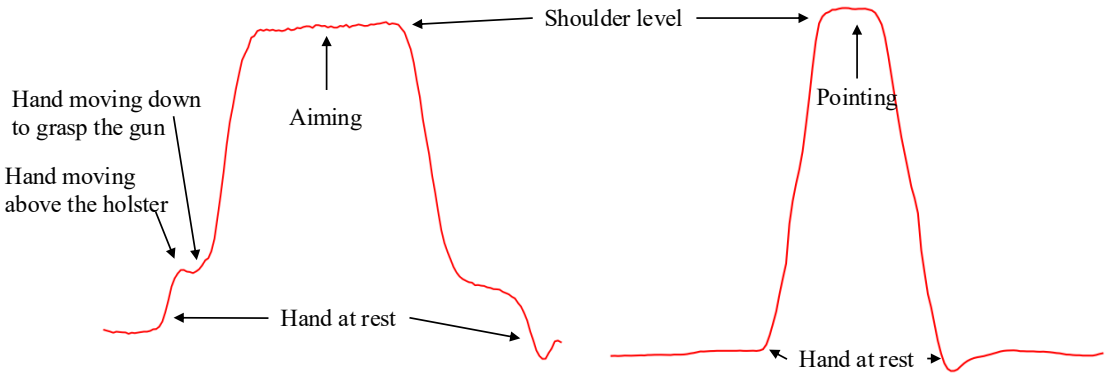


Figure 3.8: Left (Right): A time series from the "Gun" ("Point") scenario. Critical periods, such as "Aiming", are annotated.

The dataset contains two classes: "Gun" and Point". Actors aim at an eye-level target using either a replica gun or their index finger, as illustrated in Figure 3.8. The resulting time series represent the X-axis centroid of the actor's right hand. Each action lasts for 5 seconds, with the pointing/aiming phase occurring for approximately one second. Recorded at 30 fps, each sample consists of 150 data points.

A key limitation of the original dataset is the assumption that every action lasts exactly 5 seconds. In reality, actors perform actions at varying speeds. If

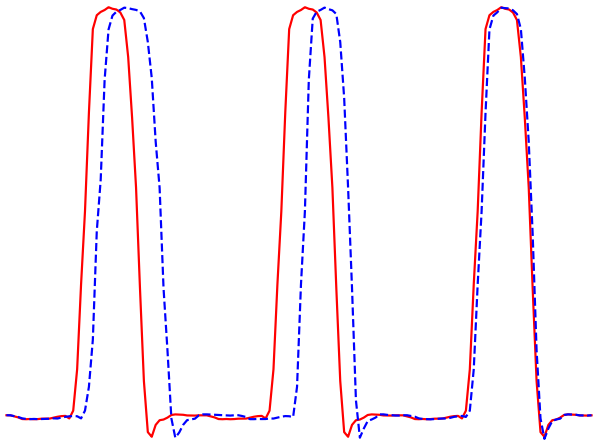


Figure 3.9: The red solid (blue dashed) time series is an instance in the target (query) set.

an actor is asked to perform the action three times continuously in a row, we are likely to observe a time series containing three phases, each with a unique rate. This phenomenon is depicted in Figure 3.9, where the red solid curve represents an ideal case that consists of three identical phases (i.e., with identical rates), while the blue dashed curve represents a realistic scenario with varying rates. Consequently, comparing them requires assigning different scaling factors to each phase to accommodate phase-specific rates (i.e., piecewise scaling distortions).

The target and query sets

The target set: We now describe the procedure for preparing the target and query sets for the retrieval task. To construct the target set, we concatenate P repetitions of each time series instance from the source dataset. To ensure a fair comparison, we keep the resulting time series to match the original length n. This is achieved by rescaling each phase to a length of n/P prior to concatenation. Note that we must handle remainders due to the division to ensure that the final time series length is exactly n. An example of such a target (where $P = 3$) is depicted by the red solid line in Figure 3.9.

The query set: To construct the query set, we first determine the specific lengths for the P segments. Starting with an expected mean length of n/P, we define the minimum segment length as $L_{\min} = (n/P)\sqrt{l}$ and the maximum as $L_{\max} = (n/P)\sqrt{l}$. This formulation ensures that the ratio of the lengths of any two segments is bounded by l. We then generate P random integers (lengths) within $[L_{\min}, L_{\max}]$ subject to the constraint that their sum is exactly n. Finally,

we construct the query by rescaling P copies of the source time series to these generated random lengths and concatenating them. The resulting time series maintains the original length n. An example of such a query (where $P = 3$) is depicted by the blue dashed line in Figure 3.9.

Both the target and query sets are derived from the same source dataset. Consequently, the ground truth target for a query is defined as the instance in the target set that is generated from the same underlying source time series. To note, we shuffled the ordering of the time series in the resulting target and query sets to prevent identical index orderings between the two sets, which could affect early abandoning performance.

Datasets

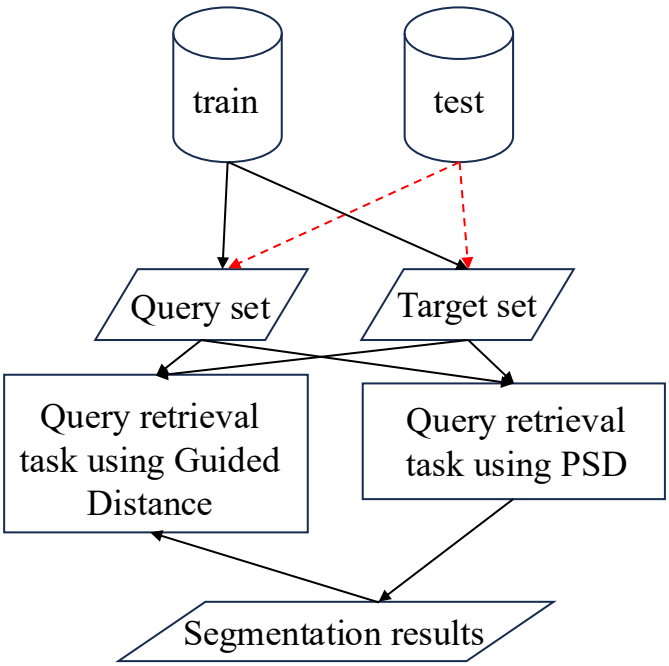


Figure 3.10: Experiment flowchart. We use the train split for the initial experiments, and both splits in the final experiments.

Table 3.2 details the ten datasets used in this study. For the initial experiments, we use only the train split. For the final experiments, we use all the data (i.e., both test split and train split) to further verify the accuracy and the efficiency of our methods. The experiment flowchart is depicted in Figure 3.10.

In our experiments, we set the warping window parameter $r = 0.1$, as suggested in the literature.

Table 3.2: Details of the ten datasets for the experiments. ECG (HAR) stands for Electrocardiogram (Human Activity Recognition).

Name	Type	Train Size	Test Size	Class	Length
SonyAIBORobotSurface1	Sensor	20	601	2	70
ECG200	ECG	100	100	2	96
MedicalImages	Image	381	760	10	99
CBF	Simulated	30	900	3	128
SwedishLeaf	Image	500	625	15	128
Plane	Sensor	105	105	7	144
PowerCons	Device	180	180	2	144
GunPoint	HAR	50	150	2	150
Adiac	Image	390	391	37	176
Epilepsy	HAR	137	138	4	207

3.5.2 Experimental Results

We employ top-k accuracy (often referred to as Precision@k [67]) as the evaluation metric. For a single query Q, this metric indicates whether the correct match is successfully retrieved within the top k candidates:

$$P@k(Q) = \begin{cases} 1 & \text{if the ground truth is in the top-}k\text{ results} \\ 0 & \text{otherwise} \end{cases} \tag{3.17}$$

To determine the top-k results, we compute the distance between Q and every time series in the target set, generating a distance profile of length equal to the dataset size. The top-k results correspond to the k instances with the smallest distances in this profile. Finally, we evaluate the overall performance by computing the mean top-k accuracy across the entire query set $\mathcal{D}$:

$$\overline{P@k} = \frac{\sum_{Q \in \mathcal{D}} P@k(Q)}{|\mathcal{D}|} \tag{3.18}$$

We choose $k \in 1, 3$. $k = 1$ refers to the exact retrieval. $k = 3$ give some tolerance for the retrieval.

Results of the GunPoint dataset (train split)

We utilize the GunPoint dataset to evaluate how the parameters P and l affect the ability of PSD to achieve invariance under piecewise scaling. The results are presented in Table 3.3, In the tables, the best performance of $\overline{P@1}$ is highlighted in bold, and the second-best is underlined. We benchmark our method against

Table 3.3: The accuracy comparison across eight distance measures on the GunPoint dataset (train split).

P	l	ED		DTW [3]		ADTW [11]		DDTW [5]		shapeDTW [7]		WDDTW [6]		WDTW [6]		PSED		PSDTW	
		$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$
2	1.25	0.30	0.56	0.82	1.00	0.54	0.84	0.84	0.92	0.96	1.00	0.88	0.96	0.82	0.98	**0.98**	1.00	0.86	1.00
	1.50	0.14	0.36	0.88	1.00	0.38	0.64	0.78	0.94	**1.00**	1.00	0.80	0.92	0.88	0.96	**1.00**	1.00	0.96	1.00
	1.75	0.16	0.34	0.82	1.00	0.38	0.66	0.64	0.80	0.90	1.00	0.66	0.80	0.82	0.96	**1.00**	1.00	0.94	1.00
	2.00	0.12	0.24	0.84	0.98	0.34	0.66	0.72	0.88	0.96	1.00	0.68	0.86	0.82	0.98	**1.00**	1.00	0.96	1.00
3	1.25	0.30	0.50	0.84	0.92	0.70	0.88	0.80	0.92	**0.96**	0.98	0.84	0.94	0.86	0.98	0.94	1.00	0.88	0.96
	1.50	0.10	0.28	0.84	0.96	0.60	0.78	0.66	0.86	0.90	1.00	0.72	0.86	0.84	0.98	**0.96**	1.00	0.86	0.96
	1.75	0.10	0.28	0.88	1.00	0.42	0.78	0.72	0.88	0.76	0.94	0.74	0.88	0.88	1.00	**1.00**	1.00	0.88	0.98
	2.00	0.02	0.12	0.86	0.94	0.38	0.52	0.60	0.78	0.70	0.94	0.60	0.80	0.82	0.92	**0.98**	1.00	0.86	0.98
4	1.25	0.34	0.50	0.70	0.86	0.68	0.88	0.60	0.78	0.70	0.90	0.64	0.80	0.70	0.86	**0.86**	0.94	0.72	0.88
	1.50	0.22	0.38	0.64	0.80	0.60	0.92	0.52	0.64	0.68	0.90	0.52	0.66	0.64	0.82	**0.86**	0.98	0.72	0.82
	1.75	0.16	0.30	0.72	0.92	0.56	0.80	0.60	0.78	0.56	0.80	0.56	0.80	0.70	0.88	**0.92**	1.00	0.80	0.94
	2.00	0.06	0.12	0.58	0.78	0.50	0.72	0.46	0.64	0.50	0.82	0.50	0.64	0.56	0.82	**0.88**	0.98	0.64	0.88

several state-of-the-art distance measures, including ADTW [11], DDTW [5], shapeDTW [7], WDDTW [6], and WDTW [6],.

PSED achieves the highest accuracy, followed closely by PSDTW. We attribute PSED's superior performance over PSDTW to the specific nature of the distortions in the query set, that is, the "pure" piecewise scaling distortions. Since the query set exhibits only piecewise scaling distortions, the corresponding segments of the query and target time series differ solely in length (scale). Consequently, the additional flexibility provided by DTW in PSDTW is unnecessary and may inadvertently increase the similarity of incorrect matches, thereby reducing discriminative power relative to the stricter PSED measure. However, PSDTW remains theoretically essential for handling the local distortion within the phase. PSDTW applies DTW within the scaled segment, enabling robust alignment across local nonlinearities.

Table 3.4: The accuracy comparison across six PSED-guided distance measures on the GunPoint dataset (train split).

P	l	DTW^{PSED}		$\text{ADTW}^{\text{PSED}}$		$\text{DDTW}^{\text{PSED}}$		$\text{shapeDTW}^{\text{PSED}}$		$\text{WDDTW}^{\text{PSED}}$		$\text{WDTW}^{\text{PSED}}$	
		$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$
2	1.25	0.86	1.00	0.98	1.00	**0.72**	0.92	0.98	1.00	**0.74**	0.96	0.86	1.00
	1.50	0.88	1.00	1.00	1.00	**0.60**	0.90	**0.98**	1.00	**0.64**	0.90	0.92	1.00
	1.75	0.94	1.00	1.00	1.00	0.84	0.94	1.00	1.00	0.86	0.96	0.94	1.00
	2.00	0.98	1.00	1.00	1.00	**0.60**	0.78	0.98	1.00	**0.66**	0.80	0.98	1.00
3	1.25	**0.82**	0.96	0.94	1.00	**0.74**	0.90	**0.92**	0.98	**0.76**	0.90	**0.82**	0.96
	1.50	0.86	1.00	0.96	1.00	0.72	0.90	0.94	1.00	0.80	0.90	0.88	1.00
	1.75	0.88	1.00	1.00	1.00	**0.68**	0.90	1.00	1.00	**0.70**	0.94	0.90	1.00
	2.00	0.86	0.96	0.98	1.00	**0.54**	0.86	0.96	1.00	0.62	0.86	0.88	0.96
4	1.25	0.74	0.88	0.86	0.94	0.78	0.80	0.80	0.92	0.78	0.82	0.74	0.88
	1.50	0.72	0.84	0.86	0.98	0.68	0.84	0.84	0.96	0.66	0.84	0.72	0.86
	1.75	0.74	0.96	0.92	1.00	**0.54**	0.90	0.90	1.00	0.56	0.88	0.74	0.96
	2.00	0.66	0.90	0.88	0.98	0.58	0.82	0.88	0.98	0.62	0.80	0.66	0.90

Guided Distance: We further investigate whether the segmentation results returned by PSED can serve as a guide to enhance other distance measures. As shown in Table 3.4, this approach generally improves accuracy. The exceptions are highlighted in bold, indicating cases where the segmentation led to worse performance. Notably, in most cases, only DDTW and WDDTW failed to benefit from PSED-guided segmentation. We argue that the performance degradation of DDTW and WDDTW stems from their derivative-based nature. They rely on matching slope or shape features. Consequently, they are highly sensitive to segmentation boundaries. A non-perfect cut that falls within a shape feature segmentize it, and these features are then destroyed. When the algorithm subsequently attempts to map these features, it fails to find correct correspondences, resulting in high distances.

Running time analysis: We evaluate two variants of PSD, PSED and PSDTW, each implemented with three levels of speedup techniques:

1. `vanilla` (i.e., Basic implementation)
2. `parallel_bsf` (i.e., With parallelization with Best-So-Far early abandoning)
3. `parallel_bsf_lb` (i.e., In addition, incorporating lower bound pruning)

Figure 3.11 illustrates the runtime performance across varying parameters P and l. The first and third columns in the figure display the running time and the number of distance calculations, respectively, for all six methods. To better visualize the performance differences among the efficient implementations, the second and fourth columns omit the `vanilla` baselines and focus exclusively on the other two optimized variants.

The results indicate that `PSDTW_vanilla` is orders of magnitude slower than the other approaches. The computation time for all methods increases with the scaling factor l. The fourth column confirms that the lower bounding strategy reduces the number of distance calls. However, the second column reveals a critical trade-off in actual runtime. While the lower bounding strategy successfully accelerates PSDTW in some cases (those with large runtime values), it always slows down PSED. We note that in some cases, the speedup for PSDTW is tiny or occasionally slows it down. It is because we use it on top of early abandoning. Early abandoning has already pruned many distance calls, resulting in not enough calls to be pruned by the lower bounding strategy to outweigh the time spent computing the lower bound in those cases.

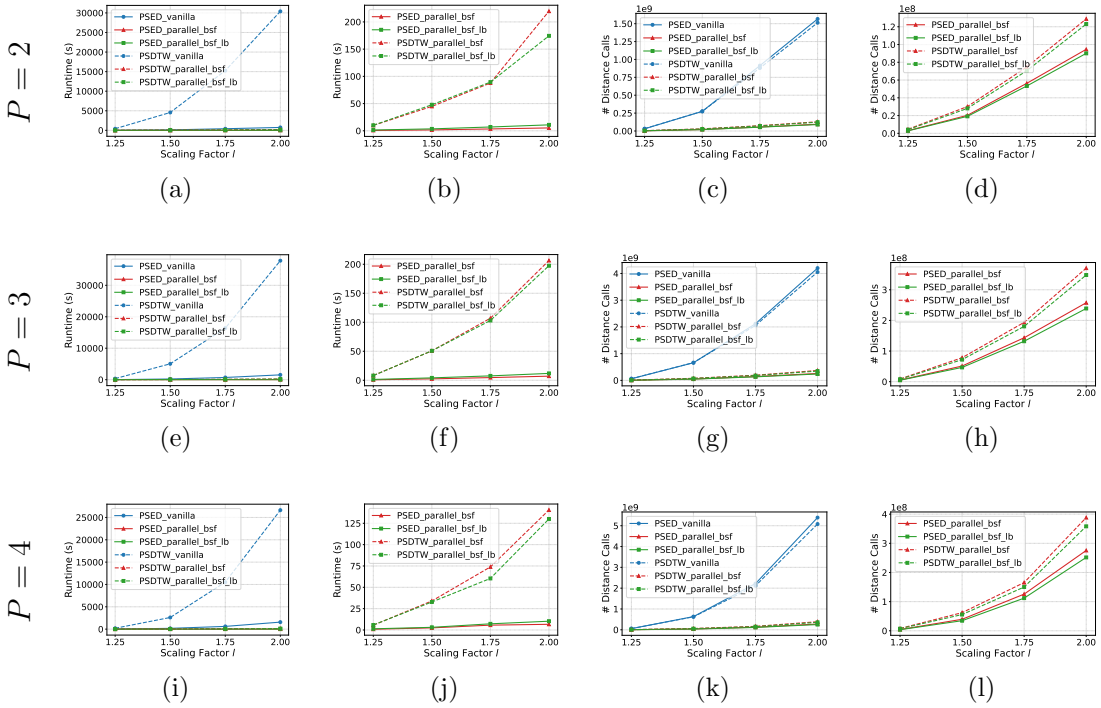


Figure 3.11: Comparing the runtime and the number of distance calls for the methods with different P and l on the GunPoint dataset (train split).

Table 3.5: The estimated time proportion of the lower bound calculation.

l	$P = 2$		$P = 3$		$P = 4$	
	PSED	PSDTW	PSED	PSDTW	PSED	PSDTW
1.25	0.585	0.106	0.475	0.099	0.448	0.130
1.50	0.565	0.128	0.464	0.083	0.305	0.073
1.75	0.540	0.074	0.438	0.033	0.302	-0.110
2.00	0.539	-0.203	0.446	0.018	0.408	0.003

We report the estimated time proportion of the lower bound calculation in Table 3.5. The runtime of `parallel_bsf_lb` consists of the time for all the distance calls and the time spent computing the lower bound, while that of `parallel_bsf` only consists of the former part. We use the average time of a distance call of `parallel_bsf` to estimate that of `parallel_bsf_lb`. We then estimate the time for all the distance calls of `parallel_bsf_lb` by multiplying the estimated average time of its distance call by the number of distance calls. By subtracting this estimated value from the runtime of `parallel_bsf_lb`, we obtain the corresponding estimated time spent computing the lower bound and hence its proportion. Note that the estimated time for all distance calls may exceed the overall runtime, leading to an estimated negative value for the time spent computing the lower bound and its

proportion. Table 3.5 shows that the estimated time proportion of the lower bound calculation for PSED is much higher than that of PSDTW. This suggests that for the computationally lighter ED, the overhead of calculating the lower bound always outweighs the time saved by pruning.

Results of the ten datasets (train split)

We have the following findings from the previous experiment:

1. From Table 3.4, PSED outperformed PSDTW in handling piecewise scaling distortions in this task.

2. From Figure 3.11, the lower bound offered no efficiency gain for PSED.

Hence, we select `PSED_parallel_bsf` as the representative method for this evaluation.

We fix the parameters at $P = 3$ and $l = 1.5$. When P in the target set is

Table 3.6: Comparison of $\overline{P@1}$ on setting of P between the ground-truth value and the user-given value on the GunPoint dataset (train split) with $l = 1.5$. The diagonal elements, corresponding to the correct matches of P, are the best results.

User-given P	**Ground-truth P**		
	2	**3**	**4**
2	**1.00**	0.66	0.48
3	0.98	**0.96**	0.62
4	**1.00**	0.90	**0.86**

not given, the value of P can be determined as follows. A heuristic approach to determine P is to test a range of values for P as shown in Table 3.6. The correct matches between the ground-truth value and the user-given value on P correspond to the best results.

The accuracy results of the ten datasets (train split) are presented in Table 3.7, while the results for the PSED-guided distance measures are detailed in Table 3.8. Finally, the runtime efficiency for all datasets is summarized in Table 3.9. It shows a significant speedup, ranging from 8.59X to 176.14X.

Results of the ten datasets (both train and test split)

With the computational efficiency `PSED_parallel_bsf`, we are now able to conduct the experiment on all the data, including both the train split and the test split.

Table 3.7: The accuracy comparison across eight distance measures on the ten datasets (train split).

Dataset	ED		DTW		ADTW		DDTW		shapeDTW		WDDTW		WDTW		PSED	
	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$
SonyAIBORobotSurface1	0.30	0.70	0.90	1.00	**1.00**	1.00	0.80	0.85	0.50	0.70	0.80	0.85	**1.00**	1.00	**1.00**	1.00
ECG200	0.12	0.20	0.70	0.81	0.78	0.83	0.60	0.74	0.59	0.76	0.59	0.73	0.70	0.80	**0.81**	0.88
MedicalImages	0.08	0.19	0.63	0.78	0.69	0.85	0.52	0.68	0.57	0.75	0.51	0.68	0.62	0.78	**0.76**	0.88
CBF	0.53	0.67	0.83	1.00	0.90	1.00	0.73	0.87	0.73	0.90	0.77	0.90	0.90	1.00	**1.00**	1.00
SwedishLeaf	0.07	0.12	0.82	0.92	0.84	0.93	0.70	0.83	0.78	0.91	0.69	0.83	0.83	0.92	**0.97**	0.99
Plane	0.09	0.14	0.50	0.74	0.59	0.78	0.38	0.64	0.53	0.79	0.37	0.61	0.49	0.75	**0.75**	0.93
PowerCons	0.26	0.43	0.77	0.98	**0.80**	1.00	0.77	0.98	0.77	0.99	0.77	0.99	0.77	0.99	**0.80**	1.00
GunPoint	0.10	0.28	0.84	0.96	0.60	0.78	0.66	0.86	0.90	1.00	0.72	0.86	0.84	0.98	**0.96**	1.00
Adiac	0.01	0.02	0.37	0.50	0.12	0.21	0.32	0.42	0.27	0.41	0.31	0.41	0.36	0.50	**0.57**	0.71
Epilepsy	0.18	0.26	0.74	0.88	0.80	0.91	0.65	0.82	0.38	0.47	0.58	0.79	0.70	0.83	**0.83**	0.91

Table 3.8: The accuracy comparison across six PSED-guided distance measures on the ten datasets (train split).

Dataset	DTW$^{\text{PSED}}$		ADTW$^{\text{PSED}}$		DDTW$^{\text{PSED}}$		shapeDTW$^{\text{PSED}}$		WDDTW$^{\text{PSED}}$		WDTW$^{\text{PSED}}$	
	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$	$\overline{P@1}$	$\overline{P@3}$
SonyAIBORobotSurface1	0.95	1.00	1.00	1.00	**0.65**	0.80	1.00	1.00	**0.75**	0.80	**0.95**	1.00
ECG200	0.73	0.84	0.80	0.88	**0.54**	0.68	0.81	0.88	**0.53**	0.69	0.73	0.84
MedicalImages	0.66	0.78	0.76	0.88	**0.51**	0.69	0.74	0.88	0.51	0.70	0.66	0.79
CBF	0.90	1.00	0.93	1.00	0.73	0.93	0.93	0.97	**0.70**	0.93	0.90	1.00
SwedishLeaf	0.82	0.92	0.97	0.99	0.72	0.87	0.97	0.99	0.73	0.88	**0.82**	0.92
Plane	0.60	0.80	0.75	0.94	0.50	0.67	0.74	0.90	0.50	0.69	0.59	0.80
PowerCons	0.78	0.99	**0.79**	1.00	0.79	1.00	0.79	1.00	0.79	1.00	0.78	1.00
GunPoint	0.86	1.00	0.96	1.00	0.72	0.90	0.94	1.00	0.80	0.90	0.88	1.00
Adiac	**0.33**	0.45	0.57	0.71	**0.21**	0.34	0.53	0.70	**0.20**	0.33	**0.34**	0.45
Epilepsy	0.77	0.88	**0.78**	0.88	0.66	0.79	0.74	0.89	0.66	0.82	0.80	0.89

The results are shown in Table 3.10. Note that we retrieve only the most similar candidate to the query here for efficiency of early abandoning, so we report only $\overline{P@1}$ in the table. It outperforms all the other methods.

3.6 Concluding Remarks

In this paper, we proposed a novel distance measure framework, Piecewise Scaling Distance (PSD), which relaxes the strict assumption of a single scaling factor across the entire time series. We presented an exact dynamic programming (DP) algorithm to solve this problem. To enhance efficiency and prevent pathological segment alignments, we introduced a constrained version, which limits the search space of the segment lengths based on a given scaling factor bound. To enhance computational efficiency, we integrated two optimization techniques for the general PSD framework. In addition, we proposed incorporating a lower-bounding strategy to accelerate PSDTW. Our experimental results demonstrate the necessity and effectiveness of PSD for identifying matches between a query Q with piecewise scaling distortions and a set of candidate time series without such distortions.

Table 3.9: The runtime of PSED_vanilla and PSED_parallel_bsf on the ten datasets (train split). We also show the percentage of distance calls that are pruned and the speedup achieved.

Name	`PSED_vanilla` Time (s)	`PSED_parallel_bsf` Time (s)	% distance calls pruned	Speedup
SonyAIBORobotSurface1	0.745	0.089	77.69%	8.40X
ECG200	96	3	90.74%	33.62X
MedicalImages	2108	36	96.44%	59.26X
CBF	41	1	74.75%	32.52X
SwedishLeaf	9785	98	96.19%	99.81X
Plane	648	5	96.66%	126.51X
PowerCons	2058	40	83.64%	51.81X
GunPoint	211	2	92.25%	99.33X
Adiac	19547	109	96.66%	178.72X
Epilepsy	7510	268	43.46%	28.01X

Table 3.10: The accuracy comparison across eight distance measures on the ten datasets (all the data: train split & test split)

Dataset	ED	DTW	ADTW	DDTW	shapeDTW	WDDTW	WDTW	PSED	Time (s)
SonyAIBORobotSurface1	0.05	0.51	0.61	0.45	0.20	0.44	0.51	**0.67**	65
ECG200	0.07	0.65	0.77	0.52	0.56	0.50	0.66	**0.79**	10
MedicalImages	0.06	0.52	0.62	0.39	0.50	0.38	0.51	**0.68**	281
CBF	0.09	0.55	0.63	0.40	0.31	0.40	0.55	**0.70**	640
SwedishLeaf	0.05	0.73	0.76	0.58	0.68	0.59	0.74	**0.95**	447
Plane	0.07	0.38	0.40	0.24	0.46	0.24	0.37	**0.68**	19
PowerCons	0.15	0.61	0.63	0.59	0.61	0.59	0.61	**0.64**	129
GunPoint	0.06	0.59	0.23	0.53	0.67	0.54	0.61	**0.88**	22
Adiac	0.01	0.34	0.09	0.28	0.25	0.27	0.34	**0.53**	417
Epilepsy	0.16	0.71	0.76	0.60	0.37	0.54	0.66	**0.82**	1113

We summarize the key deliverables.

- PSD is robust to piecewise scaling distortions. Hence, it can be used to achieve better accuracy for higher-level tasks that use it as a subroutine.
- Optimal phase cut: The algorithm outputs the cutting points that partition the two input series into P contiguous aligned segments. Hence, it reveals the regimes in the time series.
- Guided distance: The cutting points can guide the application of other distance measures to compute the overall distance between the two series.
- Phase-specific scaling factors: It identifies the local scaling factors for each phase (i.e., the p^{th} segment), defined by the ratio between $i_{p+1} - i_p$ and $j_{p+1} - j_p$. Hence, it reveals how expression rates vary across segments of the two series.

We discuss the potential impacts and implications.

- It improves accuracy when piecewise scaling distortions are present in the data, which is common in applications such as "Query-by-Humming" or "Gesture Recognition", where humans vary the speeds in different phases.

- The framework mitigates the need for manual segmentation by automatically identifying the optimal cutting points and reviewing optimal segment structures. It facilitates the continuous monitoring of sequential actions without requiring a metronome or post-manual partitioning of the raw data stream.

We discuss the challenges and future work.

- The number of pieces P currently requires manual definition. Although we can find the best value by testing a range of values, designing a more efficient method is a challenge.

- The computational time of PSD is slow. Hence, future work should focus on improving its efficiency. A lower bound specifically designed for PSD can admissibly prune unpromising candidates. We note that many subsequences that share the same portion (i.e., the suffix) are involved in the computation of the internal similarity. It would be beneficial to reuse these computations to speed up the computation. It may be possible to reuse the result of the computation on the shorter segment to compute the result for the longer segment. A similar idea can be found in studies accelerating DTW. Incremental DTW [68] allows for the reuse of the accumulating cost matrix for an incremental calculation of DTW. A difficulty in applying a similar idea to PSD is that it computes the distance measure between two growing segments after scaling them via interpolation. The interpolation alters the subsequence structure. It prevents the straightforward reuse of the computed result from the shorter segment for the longer segment.

Algorithm 4 Constrainted PSDTW with parallel computing, early abandoning and lower bounding

Input: Query series Q, Candidate series C, DTW constraint parameter r (in fraction), Number of pieces P, best_so_far bsf
Output: The final distance $D[m, n, P]$ if $D[m, n, P] \leq$ bsf, otherwise ∞

Execute Algorithm 3 for initialization.
for $p \leftarrow 1$ **to** P **do**
  for $i \leftarrow (p \cdot L^Q_{\text{gmin}})$ **to** $\min(p \cdot L^Q_{\text{gmax}}, m)$ **do** ▷ This iterations can be parallelized.
    for $L^Q \leftarrow L^Q_{\text{gmin}}$ **to** L^Q_{gmax} **do**
      $i' \leftarrow i - L^Q$ ▷ i': End point of previous segment on Q.
      if $i' < 1$ **then**
        continue
      $Q' \leftarrow \text{rev}(Q(i' + 1 : i))$ ▷ rev(T): Reverse the input series T.
      $L^C_{\text{min}} \leftarrow \max(L^C_{\text{gmin}}, \lceil L^Q/l \rceil)$
      $L^C_{\text{max}} \leftarrow \min(\lfloor L^Q l \rfloor, L^C_{\text{gmax}})$
      $r' \leftarrow r \times \max(L^Q, L^C_{\text{max}})$ ▷ Compute the integer value of r.
      for $k \leftarrow 1$ **to** L^C_{max} **do**
        $\mathbb{q}_k \leftarrow (q'_{\max(1,\lceil k/l \rceil - r')}, \cdots, q'_{\min(\lfloor kl \rfloor + r', L^Q)})$ ▷ Construct indexed collection $\mathbb{Q}$ of Q'.
        $\tilde{\mathbb{q}}_k \leftarrow \text{sort}(\mathbb{q}_k)$
      for $j \leftarrow (p \cdot L^C_{\text{min}})$ **to** $\min(p \cdot L^C_{\text{max}}, n)$ **do**
        $lb \leftarrow 0.0$, $is\_lb\_initialized \leftarrow$ FALSE
        for $L^C \leftarrow L^C_{\text{min}}$ **to** L^C_{max} **do**
          $j' \leftarrow j - L^C$
          if $j' < 1$ **then**
            continue
          $C' \leftarrow \text{rev}(C(j' + 1 : j))$
          $dist_{\text{prev}} \leftarrow D[i', j', p - 1]$
          if $dist_{\text{prev}} = \infty$ **then**
            continue
          if $dist_{\text{prev}} >$ bsf **then**
            continue
          if $dist_{\text{prev}} > D[i, j, p]$ **then** ▷ $D[i, j, p]$ stores the best-so-far.
            continue
          if $\neg is\_lb\_initialized$ **then**
            for $k \leftarrow 1$ **to** L^C **do** ▷ Compute the lower bound from sketch.
              if $k = 1$ **then** ▷ Partial distance contributed by the first alignment.
                $lb = (c'_1 - q'_1)^2$
              else
                $lb = lb + \delta(c'_k, \tilde{\mathbb{q}}_k)$
              if $lb > D[i, j, p]$ **then**
                break
              $is\_lb\_initialized \leftarrow$ TRUE
          else
            $lb = lb + \delta(c'_{L^C}, \tilde{\mathbb{q}}_{L^C})$ ▷ Compute the lower bound incrementally.
          if $lb > D[i, j, p]$ **then**
            break
          $lb_{\text{check}} = lb - \delta(c'_{L^C}, \tilde{\mathbb{q}}_{L^C}) + (c'_{-1} - q'_{-1})^2$ ▷ Tighten the lower bound by using the last alignment.
          if $dist_{\text{prev}} + lb_{\text{check}} > D[i, j, p]$ **then**
            continue
          $dist_{\text{seg}} \leftarrow \text{DTW}_r(Q'^L, C'^L)$
          if $dist_{\text{prev}} + dist_{\text{seg}} < D[i, j, p]$ **then**
            $D[i, j, p] \leftarrow dist_{\text{prev}} + dist_{\text{seg}}$ ▷ Also save the 2-tuple (i', j') for the cut.
return $D[m, n, P]$

Algorithm 5 Optimize lines 11 to 14 in Algorithm 4 by reusing the computed sorted reaches $\tilde{\mathbb{q}}_{\mathbb{1}k}$.

Add the following line after line 1 in Algorithm 4.

1: $r'_{\text{cache}} \leftarrow -1$

Replace lines 11–14 in Algorithm 4 by the following lines.

2: **if** $r'_{\text{cache}} = r'$ **then**
3: **for** $k \leftarrow 1$ **to** $L^C_{\max}$ **do**
4: **if** $k \leq |\mathbb{Q}|$ **then**
5: $e_{\text{prev}} \leftarrow \min(\lfloor kl \rfloor + r', L^Q - 1)$
6: $e_{\text{new}} \leftarrow \min(\lfloor kl \rfloor + r', L^Q)$
7: **if** $e_{\text{new}} > e_{\text{prev}}$ **then**
8: $\tilde{\mathbb{q}}_{\mathbb{1}k} \leftarrow \tilde{\mathbb{q}}_{\mathbb{1}k} \cup q'_{e_{\text{new}}}$ ▷ Preserve the sorting while inserting $q'_{e_{\text{new}}}$
9: **else**
10: $\mathbb{q}_{\mathbb{1}k} \leftarrow (q'_{\max(1,\lceil k/l \rceil - r')}, \ldots, q'_{\min(\lfloor kl \rfloor + r', L^Q)})$
11: $\tilde{\mathbb{q}}_{\mathbb{1}k} \leftarrow \text{sort}(\mathbb{q}_{\mathbb{1}k})$
12: **else**
13: **for** $k \leftarrow 1$ **to** $L^C_{\max}$ **do** ▷ Same as lines 12–14 in Algorithm 4.
14: $\mathbb{q}_{\mathbb{1}k} \leftarrow (q'_{\max(1,\lceil k/l \rceil - r')}, \ldots, q'_{\min(\lfloor kl \rfloor + r', L^Q)})$
15: $\tilde{\mathbb{q}}_{\mathbb{1}k} \leftarrow \text{sort}(\mathbb{q}_{\mathbb{1}k})$
16: $r_{\text{cache}} \leftarrow r'$

Chapter 4

Leveraging Nearest Neighbors for Time Series Forecasting with Matrix Profile

Time series forecasting is a crucial task used in many applications, including the financial and medical sectors. It is unsurprising that it has been a focus of intensive research for years and that many methods have been proposed. Deep learning methods have proved promising for time series forecasting, especially for long sequence time series forecasting (LSTF). However, they tend to be complex and computationally intensive. A recent study suggests that a well-known baseline in machine learning, namely Gradient Boosting Regression Tree (GBRT), can compete with deep learning methods by appropriately data-engineering the data. A time series forecasting task is transformed into a window-based regression problem. A sliding window is used to transform the time series into predictor-response pairs. At time point t, the previous w target values up to t concatenated with the covariates at t are flattened to form a predictor. The subsequent h target values form a response. A multi-output GBRT model is trained on the predictor-response pairs. Given a new predictor, the corresponding response can be predicted. The model's performance depends on the quality of the covariates. We propose using the past nearest neighbor's information, such as its immediate subsequence and similarity of the nearest neighbor, for each predictor as its covariates to improve the model's performance. The nearest neighbors are found using a data primitive called Matrix Profile. In brief, it allows us to identify the nearest neighbor for every m-subsequence of a time series, where m is user-given. Our approach has also been extended to the case of k-nearest neighbors. Extensive experiments show that

covariates derived from the past nearest neighbors of the predictors can improve the model's performance.

4.1 Introduction

Humans have made predictions since ancient times. In ancient societies, accurate predictions were important to the success of subsistence activities such as hunting, planting, and harvesting. There was a need to predict weather dynamics, such as rainfall and temperature. Our ancestors used divination tools such as bones to make predictions. Without doubt, the accuracy was not guaranteed. Prediction is also essential nowadays. Predicting traffic-jam patterns only a few hours ahead can save time by enabling the selection of an alternative route [69]. A wealth could be created by forecasting stock market trends. Predicting the future, also known as time series forecasting, is crucial.

With advances in hardware technologies, we collect enormous amounts of data from diverse sources continuously in the form of time series data. A time series is an ordered sequence of measures, represented in real-valued numbers, at discrete equal-interval timestamps [70]. The vast data collections have created the era of "Big Data", which provides a wealth of datasets for developing and deploying reliable, robust, data-driven forecasting techniques [71]. Applications can be found in the financial sector, such as predicting business cycles and stock market movements [72, 69, 73, 74] and the medical sector, such as the status of critical patients according to their vital signs [75, 76] and the propagation of diseases such as influenza [74, 77] and COVID-19 [75, 78].

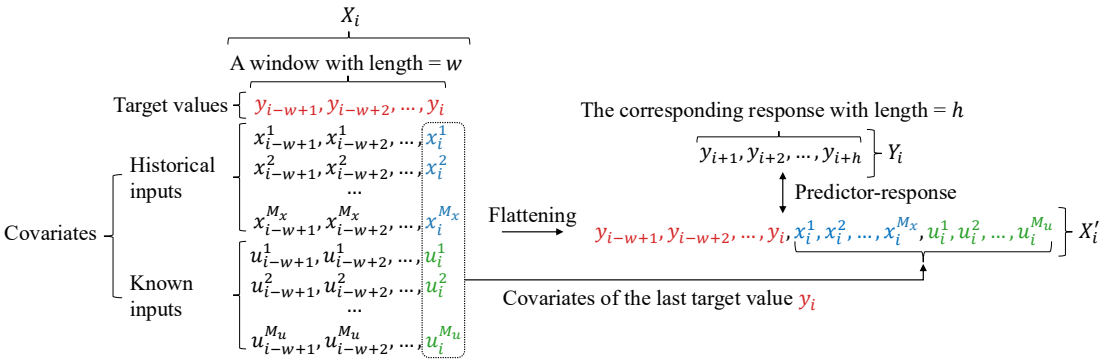


Figure 4.1: Flattening a 2D window X_i of size $(1 + M_x + M_u) \times w$ to a 1D vector X'_i of length $w + M_x + M_u$. The entries in the window are scalars. X'_i serves as the *predictor* for the regressor. Its expected *response* Y_i is a vector of length h. The regressor learns from the predictor-response pairs.

Many methods have been proposed, including traditional statistical methods (e.g., autoregressive integrated moving averages (ARIMA)) and deep learning methods (e.g., recurrent neural networks). A shortcoming of statistical methods is that they cannot handle complex, non-linear dynamics well because of their underlying assumptions of data stationarity and linearity. A shortcoming of deep learning methods is that they are complex and computationally intensive. A recent study [79] demonstrates that a well-known machine learning baseline, Gradient Boosting Regression Tree (GBRT), such as XGBoost, equipped with an appropriate data engineering of the data, can achieve competitive or even superior performance than the deep learning method. In detail, they transform the time series forecasting task into a window-based regression problem, as shown in Figure 4.1. For each training window of length w with the target value y_i of the last time point i, and its lagged values $y_{i-1}, y_{i-2}, \ldots, y_{i-w+1}$ are concatenated with the covariates of y_i to form the *predictor* for a multi-output GBRT. This transformation is called flattening.

Note that the standard GBRT returns only a single output. A variant, multi-output GBRT, is used. It is a group of GBRT models where each predicts a single output in a designated position τ in the output, where $\tau \in \{1, 2, \ldots, h\}$, while having the same input. The corresponding *response* (i.e., the horizon) is the following h target values of the last value y_i in the predictor. It provides a simple, more efficient yet accurate method for time series forecasting.

This study proposes a method to improve the performance of the GBRT approach by leveraging information from the past nearest neighbor of each subsequence in the target variable y. For each time point y_i, a window Q_i of length w is constructed with y_i as the last point, and then we retrieve the window's left nearest neighbor and use its information to create new covariates for the window, as shown in Figure 4.2. Only the left (past) nearest neighbor is considered to prevent data leakage. For the information of the left nearest neighbor NN^L, we are interested in its immediate subsequence of length l, where l is user-given, the similarity between NN^L and Q_i, NN^L's target values of length w, and covariates of the last point of NN^L. We also extend this idea to the case of k-nearest neighbors.

We make the following contributions:

- We are the first to propose leveraging the matrix profile to create covariates for each predictor to improve forecaster performance. It addresses one of the limitations that predictions are made only on observations within the lookback window by incorporating information about the window's past

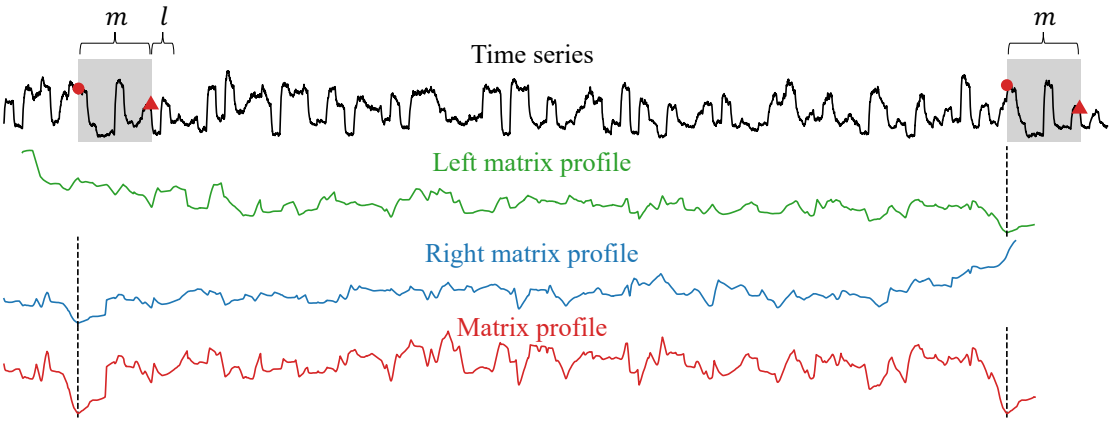


Figure 4.2: The left (right) matrix profile and the matrix profile of a time series. The matrix profile shows the distances between each m-subsequence and its nearest neighbor, where m is a user-given value. The left (right) matrix profile shows the same information but is limited to its left (right) nearest neighbor. For a particular m-subsequence indicated by the right gray box, its nearest neighbor is the left gray box, as indicated by the dashed line in the left matrix profile and the matrix profile. Similarly, the nearest neighbor of the left gray box is the right gray box, as indicated by the dashed line in the right matrix profile and the matrix profile. The first (last) point in each box is denoted by a red circle (triangle). l denotes the length of the immediate subsequence of the nearest neighbor. Intuitively, the immediate subsequence of the right gray box should be similar to this subsequence. To note, the left (right) matrix profile starts (ends) at a later (earlier) index.

nearest neighbor. The nearest neighbor provides a "lookahead" of the future trajectory by providing a similar historical trajectory.

- We have proposed two simple methods, namely, GBRT-NN, which uses the immediate subsequence of the nearest neighbor with length l as the covariates, and GBRT-NN-S, which also incorporates the similarity as a covariate. Furthermore, we study the effect of incorporating additional information from the nearest neighbor, such as its target values and covariates of the last point, to further extend the two simple methods.

- We also extend the idea to the case of k-nearest neighbors.

- Extensive experimental results show that our approach can improve the GBRT forecaster in almost all cases, tested on the four datasets, only with a small cost to compute the matrix profile, which is $\mathcal{O}(n^2)$, where n is the length of the time series.

The rest of this study is organized as follows. Section 4.2 presents the related work. In Section 4.3, we cover the necessary background knowledge. In Section 4.4,

we introduce our method. Section 4.5 contains an empirical evaluation. We discuss the findings from the experiments in Section 5.4. Finally, we conclude this study and provide future work in Section 5.5.

4.2 Related Work

Many methods have been developed for time series forecasting. We introduce them by categories.

4.2.1 Traditional methods

Traditional methods include rolling averages (RA), vector auto-regression (VAR) [69, 80], and auto-regressive integrated moving averages (ARIMA) [79, 81, 80]. Because of their rigorous statistical properties, they have long been the standard. The shortcomings of ARIMA and its variants include their high computational cost [69]. In contrast, VAR is arguably the most widely used method in this category, particularly in multivariate time series analysis, owing to its simplicity. However, most of these approaches have certain limitations. They perform well when the data meet specific statistical assumptions, such as stationarity [2], which means that the mean and the variance of the time series remain constant over time. It motivates the community to develop machine learning methods, particularly deep learning methods for time series forecasting.

4.2.2 Deep learning methods

Many deep learning models have been proposed, including RNN-based models, CNN-based models, GNN-based models, Transformer-based models, and compound models that incorporate different base models mentioned before [74]. The compound models are promising. For example, RNNs are well suited to capturing long-term dependencies, whereas CNNs are well suited to capturing short-term dependencies. A good way to improve performance is to combine them. For example, LSTnet [69] integrates CNN, RNN, and autoregressive [82] techniques to extract both short-term and long-term patterns. Using the occupancy rate of a freeway as an example [69] to explain these two patterns, the "short-term" patterns refer to the morning peaks against evening peaks, while the "long-term" patterns refer to the workday patterns against weekend patterns. Clearly, a good forecaster needs to capture and distinguish both kinds of patterns. Deep learning methods have also been

applied on chaotic time series to extract dynamic features by using complex hybrid architectures. [83] proposed a hybrid quantum neural network model which combines Broad Learning System (BLS) and quantum long short-term memory (QLSTM) network, namely BLS-QLSTM. It employs phase space reconstruction and BLS for feature augmentation. To further improve prediction accuracy, the gating structure within the LSTM network are replaced by variational quantum circuits (VQCs) to form QLSTM. A subsequent study [84] introduced a model called BLS-AttnTCN. It processes the enhanced features by a Temporal Convolutional Network (TCN) integrated with an attention mechanism to capture spatiotemporal patterns hierarchically and adaptively weight critical time steps while suppressing noise. As a result, prediction accuracy is enhanced. Despite the superior performance deep learning methods have achieved, they tend to be overly complex, opaque, and incur high computational costs. The model proposed in this study can also be regarded as a compound model.

4.2.3 Nearest neighbor-based methods

k-NN has also shown to be a promising method for time series forecasting [85, 86, 87]. The k-NN uses the $w-1$ lagged values of the last time point i to form a query Q with length w. It identifies the k previous similar subsequences to Q and uses their immediate subsequences to predict the immediate subsequence of Q with length h, The immediate subsequence of Q is also called the forecasting window. The intuition of this method is that history repeats itself. They are similar, and so are their immediate subsequences. The previous (historical) subsequences that are similar to Q can provide a hint about the future of Q. Figure 4.2 depicts this idea. Observe that the immediate subsequence of the right gray box is similar to that of the left gray box, which is the past (left) nearest neighbor of the right gray box. [88] expanded the usage of k-NN by including uncertainty quantification, namely a forecast interval, rather than traditional point forecasting. It is achieved by integrating k-NN with bootstrapping. The integrated model determines an optimal probability distribution from a range of forecasted values from different numbers of nearest neighbors. In other words, various k parameter values are used. A bootstrap sample of forecasts is generated after the optimal probability distribution is identified. Its mean, 2.5th percentile, and 97.5th are then utilized as the point forecast, lower bound, and upper bound, respectively.

The fundamental difference between this study and previous approaches [85, 86, 87] is that they directly use the subsequent points for prediction, whereas we

use the nearest neighbor's information for each predictor as covariates, which are used as inputs for other forecasters. We explain this subtle difference by Figure 4.2. The previous approaches simply use the information of the nearest neighbors of the last lookback window, which consisted of the last time point and its lagged values (i.e., the right gray box), for prediction. In contrast, we use the information of the nearest neighbors of *all* of the windows in the time series.

4.2.4 Matrix Profile-based methods

To the best of our knowledge, there are only four studies using the matrix profile in time series forecasting. [87] found the most similar historical subsequence and used its succeeding values as the prediction, similar to the nearest neighbor-based methods mentioned above. [89] used the matrix profile to visualize the financial time series data by identifying motifs. [90] used matrix profile to detect structural anomalies and incorporated the anomaly information to guide an attention-based long short-term memory (LSTM) model in forecasting COVID-19 indicators. [91] proposed an encoding-decoding methodology to learn repeating granular trends in periodic data. It first identifies a special kind of motif called "time series chain". The raw time series is then transformed into a level-encoded sequence based on the time series chain. The model predicts future labels instead of raw values. The predicted labels are decoded back into the original domain (i.e., raw values) using a weighted percentage change based on the historical centrality of the chain.

None of these studies uses the matrix profile to find nearest neighbors and uses their information as covariates, which can be fed into any forecaster, such as GBRT [79] and DeepGlo [92], that supports covariates.

4.3 Preliminaries

To begin, we define the data type of interest: *time series*.

Definition 18 (Time series)**.** A *time series* $T = t_1, t_2, \ldots, t_n$ is a sequence of real-valued numbers with length $= n$.

Since each entry t_i is one-dimensional, T is called a *univariate* time series. Definition 18 can be extended to define *multivariate* time series by using a vector with a number of dimensions greater than one for t_i. A multivariate time series T can also be viewed as a vector of univariate time series, where each univariate time series is called a *channel* of T.

The local properties of T can be analyzed through its *subsequences*.

Definition 19 (Subsequence)**.** A *subsequence* $T_{i,m}$ of a T is a sequence $t_{i:i+m-1} = t_i, t_{i+1}, \ldots, t_{i+m-1}$ that consists of a continuous subset of the entries from T of length m starting from position i.

The distance measure used is the z-normalized Euclidean distance, which computes the Euclidean distance between the two time series after they have been z-normalized to have a mean of zero and a standard deviation of one. For a time series T, the resulting z-normalized time series $\hat{T}$ would have entries $\hat{t}_i = (t_i - \mu(T))/\sigma(T)$, where $\mu(T)$ and $\sigma(T)$ are the mean and the standard deviation, of T respectively. For two time series X and Y, the z-normalized Euclidean distance $\text{dist}(X, Y)$ is defined as follows:

$$\text{dist}(X, Y) = \sqrt{\sum_{i=1}^{m} (\hat{x}_i - \hat{y}_i)^2} \tag{4.1}$$

$\text{dist}(X, Y)$ is bounded between zero and $2m$ [93].

4.3.1 Matrix Profile

We will first introduce the definitions of Distance Profile and Matrix Profile. Then, we will introduce the algorithms to compute them fast. Finally, we will focus on a specific variation of Matrix Profile, which is Left Matrix Profile. It is a main primitive to identify the left nearest neighbor for each predictor.

In brief, a matrix profile P is a time series that annotates an input time series T, storing the z-normalized Euclidean distance between each m-subsequence and its nearest neighbor in T.

Given a length m, we can select an m-subsequence from T as a query and compute its distances to all m-subsequences in T. The resulting ordered distance vector is a *distance profile*.

Definition 20 (Distance profile)**.** A *distance profile* $D_i = d_{i,1}, d_{i,2}, \ldots, d_{i,n-m+1}$ of a T is a vector of the distances between a given subsequence $T_{i,m}$ and each subsequences $T_{j,m}$ in T, where $d_{i,j}$ is the distance between $T_{i,m}$ and $T_{j,m}$, $1 \leq i, j \leq n - m + 1$.

Definition 21 (Matrix profile)**.** A *matrix profile* $P = \min(D_1), \min(D_2), \ldots, \min(D_{n-m+1})$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T.

Algorithm 6 The brute force approach to compute Matrix Profile

```
1: for i ← 1 to n − m + 1 do
2:     for j ← 1 to n − m + 1 do
3:         Compute the z-normalized Euclidean distance between t_{i,m} and t_{j,m}.
```

Another closely related time series of P, namely the Matrix Profile Index I, stores the location of the corresponding nearest neighbor.

Definition 22 (Matrix profile index)**.** A *matrix profile index* $I = I_1, I_2, \ldots, I_{n-m+1}$ of T is a vector of integers, where $I_i = j$ if $d_{i,j} = \min D_i$.

To retrieve a non-trivial result of the Matrix profile, we need to define the notion of an exclusion zone. For a given m-subsequence Q, the distance profile is zero at the location of Q (because Q must be a nearest neighbor of Q) and close to zero just before and after it. But the corresponding m-subsequences are trivial matches of Q. We need to define a zone, namely an exclusion zone, for this region. It is defined as $\lceil m/2 \rceil$ before and after the location of Q [17]. The community implementation uses $\lceil m/4 \rceil$ instead [94]. We use $\lceil m/4 \rceil$ as the exclusion zone in this study.

It may seem computationally expensive to perform Matrix Profile annotation at first glance. Algorithm 6 shows the brute force approach to compute the matrix profile. The two for-loops and the computation of z-normalized Euclidean distance, which takes $\mathcal{O}(m)$, indicate that the computational complexity is $\mathcal{O}(n^2 m)$. The space complexity is $\mathcal{O}(n^2)$ because of the pairwise distance of each subsequence with the other subsequence. However, the matrix profile can be computed in $\mathcal{O}(n^2)$ using an exact method, namely STOMP [17] or its community-open-sourced version, STUMP [94]. The main idea is using MASS [16] to compute the for-loop in line 2 in Algorithm 6 and reuse the computed results of D_{i-1} to compute D_i across the for-loop in line 1 because $t_{i-1,m}$ and $t_{i,m}$ differ only in the first point of $t_{i-1,m}$ and the last point of $t_{i,m}$ [15].

MASS

The computation of D_i may seem like costing $\mathcal{O}(nm)$, as depicted in lines 2-3 in Algorithm 6, where line 3 takes $\mathcal{O}(m)$. In fact, line 3 takes $2m$ arithmetic operations because the m-window is scanned twice, once for z-normalization and once for distance calculation. However, it can be computed in just $\mathcal{O}(n \log(n))$ by an algorithm called MASS [16]. We explain the main ideas of MASS here and direct readers to [16] for a more detailed description. We will start by explaining

the most basic component of MASS: just-in-time normalization. Equation 4.1 can be rewritten as follows.

$$\text{dist}(X, Y) = \sqrt{2m(1 - \frac{\sum_{i=1}^{m} x_i y_i - m\mu(x)\mu(y)}{m\sigma(X)\sigma(Y)})} \tag{4.2}$$

In line 3, the same query is comparing to other subsequences throughout the loop. Hence, we can perform z-normalization on the query once before the for-loop in line 2. Let's assume Y is the z-normalized query, hence $\mu(Y) = 0$ and $\sigma(Y) = 1$, and X is the non-z-normalized window. Equation 4.2 can then be reduced as follows.

$$\text{dist}(X, Y) = \sqrt{2m(1 - \frac{\sum_{i=1}^{m} x_i y_i}{m\sigma(X)})} \tag{4.3}$$

The z-normalization scan of X can be skipped in each iteration by following. The main idea is that $\sigma(X)$ of moving windows with size m can be calculated in one linear scan before entering the for-loop in line 2. It has been shown that the two cumulative sum vectors are sufficient to calculate μ and σ of any subsequence of any length at any iteration [17]. First, the cumulative sums of $C = \sum_{i=1}^{n} x$ and $C^2 = \sum_{i=1}^{n} x^2$ are calculated. The sums over any window $S_i = C_{i+m} - C_i$ and ${S^2}_i = {C^2}_{i+m} - {C^2}_i$ can be obtained by subtracting the two cumulative sums. Finally, σ of any window can be calculated in linear time using the following equation.

$$\sigma_i = \sqrt{\frac{{S^2}_i}{m} - (\frac{S_i}{m})^2} \tag{4.4}$$

The overall complexity of lines 2-3 in Algorithm 6 is still $\mathcal{O}(nm)$. However, since the z-normalization scan of X has been skipped, each iteration now takes m arithmetic operations only for calculating the dot product in the numerator of Equation 4.3.

The complexity of lines 2-3 can be further improved to $\mathcal{O}(n \log n)$ by using the fast fourier transform (FFT) to compute the dot products between the query and *all* subsequences $t_{j,m}$ [16]. The dot products are called sliding dot products because the windows X (i.e., $t_{j,m}$) are extracted through a sliding window across T. It is faster than $\mathcal{O}(nm)$ since $m > \log n$ holds in most cases. One of the strengths is that the complexity $\mathcal{O}(n \log n)$ does not depend on the length m of the subsequence. The above optimization is called MASS V1 and there are three other levels of optimization namely MASS V2, MASS V3, and MASS V4 to further reduce the running time [16].

Running time of Matrix Profile

The direct usage of MASS will make Algorithm 6 become $\mathcal{O}(n^2 \log n)$ [17]. Observe that the windows $t_{i,m}$ share most of the parts with $t_{i-1,m}$ except the first point of $t_{i,m}$ and the last point of $t_{i-1,m}$. Once we have the distance profile D_{i-1} for the subsequence $t_{i-1,m}$, the distance profile D_i for the next subsequence $t_{i,m}$ can be computed in just $\mathcal{O}(1)$. Hence, it results $\mathcal{O}(n^2)$ for computing the matrix profile [15].

Besides, the running time can be further sped up by parallelization for a single machine with multiple computation units, such as CPUs or GPUs [15].

4.3.2 Left Matrix Profile

In this study, instead of the general matrix profile (index), we are interested in the left version of it. The left version of the matrix profile informs us of the information about the left nearest neighbor of any m-subsequences in T [18]. It is shown in Figure 4.2. The left matrix profile has high values at the beginning because it can only find the left neighbor, and there are few left subsequences initially. In contrast, the right matrix profile has low values at the beginning because there are many right subsequences there. To note, each entry i in Matrix Profile P_i is simply the minimum of P_i^L and P_i^R (i.e., $P_i = \min(P_i^L, P_i^R)$).

Definition 23 (Left distance profile)**.** A *left distance profile* $D_i^L = d_{i,1}, d_{i,2}, \ldots, d_{i,i-\lceil m/4 \rceil - 1}$ of T is a vector of Euclidean distances between a given subsequence $T_{i,m}$ and each subsequence that appears before $T_{i,m}$. To note, $i - \lceil m/4 \rceil - 1$ is the index location of the last eligible subsequence before $T_{i,m}$ because of the exclusion zone.

Definition 24 (Left matrix profile)**.** A *left matrix profile* $P^L = \min(D_1^L), \min(D_2^L), \ldots, \min(D_{n-m+1}^L)$ of T is a vector of Euclidean distances between every subsequence $T_{i,m}$ of T and its nearest neighbor in T before it.

Definition 25 (Left matrix profile index)**.** A *left matrix profile index* I^L is a vector of integers $I_1^L, I_2^L, \ldots, I_{n-m+1}^L$ of T, where $I_i^L = j$ if $d_{i,j} = \min D_i^L$.

The computational complexity of the left matrix is the same as the general matrix profile, since the former one is just separating the computation of the later one into two halves, one for the left and one for the right, which is not considered in this study.

Table 4.1: Dataset Statistics. N: the number of time series in the dataset. n: the length of each time series T in the dataset. rate: measuring rate. start date: (date, time). h: the size of the forecasting window. T_{train} (T_{test}): the training (test) subsequence of T. To note, $|T_{\text{train}}| + |T_{\text{test}}| = n$.

	Data				**Forecasting Task**		
Dataset	N	n	rate	start date	h	$\lvert T_{\text{train}}\rvert$	$\lvert T_{\text{test}}\rvert$
Electricity [92]	70	26,136	hourly	2012-01-01, 00:00	24	25,968	168
Traffic [92]	90	10,560	hourly	2012-01-01, 00:00	24	10,392	168
PeMSD7 [92]	228	12,672	/5 mins	2012-05-01, 00:00	9	11,232	1,440
Exchange-Rate [69]	8	7,536	daily	1990-01-01, 00:00	24	6,048	1,488

4.3.3 Gradient boosting

Gradient boosting [95] is a boosting algorithm that ensembles a group of weak learners (usually decision trees) to make predictions. It sequentially adds learners to an ensemble, with each learner connecting its predecessor. It constructs weak learners in a way that each learner strategically corrects the predecessor's mistakes by fitting the new learner to the residual errors made by the predecessor [96, 97]. It can be used in classification and regression. In this study, we focus on its use in regression. For the usage in regression, the model is called "Gradient Boosting Regression Tree (GBRT)". Some popular optimized implementations of gradient boosting are XGBoost [98], CatBoost [99], and LightGBM [100]. In this study, we use XGBoost.

4.4 Materials and Methods

In this section, we first introduce the four datasets used in this study. We then formulate the time series forecasting problem and present the evaluation method, including the evaluation metrics. We then explain how to use a Gradient Boosting Regression Tree (GBRT) for forecasting by casting the time series into predictor-response pairs, as shown in Figure 4.1. Finally, we discuss how to leverage the nearest neighbor of a m-subsequence where the last data point is y_i to derive covariates for y_i. The derived covariates can improve the performance of the GBRT forecaster.

4.4.1 Datasets

Four datasets are used, as listed in Table 4.1. The time series are univariate and consist only of target values y_i, where $1 \leq i \leq n$. Each time series T is split

Table 4.2: Derived known input time-covariates with size M_u for each dataset.

Dataset	Derived known input time-covariates	M_u
Electricity	month, day, hour, day_of_week, day_of_year, week_of_year	6
Traffic	month, day, hour, day_of_week, day_of_year, week_of_year	6
PeMSD7	month, day, hour, minute, day_of_week, day_of_year, week_of_year	7
Exchange-Rate	month, day, day_of_week, day_of_year, week_of_year	5

into two contiguous time series, namely T_{train} and T_{test}. The N time series are considered as multiple, independent univariate time series instead of a multivariate time series with N channels.

Originally, each time point is not associated with any covariates (i.e., $M_x = M_u = 0$). Given a start date for the measurement of a time series T and its measuring rate, we can assign the (date, time) information for each y_i in T. For example, y_2 of a time series in `Traffic` is measured at 2012-01-01, 01:00. We can derive some covariates for y_i based on its (date, time) information. For example, six covariates can be derived for y_2 as follows. They are month, day, hour, day_of_week, day_of_year, and week_of_year. The corresponding values are 1, 1 (Day 1 of the month), 1 (where the first hour is 0), 6 (Sunday, where Monday is 0), 1, and 52 (Week 1 of 2012 begins on 2012-01-01 (Monday), so it is in Week 52 of 2011). These derived covariates are normalized to be from -0.5 to 0.5 [79]. They are called known inputs $\mathbf{u}_i$ because they are known even in the future (i.e., T_{test}). Table 4.2 shows the derived known inputs for the datasets.

4.4.2 Problem Formulation

Time series forecasting models predict h future values $y_{t+1}, y_{t+2}, \ldots, y_{t+h}$ of a target y at a given time point t. h is the number of steps we want to predict in the future w.r.t. t. The simplest case is one-step-ahead forecasting, where $h = 1$. It is preferable to predict multiple future points. In this case, it is called multi-horizon (i.e., multi-step) forecasting, where $h > 1$. For the actual value y_i, its predicted value is denoted as $\hat{y}_i$. A forecasting task is represented in Equation 4.5 [2].

$$\hat{y}_{t+\tau} = f(y_{t-w+1:t}, \boldsymbol{x}_{t-w+1:t}, \boldsymbol{u}_{t-w+1:t+\tau}, \tau) \tag{4.5}$$

where

- $\hat{y}_{t+\tau}$ is a prediction of the target value at $t + \tau$, where $\tau \in \{1, 2, \ldots, h\}$.
- $f(\cdot)$ is the prediction function learnt by the model.

- w is the length of a lookback window. The observations in the window are considered during forecasting.

- $y_{t-w+1:t}$ are the actual target values consisting of the current value y_t and the $w-1$ lag values before y_t. y_{t-i} is called the lag of i or i-lag.

- $\boldsymbol{x}_{t-w+1:t}$ are historical inputs. $\boldsymbol{x}_t$ is not known until time t. $\boldsymbol{x}_i$ is a vector of length M_x, where $t-w+1 \leq i \leq t$.

- $\boldsymbol{u}_{t-w+1:t+\tau}$ are known inputs. It is known for any $t \geq 1$. For example, date information, such as month and day. $\boldsymbol{u}_i$ is a vector of length M_u, where $t-w+1 \leq i \leq t+\tau$.

y_i is a scalar. $\mathbf{x}_i$ and $\mathbf{u}_i$ are vectors. $\mathbf{x}_i$ and $\mathbf{u}_i$ are called covariates of y_i.

We explain the task of forecasting using Equation 4.5. The model estimates the value of $y_{t+\tau}$, denoted by $\hat{y}_{t+\tau}$, with the aim to minimize the error function, typically represented as a function of $y_{t+\tau} - \hat{y}_{t+\tau}$ for each τ.

It is clear that the derived time-covariates from the date information are useful for forecasting when the target variable depends on the time of measurement. For example, if the target is the electricity consumption rate, there is a clear monthly pattern: consumption is higher in winter and summer, when heaters and air conditioners are used.

4.4.3 Evaluation Method

Given a dataset D of N time series T_i, where $1 \leq i \leq N$, the error made by the forecaster on D is simply the summation of errors made by the forecaster on each T_i. For each time series T, the trained model is tested on T_{test} using rolling forecasting. The prediction is always based on the ground truth, not on the model's prediction results. The model is tested for $|T_{\text{test}}|/h$ times for each T. In detail, the model first predicts the target values $\hat{y}_{t+1:t+h}$ in one batch, where $t = |T_{\text{train}}|$. The error is computed between the predicted target values $\hat{y}_{t+1:t+h}$ and the actual values $y_{t+1:t+h}$. For the second prediction, the input is the ground truth (i.e., $y_{t+1:t+h}$), rather than the predicted values (i.e., $\hat{y}_{t+1:t+h}$), and the output is $\hat{y}_{t+h+1:t+2h}$. For example, in `Traffic`, $|T_{\text{test}}| = 168$ and $h = 24$, the model is tested for $168/24 = 7$ times for each time series.

Rolling forecasting is used to prevent the accumulation of errors. This concept is also called "Teacher forcing" in RNN [101] because the ground truth (given by the "teacher") is fed into the forecaster when we roll the forecaster on T_{test} [102].

The intuition is that, suppose each question (except the first one) in an exam depends on the answers to the previous questions, rather than simply grading every answer in the end, a teacher would grade (evaluate) the answer once it is given by the student (i.e., the forecaster), and provide the correct answer (i.e., ground truth) to the student so he can answer the next question based on the correct answer.

To evaluate the forecaster, we used the following three evaluation metrics defined as:

- Root Mean Square Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{4.6}$$

- Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{4.7}$$

- Weighted Absolute Percentage Error (WAPE)

$$\text{WAPE} = \frac{\sum_{i=1}^{n}|y_i - \hat{y}_i|}{\sum_{i=1}^{n}|y_i|} \tag{4.8}$$

where n is the length of the time series, y_i and $\hat{y}_i$ are the true value and predicted value, respectively.

By rewriting Equation 4.8 to Equation 4.9, it is more obvious that it is a weighted absolute percentage error.

$$\text{WAPE} = \sum_{i=1}^{n} w_i \frac{|y_i - \hat{y}_i|}{|y_i|} \tag{4.9}$$

where the weights are given by

$$w_i = \frac{|y_i|}{\sum_{i=1}^{n}|y_i|} \tag{4.10}$$

RMSE and MAE are widely used metrics. WAPE was introduced by [103]. RMSE is sensitive to large prediction errors because it squares the error terms $(y_i - \hat{y}_i)$ before averaging them. They contribute significantly to the final RMSE. Hence, it is more sensitive to outliers. In contrast, MAE treats all errors linearly

because of the absolute sign operator. Hence, it is less sensitive to outliers. MAE can better reflect the actual error situation than RMSE [72]. MAE and WAPE are correlated. WAPE can be obtained by multiply MAE by n, followed by a division of the sum of the actual values $\sum_{i=1}^{n} |y_i|$. MAE is dependent on the scale of the data (i.e., different units used for the targets). WAPE provides a normalized percentage, as shown in Equation 4.9. It makes WAPE independent of the scale of the data. For all of them, a lower value is better.

4.4.4 Forecasting via GBRT

In order to apply GBRT into time series forecasting problem, we need to cast the input into an appropriate format to input into GBRT. The casting approach is similar to the successful time series forecasting models, which reconfigure the time series into the predictor-response pairs [79].

Figure 4.1 presents the casting. The window X_i with size $w \times (1 + M_x + M_u)$ is flattened into a 1D vector X_i' with length $w + M_x + M_u$. To note, only the covariates of the last target value y_i are kept [79]. By casting, we obtain the predictor-response pairs. In detail, the predictor X_i' is $y_{i-w+1}, y_{i-w+2}, \ldots, y_i, x_i^1, x_i^2, \ldots, x_i^{M_x} u_i^1, u_i^2, \ldots, u_i^{M_u}$. With this predictor-response formulation, the forecasting problem becomes a multi-output regression problem. The corresponding response is $y_{i+1}, y_{i+2}, \ldots, y_{i+h}$, with length h. As suggested in the original papers of the datasets, w is chosen to equal h.

Conventional XGBoost returns a single number instead of a sequence of numbers. We can treat a multi-output regression problem as a group of single-output regression problems. The forecasting task of predicting h steps ahead is treated as h individual regression problems. So, the final output is produced by h individual regressors. One may argue that the h regressors operate individually and hence the temporal relationship in the output sequence is lost. However, as the individual regressors are trained on the same flattened input X_i', the prediction Y_i would still preserve the temporal relationship [79].

We explain the above process through `Traffic` dataset, where $n = 10560$ (in which $|T_{\text{train}}| = 10392$), $h = 24$, $w = h = 24$, $M_x = 0$, and $M_u = 6$. There are $10392 - 48 + 1 = 10345$ predictor-response pairs for one time series, which are extracted by a sliding window of size 48 on T_{train}. The size of the sliding window is the sum of the length w of the window X_i and the length h of the output Y_i. $|X_i'| = w + M_x + M_u$. If the known inputs u are incorporated into the forecasting, $|X_i'| = 30$. Otherwise, $|X_i'| = 24$.

Algorithm 7 Feature Engineering via Matrix Profile

Input: Time series y, Time-covariate sequence u, Target value index i, Subsequence length m, Immediate subsequence length l

Output: Extracted covariates for y_i

Compute P^L and I^L for y with subsequence length m
$k \leftarrow i - m + 1$ ▷ Map index i to standard query start index
$idx_{\text{NN}} \leftarrow I^L[k]$ ▷ Start index of NN_i^L
$j \leftarrow idx_{\text{NN}} + m - 1$ ▷ Index of the last point of NN_i^L
$s \leftarrow P^L[k]$ ▷ Similarity
$end\_idx \leftarrow \min(j + l, i)$ ▷ To prevent data leakage
$NN_{\text{imm}} \leftarrow y[j + 1 : end\_idx]$ ▷ Immediate subsequence following NN_i^L
if $|NN_{\text{imm}}| < l$ **then**
    $NN_{\text{imm}} \leftarrow$ Pad NN_{imm} with NaN to length l
$NN_{\text{last}} \leftarrow y[j]$ ▷ Last target value of NN_i^L
$NN_{\text{tar}} \leftarrow y[idx_{\text{NN}} : j]$ ▷ All the target values of NN_i^L
$NN_{\text{cov}} \leftarrow u[j]$ ▷ Covariates at the last point of NN_i^L
return s, NN_{imm}, NN_{last}, NN_{tar}, NN_{cov}

4.4.5 Feature Engineering via Matrix Profile

We derive $\boldsymbol{x}_i$ using the information of the past nearest neighbor.

Recall that for each m-subsequence $T_{i,m}$ in a time series T, where m is user-given, we can identify its left nearest neighbor NN_i^L via the left matrix profile index P^L and retrieve the similarity between $T_{i,m}$ and NN_i^L via the left matrix profile I^L.

For each target value y_i, we construct a m-subsequence Q_i consisting of $y_{i-m+1:i}$. By using I^L, we can efficiently identify NN_i^L for each of these m-subsequences.

NN_i^L can reveal the historical reference of Q_i. History repeats itself. For example, since Q_i is similar to NN_i^L, the immediate subsequence of Q_i should be similar to that of NN^L, which tells the future trajectory after y_i. The immediate subsequence can be regarded as a guess or template for the future trajectory.

The similarity obtained via P^L can be regarded as the confidence of guessing such a future trajectory based on the immediate subsequence of Q_i.

A time series with different raw values can have the same z-normalized representation. Hence, a set of historical subsequences with the exact same z-normalized representation could correspond to different raw values. By extracting the raw values of the nearest neighbor, we restore this lost magnitude information and allow the GBRT to learn from it.

Once the location of NN_i^L is identified via the I^L, we extract the following information about NN_i^L as in Algorithm 7. They are used as $\boldsymbol{x}_i$ for y_i.

1. s: It is normalized to be from -0.5 to 0.5 as in time-derived covariates.
2. NN_{imm}: The subsequence of length l immediately following NN_i^L, where l is user-given.
3. NN_{last}: The last target values of NN_i^L.
4. NN_{tar}: All the m target values that make up NN_i^L itself.
5. NN_{cov}: The known input covariates of the last point of NN_i^L.

Note that the indexing systems differ between the matrix profile and the lookback window. In the matrix profile, an m-subsequence is indexed by its starting position, as shown in Figure 4.2. The gray boxes are indexed by their first points (the circles in the figures). The lookback windows are indexed by their last points (the triangles in the figures). Hence, for Q_i, which is a m-subsequences with y_i as its last point, we map i to the conventional starting index $k = i - m + 1$ to retrieve the correct nearest neighbor location and similarity score as in line 2 in Algorithm 7.

To prevent data leakage when extracting the immediate subsequence with length l, we enforce a boundary in line 6. For the retrieved left nearest neighbor NN_i^L which ends at index j, we restrict the extraction of its immediate subsequence to ensure that no future target values (i.e, those beyond index i) are accessed by truncating NN_{imm} at $\min(j + l, i)$ in line 7. Because GBRT requires the predictor vector to have a fixed size, NN_{imm} is padded with missing values (NaN) if it is shorter than l.

To note, because of the exclusion zone, the initial subsequences do not have left nearest neighbor as shown in the second row of Figure 4.2. Hence, their derived covariates would only contain missing values. Fortunately, XGBoost inherently supports missing values via its sparsity-aware split finding algorithm.

GBRT utilizes the baseline features, including the target values and the known input time-covariates [79]. To note, we can also only consider the target values but not the time-covariates in the forecasting. By integrating the extracted information, we propose several variants of GBRT as summarized in Table 4.3. M_x indicates the number of derived features from the nearest neighbor information. M_x = Number of ✓ $- 1$.

It is important to note how these new features are integrated into the GBRT model. The covariates as shown in Figure 4.1 are scalars. But the extracted

Table 4.3: Summary of the proposed models and their incorporated covariates. $\mathbf{NN}_{\text{imm}}$: the immediate subsequence of NN. $\mathbf{NN}_{\text{last}}$: the last target value of NN. $\mathbf{NN}_{\text{tar}}$: all the target values of NN. $\mathbf{NN}_{\text{cov}}$: time-covariates of the last point of the NN. Details of the incorporated covariates are in Algorithm 7.

Method	**Base Features**	$\mathbf{NN}_{\text{imm}}$	$\mathbf{NN}_{\text{last}}$	$\mathbf{NN}_{\text{tar}}$	$\mathbf{NN}_{\text{cov}}$	M_x
GBRT	✓					0
GBRT-NN	✓	✓				1
GBRT-NN^{+}	✓	✓	✓			2
GBRT-NN^{++}	✓	✓		✓		2
GBRT-NNC	✓	✓			✓	2
GBRT-NNC^{+}	✓	✓	✓		✓	3
GBRT-NNC^{++}	✓	✓		✓	✓	3

Note: Appending -S to any method name additionally incorporates the similarity s into the models.

features NN_{imm}, NN_{tar}, and NN_{cov} are vectors of length l, m, and M_u, respectively. To satisfy the 1D input requirement of the GBRT model, these vectors are flattened and appended to the predictor vector during the data casting step. In detail, each data point within the vector is treated as an independent scalar feature and concatenated to the predictor vector X_i' directly, instead of being encapsulated in the corresponding vector $\mathbf{x}_i$. For example, for GBRT-NN is $w + M_x + l$, given $NN_{\text{imm}} = t_{j+1}, t_{j+2}, \ldots, t_{j+l}$, it results the predictor $y_{i-w+1}, y_{i-w+2}, \ldots, y_i, t_{j+1}, t_{j+2}, \ldots, t_{j+l}, u_i^1, u_i^2, \ldots, u_i^{M_u}$ with length $w + M_x + l$. Each individual points in the immediate subsequence are appended directly to the predictor vector.

4.4.6 Extends to k-Nearest Neighbors

We further extend our methodology to incorporate features from the k-nearest neighbors (k-NN) rather than just the nearest neighbor (1-NN) of each predictor.

During the extraction process over the Left Distance Profile (D_i^L), instead of retaining only the lowest distance value, We iteratively identify the k lowest distance values as follows. We would like to retrieve the k-nearest neighbors without overlapping as defined by the exclusion zone. First, we locate the index of the minimum value in D_i^L to identify the first nearest neighbor. Before the next iteration, we mask the exclusion zone surrounding this selected index. We repeat this process to identify the remaining $k - 1$ minima. Once the locations of the k non-overlapping nearest neighbors are determined, we extract their respective

information (s, NN_{imm}, NN_{last}, NN_{tar}, NN_{cov}) and flatten them into scalar features similar to what we did for the nearest neighbor case.

To denote the variants utilizing k-nearest neighbors, we update the name by prefixing the "NN" with the integer k in the method name. For example, when $k = 2$, the GBRT-NN is denoted as GBRT-2NN.

4.5 Experiments

This section presents experiments demonstrating the effectiveness of using nearest neighbor information to improve GBRT performance on the time series forecasting task.

The experiments are conducted on a personal laptop equipped with an Apple M1 Pro chip and 16 GB of memory, using Python for implementation. We employ `stumpy` for the matrix profile implementation and `xgboost` for the XGBoost implementation. The code and data are available at `https://github.com/colemanyu/matrix-profile-motif-forecasting`.

The evaluation is conducted on four datasets, as listed in Table 4.1. Three evaluation metrics are used, namely RMSE, MAE, and WAPE. There are two scenarios for the datasets' use case.

- Without known input time-covariates $\boldsymbol{u}_i$ (i.e. $M_u = 0$): The models rely solely on the target values.
- With known input time-covariates $\boldsymbol{u}_i$ (i.e. $M_u > 0$): The models incorporate additional time-derived known input covariates $\boldsymbol{u}_i$, as listed in Table 4.2

Table 4.4: Experimental Results without known input time-covariates.

Dataset	Metric	GBRT	GBRT-NN	GBRT-NN-S	GBRT-NN$^+$	GBRT-NN$^+$-S	GBRT-NN^{++}	GBRT-NN^{++}-S
Electricity	RMSE	**136.254**	139.717	138.249	143.920	141.314	140.481	140.920
	WAPE	0.103	0.100	**0.099**	0.101	0.100	0.100	**0.099**
	MAE	57.929	55.862	55.646	56.821	55.744	55.988	**55.641**
Traffic	RMSE	0.012	0.016	0.008	0.015	**0.007**	0.015	0.008
	WAPE	0.108	0.080	0.037	0.074	**0.036**	0.063	**0.036**
	MAE	0.006	0.004	**0.002**	0.004	**0.002**	0.003	**0.002**
PeMSD7	RMSE	5.607	5.572	5.555	5.569	5.551	5.562	**5.545**
	WAPE	**0.051**	**0.051**	**0.051**	**0.051**	**0.051**	**0.051**	**0.051**
	MAE	2.979	2.960	2.953	2.958	2.950	2.955	**2.947**
Exchange-Rate	RMSE	0.020	**0.018**	**0.018**	0.019	0.019	0.021	0.021
	WAPE	0.016	**0.015**	**0.015**	**0.015**	**0.015**	0.016	0.016
	MAE	0.012	**0.011**	**0.011**	**0.011**	**0.011**	0.012	0.012

GBRT is used as our baseline for both scenarios. The results are in Tables 4.4, 4.5. In the table of experimental results, **bold** represents the best result, and

Table 4.5: Experimental Results with known input time-covariates.

Dataset	Metric	GBRT	GBRT-NN	GBRT-NN-S	GBRT-NN$^+$	GBRT-NN$^+$-S	GBRT-NN^{++}	GBRT-NN^{++}-S
Electricity	RMSE	132.669	129.102	125.436	128.999	**120.542**	129.540	133.639
	WAPE	0.093	0.090	0.090	0.090	**0.088**	0.091	0.091
	MAE	52.023	50.147	50.134	50.221	**49.396**	50.850	50.982
Traffic	RMSE	0.014	0.018	0.010	0.018	**0.009**	0.018	**0.009**
	WAPE	0.109	0.108	0.064	0.103	0.062	0.096	**0.060**
	MAE	0.006	0.006	**0.003**	0.006	**0.003**	0.005	**0.003**
PeMSD7	RMSE	5.195	5.166	5.157	5.162	5.154	5.156	**5.149**
	WAPE	0.048	0.048	0.048	0.048	**0.047**	0.048	**0.047**
	MAE	2.795	2.776	2.773	2.776	2.769	2.772	**2.768**
Exchange-Rate	RMSE	0.020	**0.018**	**0.018**	0.019	0.019	0.021	0.021
	WAPE	0.016	**0.015**	**0.015**	**0.015**	**0.015**	0.016	0.016
	MAE	0.012	**0.011**	**0.011**	**0.011**	0.012	0.012	0.012

underlined represents the second best. We first evaluate two primary variants, GBRT-NN and GBRT-NN-S, for initial testing.

4.5.1 Effect of the length l of the immediate subsequence

The effect of the length l of the immediate subsequence is investigated. We tested for five different lengths $l \in \{1, 2, \lceil h/4 \rceil, \lceil h/2 \rceil, h\}$. For PeMSD7 with $h = 9$, $l \in \{1, 2, 3, 5, 9\}$. For other datasets with $h = 24$, $l \in \{1, 2, 6, 12, 24\}$. The results

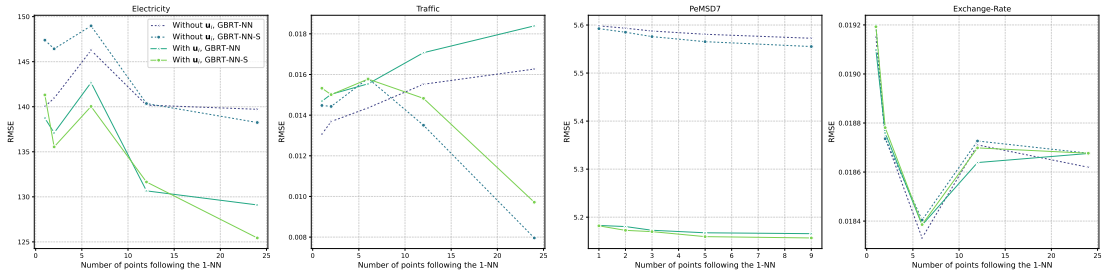


Figure 4.3: Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on RMSE.

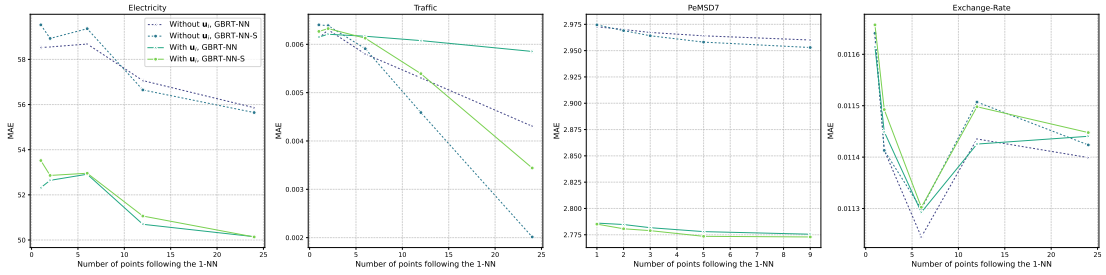


Figure 4.4: Effect of different lengths l of the immediate subsequence (i.e., Number of points following the 1-NN) on MAE.

are plotted in Figures 4.3, 4.4. Since the WAPE plot is similar to the MAE plot, only the RMSE and WAPE plots are shown here. The findings are as follows.

- For `Electricity` and `PeMSD7`, the performance in general improves for all the metrics as l increases.
- For `Traffic`, the performance of GBRT-NN-S on RMSE improves as l increases. In contrast, the performance of GBRT-NN on RMSE decays as l increases. But on MAE, both models improve as l increases.
- For `Exchange-Rate`, both models achieve the best result when $l = 6$ for all the metrics.

4.5.2 Comparison between the two scenarios

We compare the performance of the models on the two different scenarios (i.e., whether incorporating time-derived known input covariates $\boldsymbol{u}_i$).

- For `Electricity` and `PeMSD7`, the models perform better when time-covariates are incorporated.
- For `Traffic`, incorporating the covariates worsens the results.
- For `Exchange-Rate`, the performances of the models are similar between the two scenarios. This indicates that this dataset is less dependent on time-covariates $\boldsymbol{u}_i$ (i.e., date information).

4.5.3 Effect of using z-normalized instead of raw values

In this part, we study the effect of using z-normalized target values of the immediate subsequences. By default, we use the target values directly as the features. We have tested using the z-normalized representation of the target values instead of the raw values in all the previous experiments. In detail, for each dataset, there are two scenarios, and the two models have been tested five times for five different l. It results in 20 combinations for each dataset in the experimental setting. The percentage change of using the z-normalized representation instead is shown in Figure 4.5 It shows that using the raw values of the immediate subsequence is preferable.

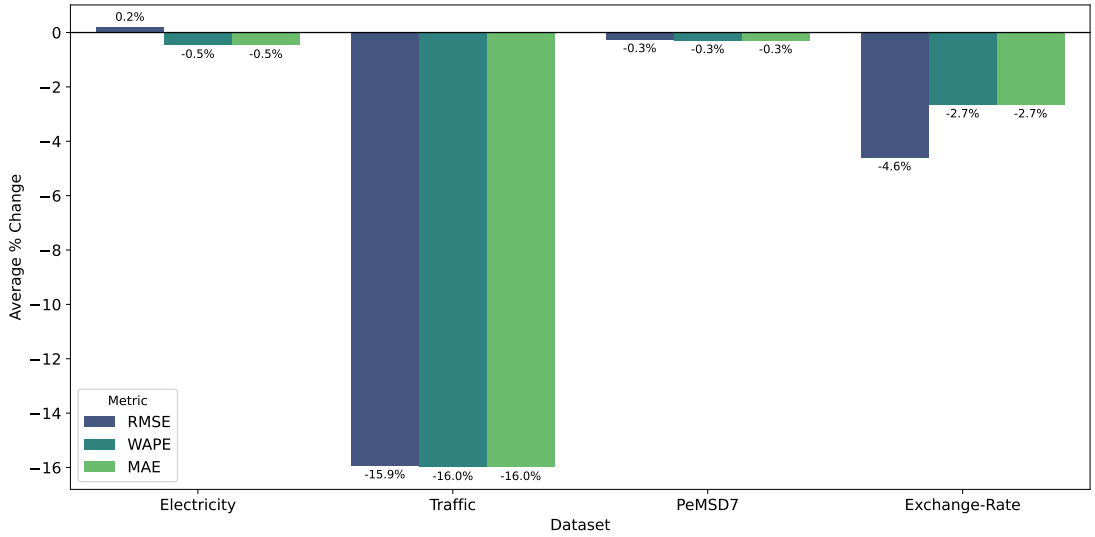


Figure 4.5: Using the z-normalized values instead of the raw values.

4.5.4 Effect of using pairwise difference representation instead of raw values

In this part, we study the effect of using pairwise differences, which captures the trend, as the features. We have tested using the pairwise difference representation of the target values instead of the raw values in all the previous experiments. For a given time series $T = t_1, t_2, \ldots, t_n$, its pairwise differences representation T' refers to $T' = t_2 - t_1, t_3 - t_2, \ldots, t_n - t_{n-1}$. To note, the first entry is computed by subtracting the first entry of the immediate subsequence from the last entry of the nearest neighbor. Hence, the length of the pairwise difference representation is the same as the original one. Same as the previous part, it has been tested for 20 combinations for using the pairwise difference representation instead. The result is plotted on Figure 4.6. It shows that again using the raw values of the immediate subsequence is preferable.

4.5.5 Summary of the experiments on the two primary models

Based on the above experiments, we present the following summary for the downstream analysis.

1. **Lag Selection**: We set $l = 6$ for `Exchange-Rate` and $l = h$ for other datasets for the downstream analysis.

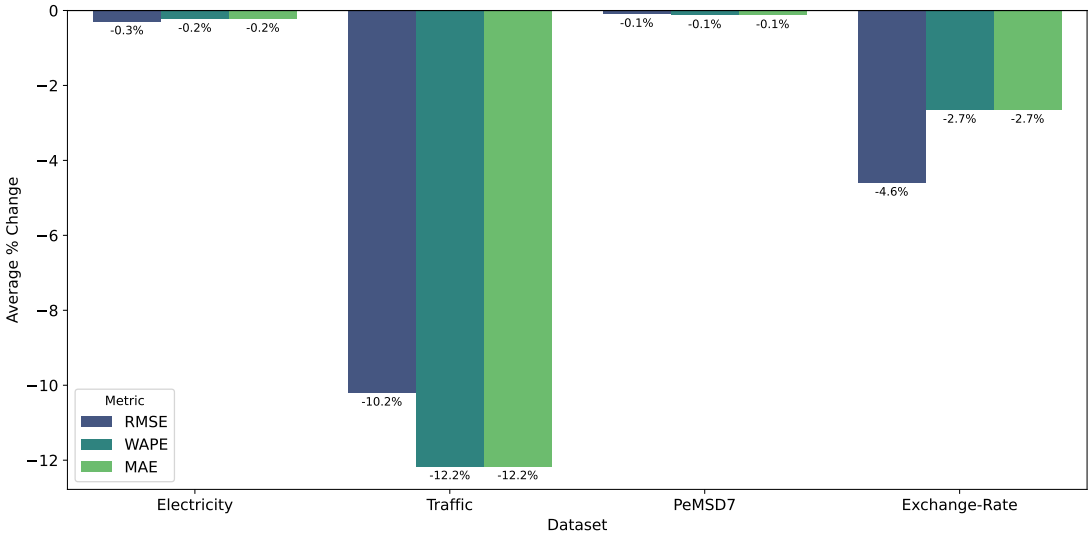


Figure 4.6: Using the pairwise differences instead of the raw values.

2. **Variable Impact of time covariates**: The inclusion of time-covariates $\boldsymbol{u}_i$ does not always help. It depends on the dataset. Our models only have a mild improvement on `Exchange-Rate`. It indicates that this dataset is not strongly dependent on historically similar occurrences. It can also be noted by the small best l (i.e., $l = 6$ given $h = 24$).

3. **Data Representation**: Using raw values for the immediate subsequences. We argue that it is because raw values preserve the original information.

4. Across 12 evaluation combinations (four datasets with three metrics), GBRT-NN-S achieved superior or consistent performance:

   (a) Excluding time-covariates: 11 out of 12 cases showed improvement (10) or consistency (1).

   (b) Including time-covariates: 100% of cases showed improvement (11) or consistency (1).

5. Including similarity as a covariate usually improves the results. In particular, GBRT-NN-S performs much better than GBRT-NN in `Traffic`. The evaluation metrics are (RMSE, WAPE, MAE). Without known input-time covariates, the improvement is from (0.016, 0.080, 0.004) to (**0.008**, **0.037**, **0.002**). With known input-time covariates, the improvement is from (0.018, 0.108, 0.006) to (**0.010**, **0.064**, **0.003**).

4.5.6 Incorporating the target values of the nearest neighbor

We have further incorporated the target values (either only the last point (i.e., GBRT-NN^{+}) or all of them (i.e., GBRT-NN^{++})) of the nearest neighbor. The results are in Tables 4.4, 4.5. By incorporating these features, the models improve on `Traffic` and `PeMSD7`. GBRT-NN^{++}-S performs the best for both the scenarios for these two datasets except on RMSE on `Traffic`.

For `Electricity`, incorporating the target values worsens the results, compared to the simpler models. But it is still better than the base line model GBRT, except for the case without known input-time covariates on RMSE. It indicates that, in the absence of time covariates, they (GBRT-NN^{+}-S, GBRT-NN^{++}-S) make incorrect predictions for some outliers.

Same as before, `Exchange-Rate` is insensitive to information of historically similar occurrences.

4.5.7 Incorporating the time-covariates of the last point of the nearest neighbor

Table 4.6: Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors.

Dataset	Metric	GBRT-NNC	GBRT-NNC-S	GBRT-NNC^{+}	GBRT-NNC^{+}-S	GBRT-NNC^{++}	GBRT-NNC^{++}-S
`Electricity`	RMSE	127.019	127.972	128.970	128.282	138.356	**123.644**
	WAPE	**0.089**	0.090	0.090	**0.089**	0.092	**0.089**
	MAE	50.061	50.383	50.387	50.076	51.804	**49.929**
`Traffic`	RMSE	0.014	0.010	0.014	**0.009**	0.014	0.010
	WAPE	0.093	0.069	0.092	**0.066**	0.086	0.069
	MAE	0.005	**0.004**	0.005	**0.004**	0.005	**0.004**
`PeMSD7`	RMSE	5.149	5.139	5.144	5.136	5.139	**5.133**
	WAPE	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**
	MAE	2.769	2.762	2.765	**2.760**	2.764	**2.760**
`Exchange-Rate`	RMSE	**0.018**	**0.018**	0.019	0.019	0.021	0.021
	WAPE	**0.015**	**0.015**	**0.015**	**0.015**	0.016	0.016
	MAE	**0.011**	**0.011**	**0.011**	0.012	0.012	0.012

We also incorporate the covariates of the last point in the nearest-neighbor. The results are shown in Table 4.6. `PeMSD7` achieves better performance in these models and reaches the best for using GBRT-NNC^{++}-S. It worsens performance on `Electricity` and `Traffic`. It does not improve performance on `Exchange-rate`.

Table 4.7: Experimental Results without known input time-covariates in the k-nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NN-S	GBRT-3NN-S	GBRT-2NN$^+$-S	GBRT-3NN$^+$-S	GBRT-2NN^{++}-S	GBRT-3NN^{++}-S
Electricity	RMSE	153.633	**153.251**	153.282	154.697	157.044	153.437
	WAPE	0.104	0.104	**0.103**	0.104	0.104	**0.103**
	MAE	58.023	58.143	57.659	58.340	58.082	**57.651**
Traffic	RMSE	**0.008**	0.009	**0.008**	0.009	**0.008**	**0.008**
	WAPE	0.039	0.039	0.038	0.039	**0.037**	0.038
	MAE	**0.002**	**0.002**	**0.002**	**0.002**	**0.002**	**0.002**
PeMSD7	RMSE	5.534	5.527	5.531	5.524	5.524	**5.511**
	WAPE	**0.050**	**0.050**	**0.050**	**0.050**	**0.050**	**0.050**
	MAE	2.942	2.938	2.939	2.936	2.935	**2.928**
Exchange-Rate	RMSE	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**
	WAPE	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**
	MAE	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**

Table 4.8: Experimental Results with known input time-covariates in the k-nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NN-S	GBRT-3NN-S	GBRT-2NN$^+$-S	GBRT-3NN$^+$-S	GBRT-2NN^{++}-S	GBRT-3NN^{++}-S
Electricity	RMSE	143.698	144.162	141.560	**138.940**	143.250	143.882
	WAPE	0.094	0.095	0.094	**0.093**	**0.093**	0.094
	MAE	52.929	53.020	52.380	**52.159**	52.334	52.699
Traffic	RMSE	0.010	**0.009**	0.011	0.010	0.010	0.010
	WAPE	0.063	0.061	0.067	0.063	**0.058**	0.060
	MAE	**0.003**	**0.003**	0.004	**0.003**	**0.003**	**0.003**
PeMSD7	RMSE	5.137	5.133	5.138	5.130	5.130	**5.118**
	WAPE	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**
	MAE	2.762	2.760	2.761	2.759	2.758	**2.751**
Exchange-Rate	RMSE	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**
	WAPE	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**
	MAE	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**

Table 4.9: Experimental Results with known input time-covariates for models incorporating the known input time-covariates of the last point of the nearest neighbors in the k-nearest neighbors case, where $k \in \{2, 3\}$.

Dataset	Metric	GBRT-2NNC-S	GBRT-3NNC-S	GBRT-2NNC$^+$-S	GBRT-3NNC$^+$-S	GBRT-2NNC^{++}-S	GBRT-3NNC^{++}-S
Electricity	RMSE	146.325	144.291	144.578	141.405	137.734	**137.570**
	WAPE	0.094	0.095	0.094	0.094	0.093	**0.092**
	MAE	52.791	52.986	52.662	52.547	52.000	**51.631**
Traffic	RMSE	**0.011**	0.012	0.013	0.012	0.012	**0.011**
	WAPE	0.076	0.079	0.086	0.081	0.079	**0.072**
	MAE	**0.004**	**0.004**	0.005	**0.004**	**0.004**	**0.004**
PeMSD7	RMSE	5.113	5.103	5.113	5.104	5.110	**5.095**
	WAPE	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**	**0.047**
	MAE	2.747	2.741	2.746	2.740	2.745	**2.739**
Exchange-Rate	RMSE	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**	**0.019**
	WAPE	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**	**0.016**
	MAE	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**	**0.012**

Table 4.10: Best models for each dataset with (RMSE, WAPE, MAE) respectively.

Dataset	Without time-covariates	With time-covariates
`Electricity`	GBRT (**136.254**, 0.103, 57.929), GBRT-NN^{++}-S (140.920, **0.099**, **55.641**)	GBRT-NN^{+}-S (**120.542**, **0.088**, **49.396**)
`Traffic`	GBRT-NN^{+}-S (**0.007**, **0.036**, **0.002**)	GBRT-NN^{+}-S (**0.009**, 0.062, **0.003**), GBRT-2NN^{++}-S (0.010, **0.058**, **0.003**)
`PeMSD7`	GBRT-3NN^{++}-S (**5.511**, **0.050**, **2.928**)	GBRT-3NNC^{++}-S (**5.095**, **0.047**, **2.739**)
`Exchange-Rate`	GBRT-NN (**0.018**, **0.015**, **0.011**)	GBRT-NN (**0.018**, **0.015**, **0.011**)

4.5.8 Extend to the case of k-nearest neighbors

Our idea has been extended to the case for k-nearest neighbors. We have tested for $k \in 2, 3$ instead of $k = 1$, which has already demonstrated in the previous parts. The results are shown in Tables 4.7, 4.8, 4.9.

4.5.9 Summary of the experiments

Table 4.10 summarizes the top-performing models for each dataset. To note, for the `Electricity` and `Traffic` datasets, no single model consistently outperforms the others across all three metrics. In this case, we report all models that achieve the best performance in at least one metric. When ties occur, we prioritize simple models (i.e., those with fewer covariates).

4.5.10 Impact of the Exclusion Zone and the Search Space Size

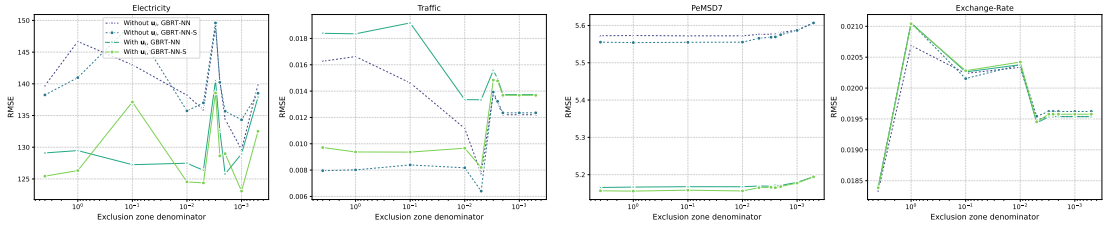


Figure 4.7: Effect of different denominators d of the exclusion zone (defined as $\lceil m/d \rceil$, where m is the subsequence length) on RMSE.

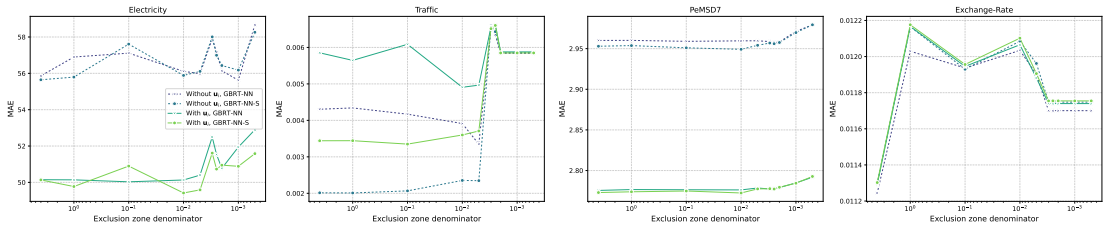


Figure 4.8: Effect of different denominators d of the exclusion zone (defined as $\lceil m/d \rceil$, where m is the subsequence length) on MAE.

Recall that the exclusion zone is defined as $\lceil m/d \rceil$, where the default value of the denominator d is 4. By manipulating d, we can adjust the exclusion zone to further limit the historical search space available for identifying the left nearest neighbor. To evaluate the sensitivity of the model to the size of this search space, we tested for ten different values of $d \in \{4, 1, 0.1, 0.01, 0.005, 0.003, 0.0025, 0.002, 0.001, 0.0005\}$. These values corresponds to 1/4, 1, 10, 100, 200, 333, 400, 500, 1000, and 2000 times the subsequence length m for the exclusion zone, respectively. The results are shown in Figures 4.7, 4.8.

As shown in the figures, we observed a general pattern. When the denominator d is exceptionally small, the resulting exclusion zone becomes overwhelmingly large. This drastically shrinks the pool of available historical candidates, forcing the models to select less similar left nearest neighbors. As a result, the predictive quality of the derived covariates of the neighbors decays and the error metrics increase.

Interestingly, we also observed that for certain datasets (`Electricity` and `Traffic`), moderately limiting the search space actually improves forecasting performance of some models. This implies that different datasets possess unique optimal historical ranges. By expanding the exclusion zone, the model is forced to ignore recent temporal nearest neighbor and instead identify more robust, long-term repeating nearest neighbor, which serve as better historical templates (i.e., lookahead) for the forecaster.

4.5.11 Decoupling Subsequence length from Lookback Window length

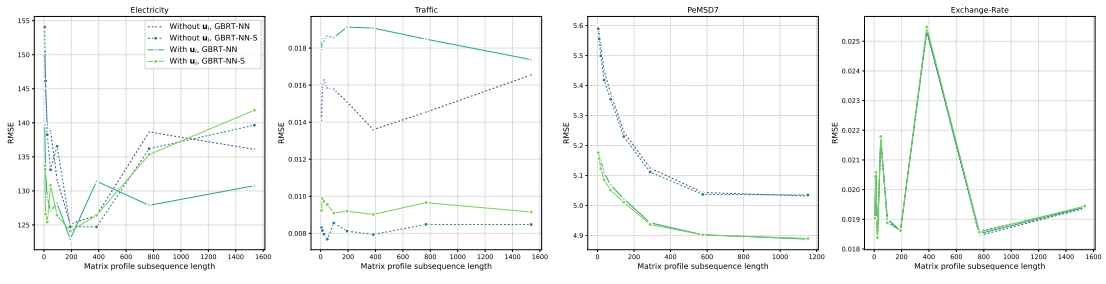


Figure 4.9: Effect of varying the matrix profile subsequence length m on RMSE.

In the above configuration of our proposed models, the subsequence length m used by the Matrix Profile to find historical nearest neighbors is set equal to the length of the lookback window w used by the GBRT forecaster. However, to evaluate

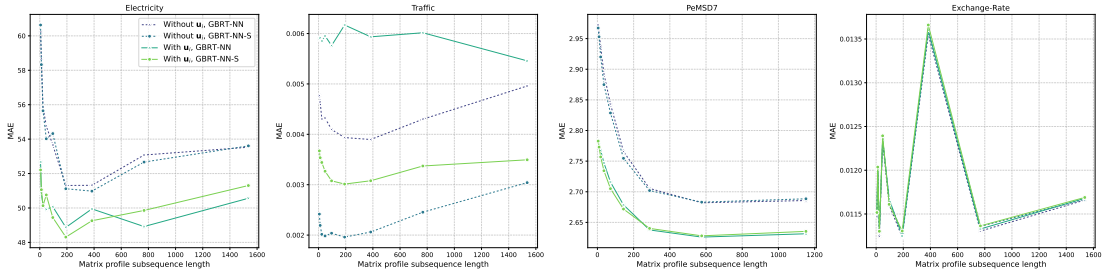


Figure 4.10: Effect of varying the matrix profile subsequence length m on MAE.

the sensitivity of m, we investigated the effect of decoupling these two parameters by testing for nine different values of $m \in \{\lfloor w/2 \rfloor, w, 2w, 4w, 8w, 16w, 32w, 64w, 128w\}$. The results are shown in Figures 4.9, 4.10.

For datasets other than `Exchange-Rate`, when we increase m to certain degree, the performance of the forecaster has been improved. The improvement is obvious in `PeMSD7` dataset. The improvement highlights that w and m play different roles in our models. The length of the lookback window w dictates the short-term tactical context required by the regression trees to map predictors to responses, as shown in Figure 4.1. In contrast, the subsequence length m defines the length of the "memory" required to identify a valid historical occurrence (i.e., left nearest neighbor). When m is small, the Matrix Profile may identify a historical neighbor that matches the immediate short-term trend but belongs to a completely different long-term trend. By using a larger m, the models require candidate historical occurrence to match a longer, broader dynamic sequence. This stricter matching requirement filters out spurious short-term matches, ensuring that the derived covariates are drawn from a genuinely comparable historical occurrence, thereby generating higher-quality covariates for the forecaster and improving the forecaster.

4.6 Discussion

4.6.1 "Lookahead" covariates help forecasting

The experimental results consistently demonstrate that leveraging nearest neighbor information significantly improves the predictive accuracy of GBRT models. By identifying a historical subsequence of a nearest neighbor that is most similar to the current lookback window, the model gains access to a lookahead/template of what happened next in a similar past situation. This addresses a fundamental limitation of the standard sliding-window regression approach to forecasting, in which predictions are restricted to observations within the immediate lookback

window (see Figure 4.1). A simple model that incorporates only NN_{imm} and s, namely GBRT-NN-S, always improves forecasting accuracy across all datasets, with few exceptions. For PeMSD7, incorporating information about the nearest neighbor itself (i.e., NN_{last}, NN_{tar}, and NN_{cov}) improves the result.

The included similarity s can be treated as a confidence measure for the "Lookahead" covariates. NN_{last}, NN_{tar}, and NN_{cov} provides more information about the similar past situation.

4.6.2 Raw values vs. other representations

While Matrix Profile requires z-normalization for subsequence similarity computation, using the raw values of the immediate subsequence allows the model to recover lost magnitude information.

Pairwise differences are used for capturing trends. However, the experiments show that this representation discards essential information required by GBRT to map predictors to responses effectively.

4.6.3 Include k-nearest neighbors information

Expanding the feature set to include covariates derived from k-nearest neighbors ($k = 2, 3$) can improve the performance for datasets, namely Electricity and PeMSD7. It can be seen from Table 4.10 that models for k-nearest neighbors such as GBRT-2NN^{++}-S, GBRT-3NN^{++}-S and GBRT-3NNC^{++}-S appeared as the best models. This suggests that the lookaheads from multiple historically similar occurrences provides a more robust predictor than a single neighbor, although it increases the dimensionality of the input vector. To note, in Electricity, the best models are GBRT and GBRT-NN for RMSE in both scenarios, respectively. It suggests that the above models can improve overall performance but cannot predict some outliers, resulting in large error terms ($y_i - \hat{y}_i$), which RMSE is sensitive to.

4.7 Conclusion

In this study, we present a method to augment the original time series by finding nearest neighbors and using their information as covariates for each subsequence. The information includes the immediate subsequence, similarity, constituent points, and the covariates of the nearest neighbor. It addresses a fundamental limitation of the standard sliding-window regression approach to forecasting, in which predictions

are restricted to observations within the immediate lookback window. Experiment shows that the augmented data allow the GBRT model to have a better prediction result in the task of time series forecasting in terms of three evaluation metrics, which are RMSE, WAPE, and MAE.

4.7.1 Future Work

Leveraging nearest neighbors' location information: It would be beneficial to retrieve the nearest neighbors for each subsequence in a specific range with respect to it. Real-world applications often require the separation of information of short-term and long-term repeating patterns for making accurate predictions [69]. Notably, the matrix profile also provides the locations of the nearest neighbors from the matrix profile index. Using this location (index) information, we can retrieve the nearest neighbors of each subsequence in a specified range with respect to it to find those "short-term" and "long-term" patterns.

Extend to multidimensional case: This study focuses on univariate time series forecasting, where there is a single target and no exogenous inputs. It only requires us to find one-dimensional nearest neighbors. When there are multiple targets or a single target with multiple exogenous inputs, we need to identify multidimensional nearest neighbors [104] and leverage their information for forecasting.

Identify outliers of immediate subsequences: When we have a set of immediate subsequences, as in the case of k-nearest neighbors, we can determine whether an immediate subsequence is an outlier or not by comparing it with others. Or we can compute the consensus immediate subsequences of them. In addition, we can also consider the similarity of each of the nearest neighbors as the weight of their corresponding immediate subsequences when computing the consensus immediate subsequences.

Chapter 5

MTSCCleav: a Multivariate Time Series Classification (MTSC)-based Method for Predicting Human Dicer Cleavage Sites

MicroRNAs (miRNAs) are small non-coding RNAs (ncRNAs) that regulate gene expression at the post-transcriptional level, thereby playing essential roles in diverse biological processes. The biogenesis of miRNAs requires Dicer to cleave at specific sites on the precursor miRNAs (pre-miRNAs). Several machine learning approaches have been proposed to predict whether an input sequence contains a cleavage site. However, they rely heavily on complex feature engineering or opaque deep neural networks. It results in a lack of generalizability and a long running time. There is a need for an alternative modeling paradigm that is accurate, fast, and simple.

We propose an approach to frame the task as a multivariate time series classification problem. Nine encoding methods have been proposed to convert the RNA sequence into a time series. The predicted secondary structure is also converted to a time series. We also leverage the probabilities of the base pairs in the predicted secondary structure. Computational experiments demonstrate that our proposed method can achieve better or comparable results in terms of using a simpler, more intuitive model and less computational time. It achieves 3.7X to 28.8X speedup. Through perturbation experiments, we found that regions close to the center of pre-miRNAs are essential for predicting human Dicer cleavage sites.

By transforming the RNA sequence and its secondary structure information into a multivariate time series and utilizing simple, state-of-the-art time series classifiers, we achieved comparable or even superior performance in a simpler and faster manner.

Code is available at: `https://github.com/colemanyu/time-series-classification-cleavage`.

5.1 Background

One of the most important theories in molecular biology is the central dogma. It depicts the flow of genetic information [105, 106]. Proteins are the functional units. The information stored in DNA is used to create them. Genes (segments) in DNA are used as templates for messenger RNAs (mRNAs) synthesis. An mRNA acts as a set of instructions to assemble a chain of amino acids, which form a linear polypeptide. To become biologically active, this chain is folded into a specific 3D structure, a proper configuration that enables it to perform its desired functions. This folded polypeptide is called a functional protein, or simply a protein. This entire process closely resembles how a computer program runs on a machine. The source code does not function by itself. First, it is translated into an assembly code (a lower-level, less human-readable form) and then into an executable file that can actually perform the intended tasks [107].

These mRNAs are called "coding RNAs" because they code for proteins. There are other genes in which the final product is the RNA molecule itself. They are called non-coding RNAs (ncRNAs). Two types of small ncRNAs are particularly important. They are microRNAs (miRNAs) and small interfering RNAs (siRNAs). Their discovery was recognized with the 2006 Nobel Prize in Physiology or Medicine[1], awarded for work completed only eight years prior [105].

In this study, we focus on miRNAs. An miRNA can regulate the expression of several proteins. Hence, understanding the biogenesis of miRNAs is of great value. It involves the processing of primary miRNAs (pri-miRNAs). RNAs are 3D molecules. While the 1D sequence of these molecules can be easily obtained by sequencing, it is hard to measure the 3D structure (tertiary structure) from the experiment and predict it directly from the 1D sequence. Many relevant properties of the 3D structure are typically investigated by analyzing the 1D sequence and the predicted 2D structure derived from it, known as secondary structure.

[1]The Nobel Prize in Physiology or Medicine 2006 - NobelPrize.org: https://www.nobelprize.org/prizes/medicine/2006/summary/ (Accessed on: 2025-06-13).

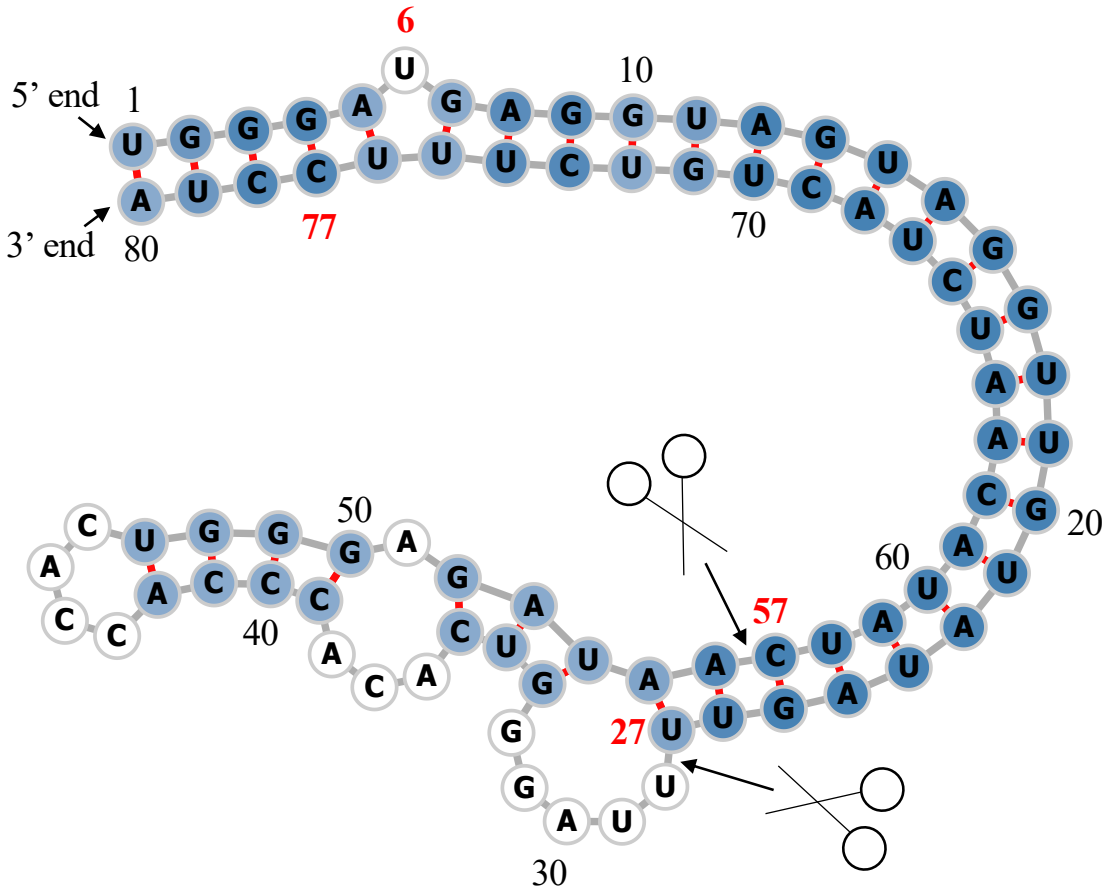


Figure 5.1: Predicted secondary structure of the sequence S of pri-miRNA "hsa-let-7a-1"[2]. Experimental evidence suggests that the two deviated mature miRNAs are $UGA\cdots GUU$ and $CUA\cdots UUC$. They are $S(6:27)$ and $S(57:77)$ (Both ends are inclusive.). The ends are highlighted in **bold**. Since $S(6:27)$ ($S(57:77)$) is near the 5' (3') end, we call it "5p (3p) mature miRNA". The two scissors indicate the two cleavage sites. The color intensity of the nodes reflects their base-pair probability in this predicted secondary structure. The deeper the color, the higher the probability. The unpaired nodes are uncolored. The raw figure is generated by RNAfold web server[3].

The sequence and its predicted secondary structure of a pri-miRNA "hsa-let-7a-1" is shown in Figure 5.1.

Recall that a pri-miRNA contains a hairpin loop, also called a stem loop. A microprocessor complex comprising Drosa and DCGR8 cleaves the pri-miRNA to form a precursor miRNA (pre-miRNA) inside the nucleus. The stem-loop is still preserved, but the two arms become shorter. After that, the pri-miRNA is

[2]Its miRBase entry: https://mirbase.org/hairpin/MI0000060. (Accessed on: 2025-06-12).

[3]RNAfold web server: http://rna.tbi.univie.ac.at/cgi-bin/RNAWebSuite/RNAfold.cgi. (Accessed on: 2025-06-12). The figure is viewed in "forna". This view option can be chosen on the website.

transported by Exportin 5 from the nucleus to the cytoplasm. It is further cleaved by an enzyme called Dicer [108]. Dicer cleaves the stem-loop from the two arms at the two cleavage sites, shown as the two scissors in Figure 5.1. The stem-loop is removed. It results in a short double-stranded miRNA molecule, known as an miRNA duplex, which consists of the 5p strand and the 3p strand[4]. These molecules may be subjected to additional trimming. The miRNA duplex is loaded into an RNA-induced silencing complex (RISC). RISC unwinds the duplex and tends to retain the strand with the less stable 5' end as the guide strand. The other strand is called the passenger strand. The retained strand guides the RISC to silence the target mRNA. Note that both strands can become the guide strand.

Dicer plays an important role in the biogenesis of miRNAs. It is reasonable to argue that the structure of the pre-miRNAs informs Dicer about the cleavage process. It would be of great benefit to understand how Dicer selects cleavage sites from the neighborhood information near the cleavage sites. Studies [109, 110, 111] revealed that the secondary structures are essential for cleavage site determination. Hence, to predict or classify whether a subsequence, extracted from the sequence of a pri-miRNA, contains a cleavage site, we can make use of both the sequence and secondary structure information. PHDcleav employed support vector machines (SVM), leveraging sequence and structure-based features for the classification [112]. LBSizeCleav improved upon it by considering the loop and bulge lengths [113]. [114] proposed an ensemble learning approach, using a gradient boosting machine for better accuracy. [115] developed a deep learning model, namely DiCleave. This model used an autoencoder to learn the secondary structure embeddings of pre-miRNAs from all the species in the miRBase database and leveraged this information. All these methods begin with curated pre-miRNA sequences from the miRBase database. Their secondary structures are predicted. Patterns are extracted from the sequence and the secondary structure. They create the positive cleavage patterns by setting the cleavage sites at the middle of the patterns. The follow-up work of [115], which created the cleavage pattern by allowing cleavage sites to appear at any position within the pattern, instead of the middle only [116]. It created a much larger dataset. This increased dataset facilitates the learning of the deep learning method at the cost of increased running time. We utilized the original dataset setting [112, 113, 114, 115]. DiCleave is the current state-of-the-art (SOTA) for this problem with the original dataset setting.

[4]The 5p strand comes from the 5' arm while the 3p strand comes from the 3' arm. For the directionality, the 5p (3p) strand retains the original 5' (3') end of the pre-miRNA.

These models suffer several limitations. They rely heavily on complicated feature engineering or opaque deep learning models [114, 115, 116]. It results in a lack of generalizability and a long running time. There is a need to design a simpler model so that it can be easily extended to other prediction tasks on RNA data. One way to analyze sequence data is to transform it into time series data. In response to this, we proposed a multivariate time series classification-based method to provide a simpler, more computationally efficient framework for this task. Our contributions are shown as follows.

1. To the best of our knowledge, we are the first to frame the prediction of the cleavage sites as a multivariate time series classification problem.

2. We introduced nine encoding methods to convert RNA sequences to time series.

3. We proposed utilizing the base-pair probabilities in the predicted secondary structure for the prediction. To our surprise, this information has been ignored in the existing studies.

4. For computational efficiency, our method achieves a 3.7X to 28.8X speedup compared to the state-of-the-art (SOTA).

5. We conducted perturbation-based experiments. It shows that regions close to the cleavage sites are important for this problem. It is consistent with the existing study [114].

5.2 Methods

The overall pipeline of this study is summarized in Figure 5.2.

5.2.1 Data Preparation

We used miRBase database [117][5]. The database comprises miRNA data from various organisms [118]. The database contains 38,589 miRNA records. Each record refers to an miRNA sequence, along with other properties such as name, accession, organism, and information on its derivative miRNA products. We are interested in pri-miRNA in humans. The derivative miRNA products are the mature miRNAs. The database also annotates the location of the mature miRNA

[5]The website is `www.mirbase.org`, and the newest version of the database is Release 22.1 (Accessed on 2025-06-22).

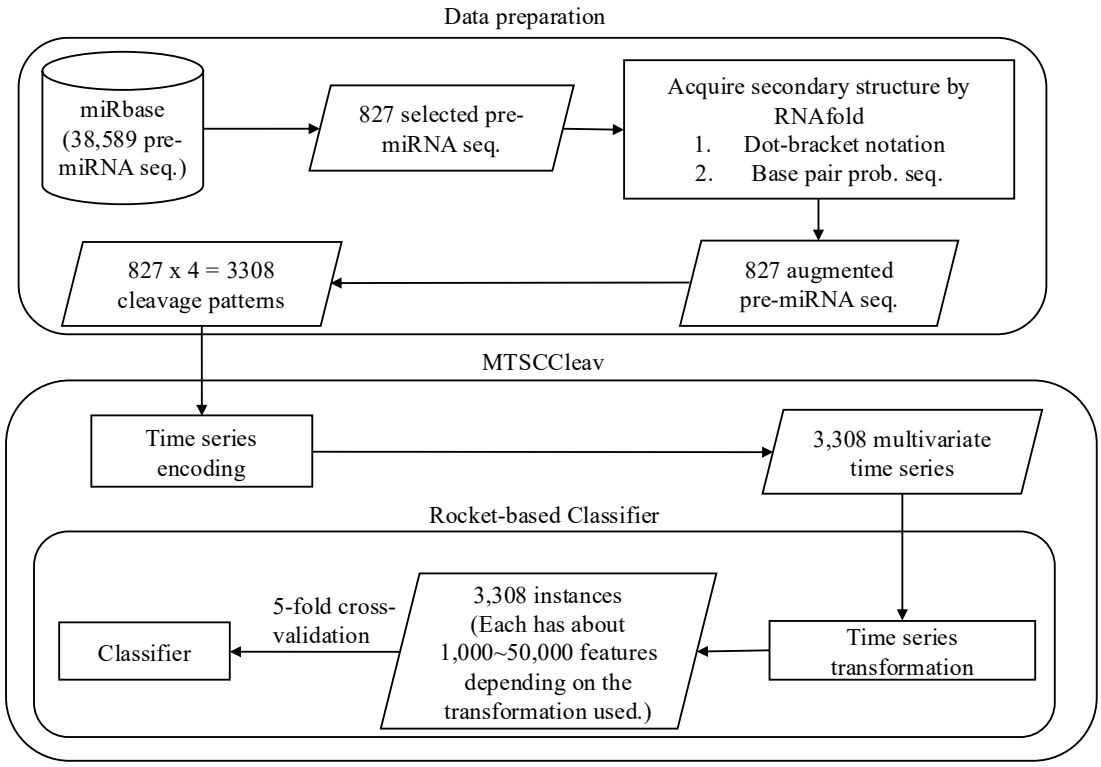


Figure 5.2: The overall pipeline of this study. Symbol notations: Cylinder - Dataset, Rectangle - Process, Parallelogram - Input / Output, Rounded Rectangle - Component.

Accession	Name	Organism	Sequence	Mature miRNA 1	Mature miRNA 2
MI0000001	cel-let-7	Caenorhabditis elegans	$UACAC \cdots UUCGA$	cel-let-7-5p 17:38 experimental	cel-let-7-3p 60:81 experimental
MI0000060	hsa-let-7a-1	Homo sapiens	$UGGGA \cdots UCCUA$	hsa-let-7a-5p 6:27 experimental	hsa-let-7a-3p 57:77 experimental
MI0000114	hsa-mir-107	Homo sapiens	$CUCUC \cdots ACAGA$	hsa-miR-107 50:72 experimental	NA
MI0000238	hsa-mir-196a-1	Homo sapiens	$GUGAA \cdots UUCAC$	hsa-miR-196a-5p 7:28 experimental	hsa-miR-196a-1-3p 45:65 not experimental

Table 5.1: Selected representative records from miRBase. For the last two columns, the first line shows the name, the second line shows its location in the original sequence, and the third line indicates whether its existence has experimental evidence. The selected one is highlighted in **bold**.

Sequence	Secondary Structure (In Dot-bracket notation)
1 UGGGA**UGAGGUAGUAGGUUGUAUAGUU** 27 28 UUAGGGUCACACCCACCACUGGGAGAU 54 55 AA**CUAUACAAUCUACUGUCUUUC**CUA 80	1 (((((.(((((((((((((((((((((27 28 UUAGGGUCACACCCACCACUGGGAGAU 54 55))))))))))))))))))))))))))) 80
Base-pair probabilities sequence (the first 10 bases)	
1 (0.549, 0.946, 0.987, 0.987, 0.904) 5 6 (**0.000**, 0.841, 0.974, 0.981, 0.890) 10	

Table 5.2: The whole sequence of "hsa-let-7a-1" and its predicted secondary structure by RNAfold. The corresponding positions of the two mature miRNAs and the probability of the unpaired "U" are highlighted in **bold**.

within the original sequence and indicates whether its existence has experimental evidence.

Table 5.1 shows its four representative records. We first selected the records from humans (Homo sapiens). It resulted in 1,917 records. To identify the actual locations of the two cleavage sites in the pri-miRNA sequence supported by experimental evidence, we selected records that have two mature miRNAs resulting from cleavage at the 5p arm and the 3p arm, both of which have experimental support. Hence, only "MI0000060" ("hsa-let-7a-1") would be selected in the table. It would serve as our running example. Its whole sequence is listed in Table 5.2. Based on the filtering criterion (i.e., from Homo sapiens and having experimental supports), we selected 827 experimentally validated pre-miRNA sequences, each with its two mature miRNA products. This formed our dataset.

Augment the Dataset with Secondary Structure Information

We leveraged the predicted secondary structure of these sequences to enhance the accuracy of the classification. Recall that a specific three-dimensional (3D) structure is required for DNA, RNA, and protein to perform functions [119]. However, finding these 3D structures using experimental methods such as X-ray crystallography or nuclear magnetic resonance (NMR) is costly and time-consuming. Hence, prediction methods for such 3D structures are necessary and helpful for downstream analysis. However, predicting the 3D structures is challenging. One of the reasons is that there are some "nonconventional" base-pair interactions (e.g., noncanonical and rare A-G) that allow an RNA sequence to fold into a 3D structure, in addition to the (G, U) wobble pair, which is common and functionally important in RNA secondary structures. It makes the search space for prediction much larger than, in the 2D case, the secondary structure. The local structures of the 3D structures, the secondary structures, only focus on the conventional base-pair interactions [106]. Hence, predicting secondary structures is easier and faster.

We employed RNAfold from the ViennaRNA Package[6] to predict the secondary structure for a given pri-miRNA S [120]. RNAfold returns the secondary structure in the dot-bracket notation and a matrix of base-pair probabilities. The matrix is a square pairwise matrix with the side length $|S|$, where each entry m_{ij} is the probability of base s_i paired up with base s_j. Dot-bracket notation is a way of representing the secondary structure of S. Open parentheses "(" (Close parentheses ")") indicates that the base is paired with a complementary base further (earlier) along in S. Dot "." indicates that the base is unpaired. Equipped with the matrix, we can construct the base-pair probability sequence of S. The predicted secondary structure and the base-pair probability sequence of our running example are shown in Table 5.2.

Extract Cleavage Patterns

The locations of the two mature miRNAs on the whole sequence indicate the probable locations of the two cleavage sites. The 5p cleavage site must be beyond and near the ending location of the 5p mature miRNA. We deemed the immediate bond next to the 5p mature miRNA's ending position the 5p cleavage site, with the knowledge that the actual cleavage site may not be this immediate bond but rather the nearby bonds after it. The same applies to the 3p cleavage site. It is located at the immediate bond before the starting position of the 3p mature miRNA.

For each arm of each whole sequence, we extracted a 14-string[7] with the cleavage site located at the center of the string. The first 7 nt (nucleotide) before the center are highlighted in **bold**. In our running example, it would be "**UAUAGUU**UUAGGGU" for the 5p cleavage site and "**GAGAUAA**CUAUACA" for the 3p cleavage site. We refer to these 14-strings as cleavage patterns. We also generate non-cleavage patterns by selecting a 14-string with the center 6 nt away from the corresponding cleavage sites towards the corresponding mature miRNA [113, 114] for each arm of each whole sequence. So, in our running example, the 5p non-cleavage pattern would be "**AGGUUGU**AUAGUUU". The 3p non-cleavage pattern would be "**ACUAUAC**AAUCUAC".

In conclusion, for a given pri-miRNA sequence, we can generate two cleavage patterns and two non-cleavage patterns. We call these four patterns simply the "four strings" of a given pri-miRNA. We also call each string a strand. The "four strings" of our running example are listed in Table 5.3.

[6]The latest stable release is Version 2.7.0 (Accessed on 2025-06-22).
[7]String with length = 14.

	5p cleav	5p non-cleav	3p cleav	3p non-cleav
Input strand	UAUAGUUUUAGGGU	AGGUUGUAUAGUUU	GAGAUAACUAUACA	ACUAUACAAUCUAC
Complementary strand	AUAUCAA_____UA	UCUAACAUAUCAA_	C_CUGUUGAUAUGU	UGAUAUGUUGGAUG

Table 5.3: The first row shows the "four strings" of "hsa-let-7a-1". Their complementary strands are shown in the second row. As a whole, they are referred to as the "eight strings".

We can construct the complementary strand of each of the strands in the "four strings" by finding the corresponding paired base for each of the bases in the input strand by considering the secondary structure information. We use "_" to denote the unpaired base in the complementary strand. For example, in Figure 5.1, "UUAGG" in the 5p cleavage pattern is unpaired, while other bases pair with some bases, the resulting complementary strand is "AUAUCAA_____UA". There is a loop/budge there. We refer to the "four strings" and the four complementary strands together as the "eight strings" of the input pre-miRNA. It is also shown in Table 5.3.

5.2.2 Time Series Encoding

A *time series* $T = t_1, t_2, ..., t_n$ is a sequence of real-valued numbers[8]. A short contiguous region of T is called a subsequence. A *subsequence* $T(i:j) = t_i, t_{i+1}, ..., t_j$ of a time series T is a shorter time series that starts from position i and ends at position j, where $i < j$.

Strings and time series are temporal sequences. The difference between strings and time series lies in their behavioral attributes [121]. For strings, an entry is a letter from a predefined set called the *alphabet*. For example, the alphabet is $\{A, C, G, T\}$ in the DNA string, while $\{A, C, G, U\}$ in the RNA string. For time series, an entry is a real number. Unlike real numbers, there is no ordering in the alphabet unless some external domain knowledge is introduced.

The study of applying signal processing techniques to genomic data is called "Genomic Signal Processing" (GSP) [122, 123]. In the field of GSP, the time series representations of DNA strings are referred to as DNA numeric representations (DNR). Many DNRs have been proposed. We noted that DNA strings and RNA strings are equivalent from a computational standpoint. Many transformation methods designed for DNA can be applied to RNA by simply substituting T with

[8]Unless otherwise specified, we denote entries of a time series (e.g., T) using the corresponding lowercase letter (e.g., t).

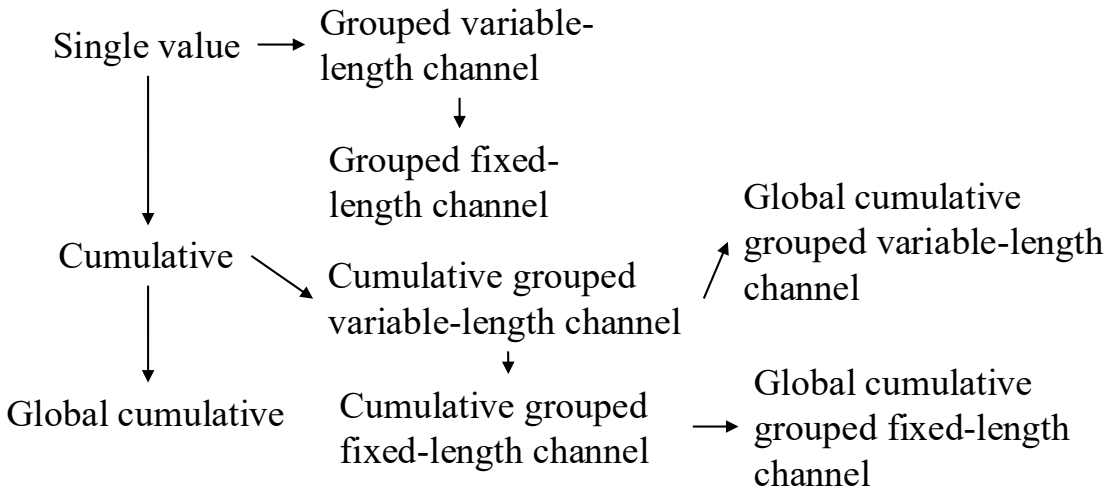


Figure 5.3: Relationship of the proposed encoding methods.

U. We present nine encoding methods. The relationship among them is shown in Figure 5.3.

Single Value versus Cumulative

One of the simple, if not the simplest, encoding is to map the letters into numbers. Domain knowledge can be utilized. This approach is called the "Single value mapping" [124, 125, 126, 127, 122]. One single value is assigned to each of the letters. [128] employed the atomic number of each nucleotide as the transformed values, where $\{G = 78, A = 70, C = 58, T = 66\}$. [129] used electron-ion interaction potential representation (EIIP) as such value, where $\{G = 0.0806, A = 0.1260, C = 0.1340, T = 0.1335\}$. Our goal is to transform the input strand and its complementary strand into time series, aiming to capture the information contained in these sequences and the secondary structure implied by them. We employed the following reasoning to assign the value:

1. We employ the complementary property [130, 126] during encoding. Recall that in the base-pairing rules, G pairs with C to form three hydrogen bonds while A pairs with U[9] to form two hydrogen bonds. G-C pairs are more stable than A-U pairs. G (U) can be regarded as the "inverse" of C (A). We can preserve these base-pairing rules in the encoding by assigning G (A) and C (U) opposite values.

[9]In DNA, A pairs with T.

2. G and A have a two-ring structure. They are purines. C and U have a single-ring structure. They are pyrimidines. Hence, we put G and A (C and U) on the same side of the number line with zero in the middle.

3. The lower stability of A-U pairs promotes strand separation, thereby facilitating the unwinding of the miRNA duplex during RISC loading. Regions rich in A and U are thus more likely to undergo strand selection and cleavage events. We assigned A (U) with a larger absolute value than G (C) to reflect this functional relevance. It aims to highlight sequence regions with higher cleavage potential.

It results in our baseline transformation method, namely "Single value mapping" as shown in row 1 of Table 5.4. S is the input strand. Note that there are advanced encoding methods that consider specific values of the assignment to better encode the nucleotide relationship. Here, we present the general approach of the mapping and focus on preserving the complementary property. When we encode S without incorporating the corresponding base-pair probability sequence P, we set $p_i = 1$ for all the entries of P. We use the first ten nucleotides of the complementary strand of the 3p cleav of "hsa-let-7a-1", as shown in Table 5.3 as S in the examples in Table 5.4.

With the assigned value to each nucleotide defined in single-value mapping, we can compute a cumulative sum of those values over time. It captures the aggregated signal by accumulating past events, allowing us to focus on the trend [131, 132]. We named this method as "Cumulative mapping", shown in row 4 of Table 5.4.

Grouped Variable-Length Channel versus Grouped Local-Length Channel

We can transform the input strand into a multivariate time series with two channels using grouped binary encoding, where nucleotides are grouped into (A, U) and (G, C). It releases our third assumption that A (U) has a larger absolute value than G (C). We proposed two variations. The first one allows the output to be variable-length sequences per channel, depending on group-specific occurrences. The second one always returns two resulting sequences of a fixed length. Two variations extended from single value mapping are shown in rows 2 and 3, while those extended from cumulative mapping are shown in rows 5 and 6 in Table 5.4.

	Encoding	Algorithm	Example $S = C, \_, C, U, G, U, U, G, A, U$ $P = 0.843, 0.000, 0.807, 0.807, 0.793,$ $0.914, 0.982, 1.000, 0.999, 0.999$
1	Single value mapping [124, 125, 126, 127, 122]	for $i = 1$ to $\|S\|$: $t_i = \begin{cases} 2 \cdot p_i & \text{if } s_i = A \\ 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \\ -2 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$ return T	Without base-pair probability sequence: $T = -1, 0, -1, -2, 1, -2, -2, 1, 2, -2$ With base-pair probability sequence: $T = -0.843, 0.000, -0.807, -1.614,$ $0.793, -1.829, -1.963,$ $1.000, 1.999, -1.998$

2	Grouped
	variable-length
	channel mapping
		$j = 1, k = 1$
		for $i = 1$ to $\|S\|$:
		$t^1_j = \begin{cases} 1 \cdot p_i & \text{if } s_i = A \\ -1 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$
		$t^2_k = \begin{cases} 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \end{cases}$
		if $(s_i = G)$ or $(s_i = C)$:
		increment k by 1
		else:
		increment j by 1
		return T^1, T^2
			Without base-pair probability sequence:
			$T^1 = 0, -1, -1, -1, 1, -1$
			$T^2 = -1, -1, 1, 1$
			
			With base-pair probability sequence:
			$T^1 = 0.000, -0.807, -0.914, -0.982, 0.999, -0.999$
			$T^2 = -0.843, -0.807, 0.793, 1.000$

3	Grouped fixed-length channel mapping	for $i = 1$ to $\|S\|$: $t^1_i = \begin{cases} 1 \cdot p_i & \text{if } s_i = A \\ -1 \cdot p_i & \text{if } s_i = U \\ 0 & \text{otherwise} \end{cases}$ $t^2_i = \begin{cases} 1 \cdot p_i & \text{if } s_i = G \\ -1 \cdot p_i & \text{if } s_i = C \\ 0 & \text{otherwise} \end{cases}$ return T^1, T^2	Without base-pair probability sequence: $T^1 = 0, 0, 0, -1, 0, -1, -1, 0, 1, -1$ $T^2 = -1, 0, -1, 0, 1, 0, 0, 1, 0, 0$ With base-pair probability sequence: $T^1 = 0.000, 0.000, 0.000, -0.807,$ $0.000, -0.914, -0.982,$ $0.000, 0.999, -0.9999$ $T^2 = -0.843, 0.000, -0.807, 0.000,$ $0.793, 0.000, 0.000,$ $1.000, 0.000, 0.000$
4	Cumulative mapping [131, 132]	$t_1 = 0$ for $i = 1$ to $\|S\|$: $t_{i+1} = \begin{cases} t_i + 2 \cdot p_i & \text{if } s_i = A \\ t_i + 1 \cdot p_i & \text{if } s_i = G \\ t_i - 1 \cdot p_i & \text{if } s_i = C \\ t_i - 2 \cdot p_i & \text{if } s_i = U \\ t_i & \text{otherwise} \end{cases}$ return T // $\|T\| = \|S\| + 1$	Without base-pair probability sequence: $T = 0, -1, -1, -2, -4, -3, -5, -7, -6, -4, -6$ With base-pair probability sequence: $T = 0.000, -0.843, -0.843, -1.650,$ $-3.265, -2.471, -4.300, -6.263,$ $-5.264, -3.265, -5.263$

5	Cumulative grouped
	variable-length
	channel mapping
		$t^1_1 = 0, t^2_1 = 0$
		$j = 1, k = 1$
		for $i = 1$ to $\|S\|$:
		$t^1_{j+1} = \begin{cases} t^1_j + 1 \cdot p_i & \text{if } s_i = A \\ t^1_j - 1 \cdot p_i & \text{if } s_i = U \\ t^1_j & \text{if } s_i = \_ \end{cases}$
		$t^2_{k+1} = \begin{cases} t^2_k + 1 \cdot p_i & \text{if } s_i = G \\ t^2_k - 1 \cdot p_i & \text{if } s_i = C \end{cases}$
		if $(s_i = G)$ or $(s_i = C)$:
		increment k by 1
		else:
		increment j by 1
		return T^1, T^2
			Without base-pair probability sequence:
			$T^1 = 0, -1, -2, -3, -2, -3$
			$T^2 = 0, -1, -2, -1, 0$
			
			With base-pair probability sequence:
			$T^1 = 0.000, -0.807, -1.722,$
			$-2.703, -1.704, -2.703$
			$T^2 = 0.000, -0.843, -1.650,$
			$-0.857, 0.143$

6	Cumulative grouped fixed-length channel mapping	$t^1_1 = 0, t^2_1 = 0$ for $i = 1$ to $\|S\|$: $t^1_{i+1} = \begin{cases} t^1_i + 1 \cdot p_i & \text{if } s_i = A \\ t^1_i - 1 \cdot p_i & \text{if } s_i = U \\ t^1_i & \text{otherwise} \end{cases}$ $t^2_{i+1} = \begin{cases} t^2_i + 1 \cdot p_i & \text{if } s_i = G \\ t^2_i - 1 \cdot p_i & \text{if } s_i = C \\ t^2_i & \text{otherwise} \end{cases}$ return T^1, T^2 // $\|T^1\| = \|T^2\| = \|S\| + 1$	Without base-pair probability sequence: $T^1 = 0, 0, 0, 0, -1, -1, -2, -3, -3, -2, -3$ $T^2 = 0, -1, -1, -2, -2, -1, -1, -1, 0, 0, 0$ With base-pair probability sequence: $T^1 = 0.000, 0.000, 0.000, 0.000,$ $-0.807, -0.807, -1.722, -2.703,$ $-2.703, -1.704, -2.703$ $T^2 = 0.000, -0.843, -0.843, -1.650,$ $-1.650, -0.857, -0.857, -0.857,$ $0.143, 0.143, 0.143$

Table 5.4: Time series encoding. P is the corresponding base-pair probability sequence of S. $p_i = 1$ if we encode S without incorporating base-pair probability sequence.

Global Cumulative versus Local Cumulative

In cumulative mapping and its variations, we can choose where to start the accumulation. For a given subsequence S' of the whole sequence S, accumulation can start from the beginning of S even if only S' is used downstream. It can also begin just at the start of the S'. The first one preserves the global context. It can be useful when previous nucleotides (those before S') influence later interpretation. The second one focuses solely on local history in S', ignoring global history. It is helpful if the previous nucleotides do not affect the chemical property of S'.

Consider $T = 0, -1, ..., -6$ of the input string S in "Cumulative mapping" in Table 5.4, which accumulates from 0. S is the suffix with length = 10 of the constructed complementary strand of $S(1 : 63)$ in Figure 5.1. If we start the accumulation from the first entry of the constructed complementary strand instead, it will yield a different result. Suppose that the last entry of the time series encoded in the cumulative mapping of the constructed complementary strand is -8, the time series encoded in the "Global cumulative mapping" for S would accumulate from -8 instead of 0. The result is $T = -8, -9, ..., -14$. Note that it has the same trend as the original T. This "Global cumulative" concept can be applied to every cumulative-based method, as shown in Figure 5.3.

Incorporating Base-Pair Probabilities

We can incorporate the base-pair probabilities P in the encoding by thinking of it as the weight or confidence p_i in the value assignment of each nucleotide s_i. It is implemented by multiplying the base-pair probability p_i of the nucleotide s_i with the assigned value of the kind of nucleotide of s_i during encoding, as shown in Table 5.4.

Transforming the Secondary Structure into a Time Series

We can transform the secondary structure in the dot-bracket notation into a time series by "Single value mapping", where "(" maps to 1, "." maps to 0, and ")" maps to -1.

5.2.3 Time Series Classification

In univariate time series classification, an instance in the dataset consists of a time series $x = x_1, x_2, ..., x_m$ with m observations and a discrete class label y, which takes c possible values [37, 133]. If $c = 2$, we refer to binary classification. If

$c > 2$, we refer to multi-class classification. In multivariate time series classification, the time series is not a single sequence but a list of sequences. Each sequence is called a channel. There are many classifiers defined for time series data, including distance-based, feature-based, interval-based, shapelet-based, dictionary-based, convolution-based, and deep learning-based classifiers. Additionally, two or more of the above approaches can be combined, resulting in hybrid approaches [1, 133, 37]. We employed convolution-based classifiers due to their simplicity and accuracy.

Convolution-Based Classifiers

Convolution-based classifiers first use randomly parameterized kernels to perform convolutions on the original time series T. A kernel is referred to as parameterized because its behavior is governed by a set of parameters, which will be discussed in detail later. Convolution is an operation to transform T to another time series M, where M is called the activation map. Its entry M_i is calculated by applying a kernel ω with length l to T at position i, defined as follows:

$$M_i = T(i : i + l - 1) * \omega = \sum_{j=0}^{l-1} t_{i+j} \cdot \omega_{1+j}$$

To note, $|T(i : i + l - 1)| = |\omega| = l$. Entries M_i's are calculated by sliding ω across T and computing a dot product. Additionally, although the original paper [25] used the term "convolution" to refer to the above operation, "cross-correlation" may be a more suitable term for this operation. Recall T with length m has $(m - l + 1)$ sliding windows of length l, given that the increment is 1[10], which defines the length of M.

Figure 5.4 shows two kernels ω^1 and ω^2 with lengths 3 and 5, respectively. Each of which performs a convolution with T and returns two activation maps, M^1 and M^2, respectively. For example, $M_1^1 = T(1 : 3) * \omega^1 = 3$. By sliding ω^1 one time stamp at a time, an activation map M^1 with length $= (m-l+1) = 11-3+1 = 9$ is obtained. Then, pooling operations, such as the maximum (MAX) and proportion of positive values (PPV), are applied on M^1 to derive the summary features. In Figure 5.4, MAX and PPV are applied on M^1 and M^2. The summary features of M^1 are 4 and 5/9, which correspond to MAX and PPV, respectively. Dilation refers to a method that enables a kernel to cover a larger portion by creating empty spaces between entries in the kernel. The dilation d of ω^2 is 2. It introduces a gap of 1 in every two values of ω^2.

[10]One step to the right per time.

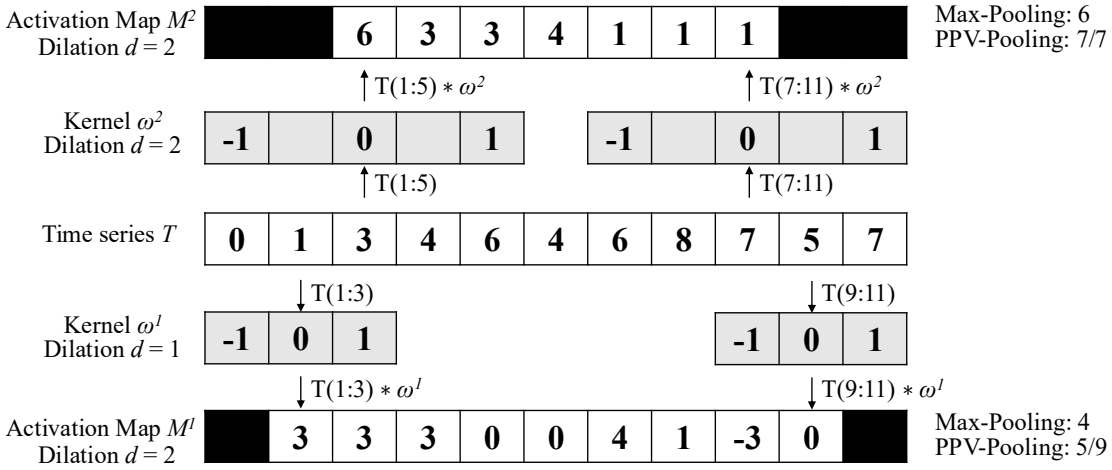


Figure 5.4: Features generation in the transformation

The most popular convolution-based approach is the Random Convolutional Kernel Transform (ROCKET) [25]. It generates a large number of randomly parameterized kernels, ranging from thousands to tens of thousands. The kernel's parameters include length, weights (the entries inside the kernel), bias (the value added to the result of the convolution operation), and dilation. Additionally, padding can be applied to T at the start and end, ensuring M has the same length as the input. To note, T, M_1, and M_2 in Figure 5.4 have different lengths. The summary statistics of the activation map are obtained through two pooling operations: MAX and PPV. Hence, for k kernels, the transformed data has $2k$ features. The default value of k is 10,000.

There are two extensions of ROCKET. They are MiniROCKET [26] and MultiROCKET [27]. MiniROCKET removes unnecessary operations and many of the random components in the definition of kernels used by ROCKET. It speeds up Rocket by over an order of magnitude with no significant difference in accuracy, making the classifier almost deterministic. For example, the kernel length is fixed, and only two weight values are used. Only PPV is used for the summary statistics. MultiROCKET is extended from MiniROCKET. The main improvement of it is to extract features from first-order differences as defined in Table 5.5 and add three new pooling operations [27]. The three added operations are mean of positive values (MPV), mean of indices of positive values (MIPV) and longest stretch of positive values (LSPV).

The HYbrid Dictionary-ROCKET Architecture (Hydra) combines dictionary-based and convolution-based models [28]. Similar to ROCKET-based classifiers, it uses random kernels to extract features from the input time series. But it groups

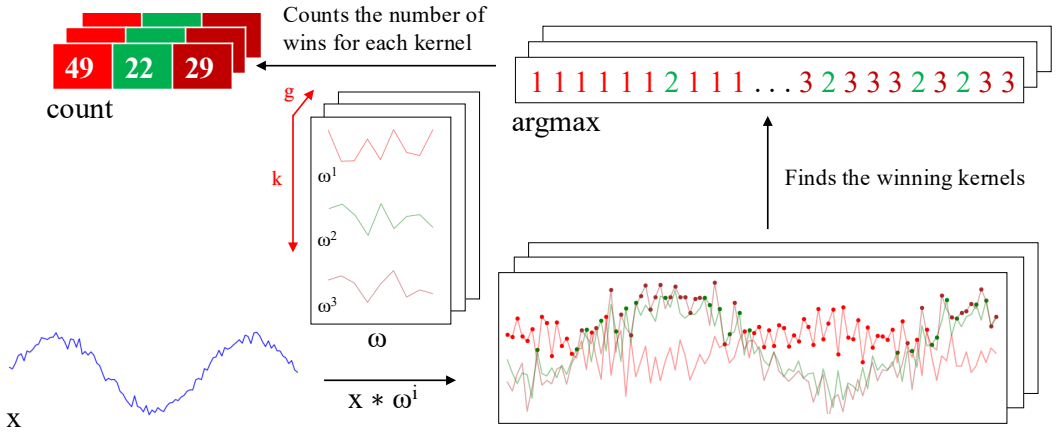


Figure 5.5: Convolutions of HYDRA for each input time series with a set of random kernels w, organized into g groups with k kernels each.

the kernels into g groups of k kernels each, as shown in Figure 5.5. Each time series is passed through all the groups. For each group of kernels, we slide them across T and compute the dot product at each timestamp. Recall that the dot product of two input vectors (x and w_i) has the maximum value when the two vectors align in the same direction and the minimum value when they are oriented in opposite directions. We record the kernel that best matches the subsequence of T at each timestamp in each group (i.e., argmax). We refer to these kernels as the winning kernels. This results in a k-dimensional count vector for each of the g groups, where $k = 3$ in Figure 5.5. This results in a total of $g \times k$ features, with default values of $g = 64$ and $k = 8$. It uses a total of $k \times g = 512$ kernels per dilation. In addition to recording the kernel with the maximum response, we can also record the kernel with the minimum response, knowing that this kernel will be the best match with the "inverted" subsequence of T. Hydra is applied to both the original time series and its first-order differences. Hydra generated approximately 1000 features for each instance in our dataset. [28] found that it can improve the accuracy by concatenating features generated from Hydra with those from MultiRocket. This classifier is called MultiROCKET-Hydra.

These five classifiers share the same simple design pattern. It involves the overproduction of features followed by a selection strategy. A large number of features (1,000 $\sim$ 50,000) are generated for each instance. The features are then fed into a simple linear classifier. It determines which features are most useful and returns the final classification result. A ridge classifier is used in this study. It is a linear classifier that extends ridge regression to classification tasks by applying a threshold to the predicted values. It uses L2 regularization to prevent

	ROCKET	MiniROCKET	MultiROCKET	Hydra
kernel length	{7, 9, 11}	9	9	9
kernel weights	$\mathcal{N}(0,1)$	{-1, 2}	{-1, 2}	$\mathcal{N}(0,1)$
bias	$\mathcal{U}(0,1)$	from output	from output	none
dilation	random	fixed (input-relative)	fixed (input-relative)	random
padding	random	fixed	fixed	always
pooling operations	MAX, PPV	PPV	PPV, MPV, MIPV, LSPV	Response per Kernel/Group
1^{st} order difference	no	no	yes	yes
feature vector size	20k	10k	50k	relative to input

Table 5.5: Comparison of rocket-based classifiers [1]. $\mathcal{N}(0,1)$: a standard normal distribution, $U(0,1)$: a uniform distribution between 0 and 1, 1^{st} order difference: $\Delta T = t_2 - t_1, t_3 - t_2, ..., t_n - t_{n-1}$.

overfitting. The regularization strength is selected by internal cross-validation. A Ridge classifier is suggested for small datasets, as in our case, while a logistic regression classifier is suggested for large datasets [1].

While these five classifiers are often referred to as classifiers [1], they are technically time series transformation methods for generating features that are then fed to a downstream classifier. The comparison of them is shown in Table 5.5. For MiniROCKET and MultiROCKET, the bias is determined from the convolution output, and the dilation depends on the length of the input time series [26, 27]. The main differences among ROCKET-based classifiers lie in how the summary features are generated. The generation of the summary features depends on:

1. Kernels, which are defined based on the parameters, which consist of kernel length, kernel weights, bias, and dilation.

2. The way that padding applies to T, which leads to activation maps with different lengths.

3. The pooling operations, which are used in extracting features on the activation map.

5.2.4 Evaluation Metrics

To evaluate the performance of our time series-based classification (MTSC) model, we adopted five standard classification metrics. They are Accuracy (Acc), Specificity (Sp), Sensitivity (Sn), F1 score (F1), and Matthews Correlation Coefficient

(MCC) [134].

$$
\begin{aligned}
Acc &= \frac{TP + TN}{TP + TN + FP + FN} \\
Sp &= \frac{TN}{TN + FP} \\
Sn &= \frac{TP}{TP + FN} \\
F1 &= \frac{2 \times TP}{2 \times TP + FP + FN} \\
MCC &= \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}
\end{aligned}
$$

where TP, TN, FP, and FN are the number of true positives, true negatives, false positives, and false negatives, respectively.

To extend a binary metric to multi-class problems, we can treat the data as a collection of binary problems, one for each class. One class is treated as positive while the other classes are treated as negative. Then, the multi-class metrics can be obtained by averaging binary metric calculations across the set of classes. There are different ways of doing the averaging. Here, we adopted a macro-averaging approach. It treats each class equally and calculates the mean of the binary metrics. To use MCC in the multiclass case, it can be defined in terms of a confusion matrix C for K classes, where $C_{i,j}$ is the number of observations that are actually in class i and predicted to be in class j [135].

$$
MCC_{multi} = \frac{c \times s - \sum_k^K p_k \times t_k}{\sqrt{(s^2 - \sum_k^K p_k^2) \times (s^2 - \sum_k^K t_k^2)}}
$$

where $t_k = \sum_i^K C_{i,k}$ (denoting the number of times class k actually occurred), $p_k = \sum_i^K C_{k,i}$ (denoting the number of times class k was predicted), $c = \sum_k^K C_{k,k}$ (denoting the total number of samples correctly predicted) and $s = \sum_i^K \sum_j^K C_{i,j}$ (denoting the total number of samples).

5.3 Results

The code implementing our method is available at `https://github.com/colemanyu/time-series-classification-cleavage`. The dataset of this study is available at `https://www.mirbase.org`.

In all experiments, the models were trained and tested using 5-fold cross-validation. We retrieved 827 empirically validated sequences of pre-miRNAs. There are 5p arm and 3p arm in each sequence. For each arm, we defined a cleavage pattern and a non-cleavage pattern. Three datasets, namely "5p arm", "3p arm", and "multi-class" were constructed by these patterns. We refer to the cleavage patterns as positive instances and the non-cleavage patterns as negative instances. The 5p arm dataset comprises 827 positive instances and an equal number of negative instances. The 5p arm and 3p arm datasets are binary-class datasets. The multi-class dataset comprises all patterns from both the 5p arm and the 3p arm. There are 827 "5p" instances[11], 827 "3p" instances, and 1,654 negative instances.

For every fold in 5-fold cross-validation, the dataset was divided into a training set and a test set with sizes of 80% and 20% of the whole dataset, respectively. We kept the class distribution approximately the same in each fold, since it is in the original dataset. In each fold derived from the 5p arm and 3p arm datasets, the training set has a size of 1,323, and the test set has a size of 331. In each fold derived from the multi-class dataset, the training set has a size of 2,262, and the test set has a size of 662. We reported the average of the five classification metrics.

The ROCKET-based classifiers require all channels in the multivariate time series to have equal length. We applied padding to the shorter channels with the constant value 100, which does not appear in the original time series. It ensures the padding does not introduce ambiguity or interfere with the semantic meaning of the encoded nucleotide signals.

5.3.1 Channel Importance Study

We utilized three types of data as the input features for each instance. They are (1) the RNA sequence, which consists of the primary strand and its complementary strand, (2) the secondary structure information, and (3) the base-pair probability sequence. To input the data into our time series-based classifiers, we converted them into multivariate time series. The primary strand and its complementary strand are each encoded into one or two channels, using the encoding methods in Table 5.4. For example, single value mapping encodes a strand in one channel, while grouped variable-length channel mapping encodes in two channels. The secondary structure information is converted into a univariate time series. The base-pair probability sequence is already in numerical form and does not require further

[11] Cleavage patterns from the 5p arm.

	Classifier	5p arm					3p arm					multi-class				
		Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC
Baseline (cfg 1)	ROCKET	0.781	0.743	0.819	0.789	0.563	0.790	0.773	0.807	0.793	0.580	0.717	0.838	0.685	0.700	0.538
	MiniROCKET	0.755	0.728	0.782	0.762	0.512	0.788	0.781	0.794	0.789	0.576	0.685	0.823	0.653	0.662	0.486
	MultiROCKET	0.784	0.767	0.801	0.787	0.569	0.803	0.792	0.814	0.805	0.606	0.691	0.830	0.667	0.672	0.501
	Hydra	0.830	0.800	0.860	0.835	0.663	0.808	0.797	0.820	0.810	0.617	0.731	0.844	0.696	0.712	0.560
	MultiROCKET-Hydra	0.796	0.778	0.815	0.800	0.594	0.807	0.767	0.816	0.808	0.614	0.701	0.836	0.681	0.686	0.520
Baseline + Secondary Structre (cfg 2)	ROCKET	**0.847**	**0.832**	0.862	**0.849**	**0.695**	**0.855**	0.842	0.868	**0.857**	**0.711**	**0.836**	**0.907**	**0.828**	**0.833**	0.736
	MiniROCKET	0.825	0.807	0.843	0.827	0.652	0.822	0.802	0.843	0.826	0.646	0.823	0.900	0.812	0.818	0.715
	MultiROCKET	0.812	0.803	0.822	0.814	0.626	0.824	0.809	0.839	0.826	0.649	0.796	0.888	0.791	0.792	0.673
	Hydra	0.845	0.816	**0.873**	**0.849**	0.691	0.846	0.817	**0.874**	0.850	0.693	0.830	0.901	0.814	0.826	0.724
	MultiROCKET-Hydra	0.817	0.809	0.826	0.819	0.635	0.825	0.816	0.834	0.826	0.652	0.803	0.891	0.798	0.800	0.684
Baseline + Base-pair probability (Standalone) (cfg 3)	ROCKET	0.842	0.828	0.855	0.844	0.684	**0.855**	**0.856**	0.854	0.855	0.710	0.795	0.885	0.783	0.789	0.670
	MiniROCKET	0.817	0.820	0.814	0.816	0.634	0.836	0.834	0.838	0.836	0.673	0.772	0.872	0.757	0.764	0.632
	MultiROCKET	0.822	0.813	0.832	0.824	0.645	0.825	0.831	0.820	0.824	0.651	0.758	0.866	0.747	0.750	0.612
	Hydra	0.846	0.827	0.865	**0.849**	0.693	0.851	0.840	0.861	0.852	0.702	0.789	0.879	0.769	0.780	0.658
	MultiROCKET-Hydra	0.822	0.809	0.834	0.824	0.644	0.835	0.840	0.830	0.834	0.670	0.759	0.866	0.746	0.750	0.611
Baseline + Base-pair probability (Incorporated) (cfg 4)	ROCKET	0.799	0.771	0.827	0.805	0.600	0.809	0.786	0.832	0.813	0.619	0.737	0.850	0.712	0.724	0.573
	MiniROCKET	0.776	0.756	0.797	0.781	0.554	0.801	0.808	0.794	0.799	0.603	0.705	0.835	0.675	0.684	0.521
	MultiROCKET	0.814	0.801	0.828	0.817	0.630	0.816	0.812	0.820	0.816	0.634	0.726	0.848	0.706	0.712	0.556
	Hydra	0.822	0.787	0.857	0.828	0.647	0.834	0.828	0.840	0.835	0.669	0.759	0.862	0.734	0.746	0.608
	MultiROCKET-Hydra	0.814	0.802	0.820	0.817	0.629	0.820	0.825	0.816	0.819	0.642	0.736	0.853	0.717	0.723	**0.874**

Table 5.6: Channel importance study. The best results are highlighted in **bold**.

transformation. It can be used either as a standalone channel or incorporated into the encoding of the complementary strand. We performed a channel importance study to determine the most informative combination of the above channels.

We referred to the multivariate time series that consists of the channels from the RNA sequence only as the baseline setting. We added the other channels to this baseline. It leads to the following configurations (cfgs):

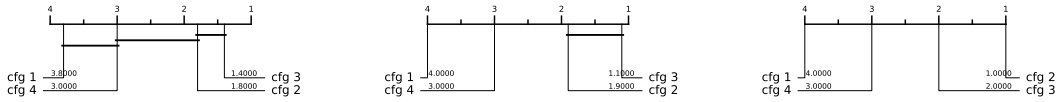
1. (cfg 1) Baseline: Time series derived only from the RNA sequence.

2. (cfg 2) Baseline + Secondary structure: Baseline + time series representation of the secondary structure.

3. (cfg 3) Baseline + Base-pair probability (Standalone): Baseline + the base-pair probability sequence as a standalone channel.

4. (cfg 4) Baseline + Base-Pair probability (Incorporated): Baseline with the base-pair probability sequence incorporated into the encoding of the complementary strand.

We used single value mapping as the encoding method. Table 5.6 shows the result. From the table, we can see that the addition of secondary structure, base-pair probability as a standalone channel, and base-pair probability incorporated in the encoding of the complementary strand can improve the performance. We plotted the critical difference (CD) diagram as shown in Figure 5.6 to visualize Table 5.6 to make the performances of different combinations more obvious. In CD diagrams, lower-ranked methods (toward the right) are better. A horizontal bar connecting combinations indicates no statistically significant difference.



(a) 5p arm (b) 3p arm (c) multi-class

Figure 5.6: CD diagrams of channel importance study.

From Figure 5.6, we can see that including time series derived from secondary structure information and base-pair probability as a separate channel can significantly improve the performance of the classifiers. Incorporating the base-pair probability sequence in the time series encoding of the complementary strand can also improve the classifier, but to a minor degree compared to serving as a standalone channel. In our downstream analysis, we adopted the combination of RNA sequence time series (2 to 4 channels), secondary structure time series (1 channel), and base-pair probability time series (1 channel) as our multivariate time series input, with 4 to 6 channels, depending on the encoding used.

5.3.2 Predictive Performance

The experiment was conducted on three datasets: the 5p arm, the 3p arm, and the multi-class datasets. Recall that we have nine encoding methods and five ROCKET-based classifiers. It results in 45 combinations of encoding methods and classifiers.

The result is shown in Table 5.7. The best combination of encoding method and classifier is shown in Table 5.8. For the 5p arm dataset, the best combination is "Global Cumulative grouped fixed-length channel mapping + ROCKET". For all five classification metrics, it outperforms the state-of-the-art (SOTA) method, DiCleave. For the 3p arm dataset, the best combination is "Global Cumulative grouped fixed-length channel mapping + ROCKET". Out of the five classification metrics, it outperforms DiCleave, except in specificity. For the multi-class dataset, the best combination is "Global Cumulative grouped fixed-length channel mapping + ROCKET". For all five classification metrics, it outperforms DiCleave. Note that for the 3p arm and the multi-class datasets, the combination of "Cumulative grouped fixed-length channel mapping + ROKCET" also attains the best result.

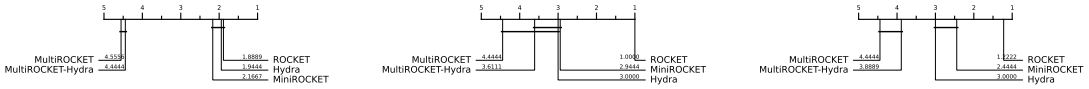
To summarize Table 5.7, we plot the CD diagrams for finding the best classifier, which is ROCKET in all cases, as shown in Figure 5.7, and the best encoding method, which is enc 9 in the 5p arm dataset and enc 7 for the others, as shown

	Classifier	5p arm					3p arm					multi-class				
		Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC	Acc	Sp	Sn	F1	MCC
Single value mapping (enc 1)	ROCKET	0.849	0.842	0.857	0.851	0.699	0.863	0.854	0.873	0.865	0.727	0.853	0.917	0.847	0.851	0.764
	MiniROCKET	0.823	0.809	0.837	0.825	0.647	0.823	0.828	0.817	0.822	0.647	0.835	0.906	0.828	0.833	0.735
	MultiROCKET	0.821	0.802	0.840	0.824	0.643	0.839	0.826	0.852	0.841	0.679	0.811	0.894	0.806	0.809	0.697
	Hydra	0.843	0.820	0.867	0.847	0.688	0.838	0.819	0.857	0.841	0.677	0.831	0.901	0.815	0.827	0.727
	MultiROCKET-Hydra	0.820	0.803	0.837	0.823	0.640	0.840	0.830	0.850	0.841	0.680	0.816	0.896	0.810	0.814	0.704
Grouped variable-length channel mapping (enc 2)	ROCKET	0.835	0.826	0.844	0.836	0.670	0.855	0.849	0.861	0.856	0.710	0.846	0.913	0.839	0.844	0.752
	MiniROCKET	0.843	0.833	0.853	0.844	0.686	0.831	0.821	0.842	0.833	0.663	0.837	0.907	0.828	0.834	0.737
	MultiROCKET	0.819	0.809	0.828	0.820	0.638	0.817	0.814	0.820	0.818	0.634	**0.890**	0.894	0.806	0.808	0.695
	Hydra	0.825	0.780	0.869	0.832	0.653	0.811	0.769	0.854	0.819	0.626	0.818	0.892	0.765	0.812	0.705
	MultiROCKET-Hydra	0.818	0.814	0.822	0.819	0.636	0.831	0.825	0.837	0.832	0.662	0.820	0.900	0.815	0.818	0.710
Grouped fixed-length channel mapping (enc 3)	ROCKET	0.851	0.843	0.859	0.852	0.702	0.863	0.850	0.875	0.864	0.726	0.849	0.915	0.843	0.847	0.757
	MiniROCKET	0.844	0.836	0.853	0.845	0.689	0.840	0.826	0.855	0.843	0.682	0.851	0.915	0.844	0.849	0.760
	MultiROCKET	0.831	0.815	0.848	0.834	0.663	0.824	0.813	0.836	0.826	0.649	0.811	0.896	0.808	0.808	0.698
	Hydra	0.848	0.816	0.880	0.853	0.699	0.862	0.839	**0.884**	0.864	0.724	0.843	0.908	0.837	0.839	0.746
	MultiROCKET-Hydra	0.836	0.813	0.859	0.839	0.672	0.833	0.820	0.845	0.835	0.665	0.828	0.905	0.824	0.826	0.725
Cumulative mapping (enc 4)	ROCKET	0.850	0.834	0.866	0.852	0.701	0.863	0.855	0.871	0.864	0.726	0.852	0.915	0.842	0.850	0.762
	MiniROCKET	0.840	0.821	0.860	0.843	0.682	0.840	0.837	0.844	0.841	0.682	0.843	0.911	0.835	0.840	0.747
	MultiROCKET	0.822	0.809	0.834	0.824	0.644	0.832	0.830	0.834	0.832	0.665	0.820	0.898	0.810	0.816	0.709
	Hydra	0.848	0.819	0.878	0.853	0.698	0.853	0.856	0.869	0.855	0.705	0.845	0.910	0.830	0.841	0.749
	MultiROCKET-Hydra	0.824	0.811	0.856	0.825	0.647	0.838	0.833	0.843	0.839	0.677	0.821	0.898	0.810	0.817	0.711
Cumulative grouped variable-length channel mapping (enc 5)	ROCKET	0.843	0.821	0.866	0.847	0.688	0.856	0.840	0.871	0.857	0.712	0.855	0.916	0.843	0.851	0.766
	MiniROCKET	0.845	0.826	0.865	0.848	0.691	0.836	0.833	0.838	0.836	0.672	0.840	0.909	0.833	0.838	0.742
	MultiROCKET	0.826	0.814	0.838	0.828	0.653	0.815	0.820	0.810	0.814	0.631	0.826	0.902	0.820	0.824	0.721
	Hydra	0.850	0.819	0.880	0.854	0.701	0.834	0.807	0.861	0.838	0.669	0.833	0.903	0.818	0.829	0.731
	MultiROCKET-Hydra	0.824	0.810	0.838	0.826	0.649	0.833	0.833	0.833	0.833	0.666	0.830	0.903	0.821	0.827	0.726
Cumulative grouped fixed-length channel mapping (enc 6)	ROCKET	0.856	0.836	0.876	0.858	0.712	**0.870**	**0.861**	0.879	**0.871**	**0.741**	0.863	**0.921**	0.852	**0.860**	**0.780**
	MiniROCKET	0.856	0.837	0.874	0.858	0.712	0.842	0.839	0.845	0.843	0.685	0.845	0.912	0.837	0.843	0.751
	MultiROCKET	0.820	0.802	0.839	0.824	0.642	0.798	0.798	0.798	0.798	0.597	0.809	0.894	0.806	0.807	0.694
	Hydra	0.850	0.814	0.885	0.855	0.701	0.855	0.840	0.869	0.857	0.711	0.847	0.910	0.831	0.843	0.752
	MultiROCKET-Hydra	0.820	0.801	0.839	0.823	0.641	0.807	0.813	0.802	0.806	0.615	0.821	0.900	0.817	0.819	0.713
Global Cumulative mapping (enc 7)	ROCKET	0.850	0.834	0.866	0.852	0.701	0.863	0.855	0.871	0.864	0.726	0.852	0.915	0.842	0.850	0.762
	MiniROCKET	0.847	0.832	0.862	0.849	0.695	0.848	0.839	0.857	0.850	0.697	0.845	0.911	0.836	0.843	0.750
	MultiROCKET	0.827	0.819	0.834	0.828	0.653	0.847	0.842	0.853	0.848	0.695	0.825	0.901	0.817	0.822	0.718
	Hydra	0.851	0.821	0.880	0.855	0.703	0.861	0.848	0.874	0.863	0.722	0.847	0.911	0.834	0.844	0.753
	MultiROCKET-Hydra	0.829	0.823	0.834	0.830	0.658	0.843	0.838	0.849	0.844	0.688	0.832	0.905	0.823	0.829	0.730
Global Cumulative grouped variable-length channel mapping (enc 8)	ROCKET	0.840	0.814	0.867	0.844	0.682	0.853	0.838	0.867	0.854	0.706	0.856	0.917	0.845	0.853	0.768
	MiniROCKET	0.848	0.834	0.862	0.850	0.697	0.841	0.824	0.859	0.844	0.683	0.844	0.911	**0.856**	0.842	0.748
	MultiROCKET	0.834	0.828	0.839	0.834	0.668	0.831	0.821	0.842	0.833	0.663	0.828	0.904	0.823	0.826	0.724
	Hydra	**0.857**	0.821	**0.894**	**0.862**	**0.717**	0.822	0.786	0.857	0.828	0.645	0.826	0.898	0.806	0.820	0.717
	MultiROCKET-Hydra	0.837	0.834	0.839	0.837	0.674	0.834	0.827	0.840	0.835	0.668	0.835	0.907	0.828	0.832	0.734
Global Cumulative grouped fixed-length channel mapping (enc 9)	ROCKET	0.856	0.836	0.876	0.858	0.712	**0.870**	**0.861**	0.879	**0.871**	**0.741**	0.863	**0.921**	0.852	**0.860**	**0.780**
	MiniROCKET	**0.857**	**0.845**	0.870	0.859	0.715	0.840	0.821	0.859	0.843	0.681	0.844	0.911	0.837	0.842	0.749
	MultiROCKET	0.829	0.825	0.833	0.830	0.658	0.820	0.816	0.823	0.820	0.640	0.819	0.900	0.816	0.817	0.710
	Hydra	0.856	0.817	**0.894**	0.861	0.713	0.859	0.838	0.880	0.862	0.719	0.846	0.911	0.832	0.843	0.752
	MultiROCKET-Hydra	0.829	0.824	0.834	0.830	0.658	0.822	0.825	0.819	0.821	0.644	0.827	0.904	0.823	0.824	0.722

Table 5.7: Performance on the 45 combinations between encoding methods and the ROCKET-based classifiers. The best results are highlighted in **bold**.

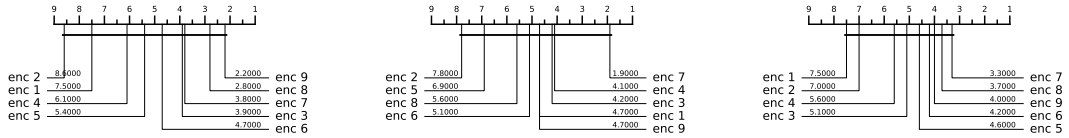
Dataset	Methods	Acc	Sp	Sn	F1	MCC	Time (s)
5p arm	enc 9 + MiniROCKET	**0.857**	**0.845**	**0.870**	**0.859**	**0.715**	**0.787**
	DiCleave	0.818	0.790	0.846	0.822	0.653	21.249
3p arm	enc 9 + ROCKET	**0.870**	0.861	**0.879**	**0.871**	**0.741**	4.311
	enc 7 + MiniROCKET	0.848	0.839	0.857	0.850	0.697	**0.989**
	DiCleave	0.854	**0.891**	0.817	0.847	0.715	15.919
multi-class	enc 9 + ROCKET	**0.863**	**0.921**	**0.852**	**0.860**	**0.780**	12.208
	enc 3 + MiniROCKET	0.851	0.915	0.844	0.849	0.760	**4.550**
	DiCleave	0.820	0.895	0.804	0.815	0.710	131.151

Table 5.8: Comparative analysis between MTSCCleav with the best combination of the encoding method and classifier, with the SOTA, DiCleave, on the three datasets. The best results of using MiniROCKET have also been shown to compare the computational efficiency. The best results are highlighted in **bold**.



(a) 5p arm (b) 3p arm (c) multi-class

Figure 5.7: CD diagrams to compare different classifiers.



(a) 5p arm (b) 3p arm (c) multi-class

Figure 5.8: CD diagrams to compare different encoding methods.

in Figure 5.8. It is suggested to use ROCKET as the baseline classifier. For the encoding, it is suggested to use enc 7 or enc 9.

5.3.3 Running Time Analysis

To compare the computational efficiency of MTSCCleav and DiCleave, we conducted a comparative analysis of their running times. For DiCleave, we employed the code from its supporting website[12], without any modifications. All experiments were conducted on the same machine (a personal laptop equipped with an Apple M1 Pro chip and 16 GB of memory) and using the same splits of the training and test datasets under 5-fold cross-validation to ensure fairness. The reported running times are the averages of the five runs. The timing results were measured from the training phase to the return of the five classification metrics. The result is shown in Table 5.8. MiniROCKET is the most computationally efficient of the five rocket-based classifiers. We also included its best result, along with the corresponding encoding method, even though this combination may not be the best overall.

MTSCCleav demonstrated a significant advantage in computational efficiency, achieving an average 27.0X, 3.7X, and 10.7X speedup over DiCleave, for the 5p arm, 3p arm, and multi-class datasets, respectively. If we consider using the MiniROCKET in the case of 3p arm and multi-class datasets, it achieves 16.1X and 28.8X speedup. To note, in the case of the 3p arm dataset, the performance of MiniROCKET is only slightly worse than DiCleave. In the case of the multi-class

12 https://github.com/MGuard0303/DiCleave (Accessed on: 2025-07-13).

dataset, even the performance of MiniROCKET is better than DiCleave. DiCleave is a deep learning-based method that requires substantial time for model inference, while MTSCleav leverages efficient ROCKET-based classifiers. This significant reduction in runtime makes MTSCCleav more suitable for large-scale data and real-time applications.

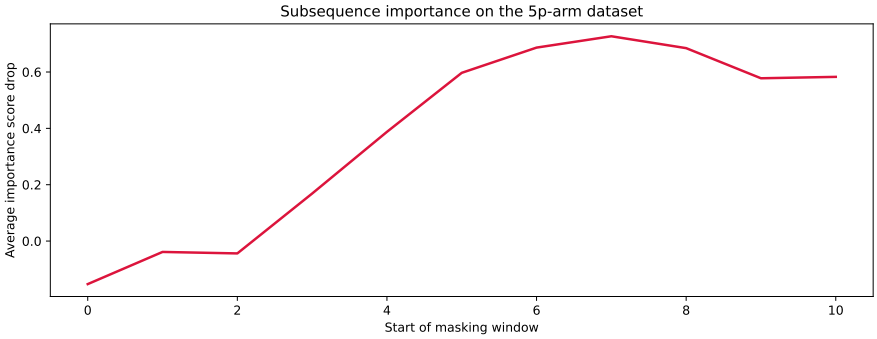
5.3.4 Subsequence Importance

To evaluate the sensitivity of MTSCCleav to subsequences of the input, we conducted a perturbation experiment to evaluate the importance of subsequences based on masking windows. The goal of this experiment is to identify which subsequences of the entire time series are critical for classification. We examine how various modifications to the original input impact model performance. It suggests which features are essential for classification.

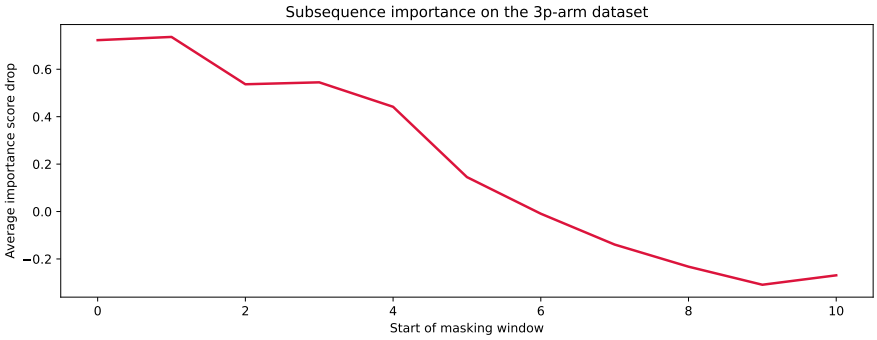
The model was trained on the original training dataset. For each instance in the test dataset, we measure its original score and the masked score. We slid a masking window w with a fixed length over the input time series T. $|w|$ was set to 4, which is about one third of the whole length. For each window position $i \in \{1, 2, ..., |T| - |w| + 1\}$, we masked all entries across all the channels of T within the window. Hence, we removed or hid that portion of information from the model during inference. The changes in classification performance in terms of accuracy relative to the unmasked original score of each i are recorded. Intuitively, if the information of a subsequence is critical for the classification, the masking of this subsequence would lead to a great drop in classification performance. We aggregated the importance score across the test dataset.

The result is shown in Figure 5.9. For the encoding methods, we cannot use the methods derived from the cumulative mapping because the accumulation would leak information from the masked region. We adopted "Grouped fixed-length channel mapping" as the encoding method and ROCKET as the classifier. "Grouped fixed-length channel mapping" is the best encoding, other than the methods derived from the cumulative mapping, in all datasets, as shown in Figure 5.8. ROCKET is the best classifier, as shown in Figure 5.7.

In the 5p arm dataset, we found that masking subsequences at the tailing part caused a significant drop in the importance score, as shown in Figure 5.9 (a). In the 3p arm dataset, we found that masking subsequences at the leading part caused a significant drop in the importance score, as shown in Figure 5.9 (b).



(a) 5p arm



(b) 3p arm

Figure 5.9: Results of the perturbation experiment.

5.3.5 Summary

Our method achieves better or comparable predictive results and a 3.7X to 28.8X speedup compared to the state-of-the-art (SOTA).

5.4 Discussion

The channel importance study reveals that the involvement of the time series derived from the secondary structure can improve accuracy. It suggests the importance of RNA folding in Dicer processing. Futhermore, we found that the base-pair probability sequence of the secondary structure can also enhance accuracy. To the best of our knowledge, it is a novel application of the base-pair probability sequence. Experiments show that using the probability sequence as an additional channel can enhance accuracy more than incorporating it in the encoding. It is likely because keeping it as an additional channel can preserve more information, of both the probability sequence itself and the complementary strand.

Out of the three datasets, the best classifier is ROCKET. The ranking of the five classifiers by performance, starting from the best, is as follows: ROCKET, MiniROCKET, Hydra, MultiROCKET-Hydra, and MultiROCKET, where the ranking of Hydra and MiniROCKET is interchanged in different cases. It indicates that the features created from the pooling operations that are only in MultiROCKET but not in MiniROCKET, confuse the final classifier. They are mean of positive values (MPV), mean of indices of positive values (MIPV) and longest stretch of positive values (LSPV) [27]. In contrast, the pooling operator that is only present in ROCKET but not in MiniROCKET, enhances the classification performance. It is maximum (MAX).

For the encoding methods, we have the following observations. Fixed-length grouped channel mappings outperform variable-length counterparts with one exception in the multi-class dataset, likely because fixed-length schemes better preserve the original positional information of nucleotides within the sequence. Global cumulative methods consistently yield better performance than local cumulative methods. It suggests that the upstream information of the cleavage pattern plays a critical role in identifying cleavage sites. Cumulative-based encodings perform better than single-value mappings, with one exception in the 3p dataset, suggesting that the accumulated nucleotide signal is more informative for cleavage site prediction than the local or isolated presence of nucleotides. According to Figure 5.8 (b), for the 3p arm dataset, encoding RNA sequence in two channels in a global cumulative manner (i.e., enc 9) appears to worsen the result. This suggests that the 5p arm and 3p arm datasets may require different nucleotide grouping methods for encoding.

One limitation of DiCleave is overfitting during training because of the relatively small size of the dataset [115]. DiCleave is a deep learning-based method. Deep learning models typically require a large amount of training data to generalize effectively. They are data-hungry. In contrast, MTSCCleav leverages ROCKET-based methods for the classification. They rely on random convolutional feature extraction followed by a simple linear classifier. The Ridge classifier used in this study is less data-hungry compared to deep learning methods due to its use of L2 regularization and the simplicity of its linear model nature. It allows ROCKET-based classifiers, and hence MTSCCleav, to maintain strong predictive performance even in settings with a relatively small dataset size.

The subsequence importance reveals some connections between RNA secondary structure and human Dicer cleavage site prediction. The perturbation experiment shows that the leading part of 5p arm and the tailing part of 3p arm are important

for the classification. These parts are close to the center of the RNA secondary structure of pre-miRNA. It indicates that the center region is more crucial for human Dicer cleavage site prediction. It is consistent with the previous study [114].

5.5 Concluding Remarks

We proposed an accurate, fast, and simple multivariate time series classification (MTSC)-based method, termed MTSCCleav, for predicting human Dicer cleavage sites. Base-pair probability sequences of the secondary structures have also been leveraged in the classification. MTSCCleav consists of three parts: time series encoding, time series transformation, and classification. ROCKET-based methods were used for time series transformation. Ridge Classifier was used for classification. For the computational experiments, we evaluated nine time series encoding methods in conjunction with five time series transformation methods. MTSCCleav outperformed the SOTA method in all five evaluation metrics for the 5p-arm and multi-class datasets, and four of the metrics for the 3p-arm dataset. In terms of computational efficiency, MTSCCleav with the optimal setting achieved an average 3.7X to 27.0X speedup over the SOTA method on the three datasets. With the use of a less accurate but faster time series classification method, MTSCCleav achieved an average speedup of 16.1X to 28.8X, respectively. We analyzed the subsequence importance of the input multivariate time series. The results show that subsequences near the center of the pre-miRNA sequences are more important. This aligns with the findings from previous work. This study demonstrates that time series analysis provides a powerful alternative to conventional modeling in the context of RNA processing. This framework may be extended to other RNA-processing tasks. Notably, the encoding of RNA sequence into time series enables us to utilize any well-established tools from the time series community.

Chapter 6

Conclusion and Future Directions

This thesis contributes to the field of time series analysis by addressing three challenges via subsequence analysis. Specifically, we explored (1) the development of a new distance measure framework that is invariant to multiple local scaling factors (i.e., piecewise distortions), (2) the augmentation of forecasting models using subsequence-derived covariates, and (3) the transformation of biological sequence prediction into multivariate time series classification, where the sequence is a subsequence of the source time series.

6.1 Piecewise Similarity

The first study addressed a fundamental limitation of existing time series distance measures, which is the assumption of a single, global scaling rate. In reality, data such as human motion or music often exhibit multiple expression rates across different phases, each with its own rate. We introduced the Piecewise Scaling Distance (PSD) framework, the first to account for multiple local scaling factors. PSD is agnostic to the underlying base distance measure, and we evaluated two primary instantiations: PSED (Euclidean-based) and PSDTW (DTW-based). We provided a dynamic programming solution and introduced several acceleration techniques, including a constrained version that limits the search space based on allowable segment lengths, parallel computing, and early abandoning. Experiments demonstrated that PSD, particularly PSED, outperforms traditional measures like ED, DTW, and DTW variants when processing data with multi-rate distortions.

Future research will focus on developing a heuristic or algorithmic approach to automatically determine the optimal number of segments. We also aim to design a specialized lower bound for PSED to further improve its runtime and investigate

methods to adapt "incremental DTW" for use with interpolated time series, as in our case.

6.2 Covariate Augmentation

The second study developed a novel method for time series forecasting by generating covariates from the historical nearest neighbors of each forecasting window. By utilizing the Matrix Profile to identify left nearest neighbors, we extracted information, such as immediate subsequences and similarity scores, to augment the input for Gradient Boosting Regression Tree (GBRT) models, which serve as the underlying forecasters. We have also extended the ideas for the use of k-nearest-neighbor information. Our results showed that models trained on the augmented data significantly outperform those trained on the original data.

Future directions for this research include refining nearest-neighbor searches to specific temporal ranges to capture distinct short-term and long-term patterns by leveraging the location information of the nearest neighbors provided by the matrix profile. We also plan to extend the framework to multivariate forecasting tasks, utilize motif families rather than individual neighbors, and develop robust methods for handling outliers within the retrieved immediate subsequences.

6.3 Encoding

In the third study, we investigated the prediction of human Dicer cleavage sites. It is essential for understanding microRNA (miRNA) biogenesis and post-transcriptional gene regulation. We framed this as a multivariate time series classification (MTSC) problem and curated a dataset of 14-string patterns from a longer string, and transformed them into time series using nine novel encoding methods for RNA sequences. Notably, we were the first to incorporate base-pair probabilities from predicted secondary structures into the encoding process. By employing state-of-the-art ROCKET-based classifiers, our proposed framework, MTSCCleav, achieved predictive performance comparable to or exceeding deep learning-based methods while delivering substantial computational speedups of 3.7X to 28.8X. Furthermore, perturbation-based experiments identified that regions near the center of the pre-miRNA secondary structure are most critical for determining cleavage sites, aligning with existing biological literature.

Future work includes leveraging the full ensemble of potential secondary structures, rather than a single secondary structure prediction result, for a given RNA

sequence by encoding them as multivariate time series with more channels. Additionally, we propose the use of interpretable classifiers, such as those based on time series shapelets, to gain deeper insights into the specific subsequences that discriminate between cleavage and non-cleavage sites.

6.4 Final Remarks

This thesis has explored the potential of subsequence-centric methods to enhance various aspects of time series analysis. By focusing on the local information embedded within subsequences, we have demonstrated that targeted exploitation of subsequences can yield significant improvements across diverse tasks.

List of Publications

This thesis is based on the following papers.

- (Chapter 3) **Coleman Yu**, Tatsuya Akutsu, and Raymond Chi-Wing Wong, "Scaling with Multiple Scaling Factors in Time Series Searching", IEEE Access, pp. 53406 - 53424, Vol.14, 2026, 10.1109/ACCESS.2026.3680034
- (Chapter 4) **Coleman Yu**, Tatsuya Akutsu, and Raymond Chi-Wing Wong, "Leveraging Nearest Neighbors for Time Series Forecasting with Matrix Profile", in revision
- (Chapter 5) **Coleman Yu**, Raymond Chi-Wing Wong, and Tatsuya Akutsu, "MTSCCleav: a Multivariate Time Series Classification (MTSC)-based Method for Predicting Human Dicer Cleavage Sites", IEEE Access, pp. 11048 - 11063, Vol.14, 2026, 10.1109/ACCESS.2026.3656449

Other publications

- **Coleman Yu** and Raymond Chi-Wing Wong, "A Melody Composer for both Tonal and Non-Tonal Languages", the 43rd International Computer Music Conference 2017, Shanghai, China on 16-20 Oct, 2017
  - We applied a data mining method called frequent pattern mining to capture the relationships between the pitch trend in the melody and the tone trend in lyrics, and used these relationships to create a new melody for the user-given lyrics. The pitch and melody trends are both time-series data. It served as the initial motivation to pursue time series analysis from a data mining perspective.
- Yi Zheng, Bogdan Enescu, Jiancang Zhuang, and **Coleman Yu**, "Data replenishment of five moderate earthquake sequences in Japan, with semi-automatic cluster selection", Earthquake Science, 34:310-322, 2021

- We applied the DBSCAN clustering method to seismicity data to automatically identify the nearest significant earthquake cluster to a given mainshock. The clustering results were then fed into a downstream replenishment method to discover missing early aftershocks, which follow relatively large or moderate earthquakes.

Poster presentations

- **Coleman Yu** and Tatsuya Akutsu, "Aligning gene expression time series with invariance to uniform scaling with multiple scaling factors", International Workshop on Bioinformatics and Systems Biology, Boston, USA (IBSB 2018) on 16-18 July, 2018

  - We presented an idea that, for time series analysis, rather than focusing on the whole sequence analysis, it is more important to focus on the consistent subsequences of a time series. In this poster, we use gene expression data as an example to explain. Genes are expressed over time. But the expression rate is not a constant. Hence, we need to find the corresponding phases between the two aligned gene expression time series before applying the distance measure. The varying rate would be discussed in Chapter 3.

Bibliography

[1] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, Jul. 2024.

[2] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2194, p. 20200209, Feb. 2021.

[3] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, Feb. 1978.

[4] F. Itakura, "Minimum prediction residual principle applied to speech recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 23, no. 1, pp. 67–72, Feb. 1975.

[5] E. J. Keogh and M. J. Pazzani, "Derivative dynamic time warping," in *Proceedings of the 2001 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, Apr. 2001, pp. 1–11.

[6] Y.-S. Jeong, M. K. Jeong, and O. A. Omitaomu, "Weighted dynamic time warping for time series classification," *Pattern Recognition*, vol. 44, no. 9, pp. 2231–2240, Sep. 2011.

[7] J. Zhao and L. Itti, "**shapeDTW**: Shape dynamic time warping," *Pattern Recognition*, vol. 74, pp. 171–184, Feb. 2018.

[8] E. Keogh, K. Chakrabarti, M. Pazzani, and S. Mehrotra, "Dimensionality reduction for fast similarity search in large time series databases," *Knowledge and Information Systems*, vol. 3, no. 3, pp. 263–286, Aug. 2001.

[9] B.-K. Yi and C. Faloutsos, "Fast time sequence indexing for arbitrary **Lp** norms," in *Proceedings of the 26th International Conference on Very Large Data Bases*, ser. VLDB '00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Sep. 2000, pp. 385–394.

[10] J. Zhao and L. Itti, "Decomposing time series with application to temporal segmentation," in *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*, Mar. 2016, pp. 1–9.

[11] M. Herrmann and G. I. Webb, "**Amercing**: An intuitive and effective constraint for dynamic time warping," *Pattern Recognition*, vol. 137, p. 109333, May 2023.

[12] G. E. A. P. A. Batista, E. J. Keogh, O. M. Tataw, and V. M. A. de Souza, "**CID**: An efficient complexity-invariant distance for time series," *Data Mining and Knowledge Discovery*, vol. 28, no. 3, pp. 634–669, May 2014.

[13] D. F. Silva, G. E. A. P. A. Batista, and E. Keogh, "Prefix and suffix invariant dynamic time warping," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*, Dec. 2016, pp. 1209–1214.

[14] Y. Zhu, S. Gharghabi, D. F. Silva, H. A. Dau, C.-C. M. Yeh, N. Shakibay Senobari, A. Almaslukh, K. Kamgar, Z. Zimmerman, G. Funning, A. Mueen, and E. Keogh, "The swiss army knife of time series data mining: Ten useful things you can do with the matrix profile and ten lines of code," *Data Mining and Knowledge Discovery*, vol. 34, no. 4, pp. 949–979, Jul. 2020.

[15] Y. Zhu, Z. Zimmerman, N. S. Senobari, C.-C. M. Yeh, G. Funning, A. Mueen, P. Brisk, and E. Keogh, "**Matrix Profile II**: Exploiting a novel algorithm and gpus to break the one hundred million barrier for time series motifs and joins," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. Barcelona, Spain: IEEE, Dec. 2016, pp. 739–748.

[16] S. Zhong and A. Mueen, "**MASS**: Distance profile of a query over a time series," *Data Mining and Knowledge Discovery*, vol. 38, no. 3, pp. 1466–1492, May 2024.

[17] C.-C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, D. F. Silva, A. Mueen, and E. Keogh, "**Matrix Profile I**: All pairs similarity joins for time series: A unifying view that includes motifs, discords and

shapelets," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. Barcelona, Spain: IEEE, Dec. 2016, pp. 1317–1322.

[18] Y. Zhu, M. Imamura, D. Nikovski, and E. Keogh, "**Matrix Profile VII**: Time series chains: A new primitive for time series data mining," in *2017 IEEE International Conference on Data Mining (ICDM)*, Nov. 2017, pp. 695–704.

[19] C.-C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, Z. Zimmerman, D. F. Silva, A. Mueen, and E. Keogh, "Time series joins, motifs, discords and shapelets: A unifying view that exploits the matrix profile," *Data Mining and Knowledge Discovery*, vol. 32, no. 1, pp. 83–123, Jan. 2018.

[20] S. Gharghabi, S. Imani, A. Bagnall, A. Darvishzadeh, and E. Keogh, "**Matrix Profile XII**: **MPdist**: A novel time series distance measure to allow data mining in more challenging scenarios," in *2018 IEEE International Conference on Data Mining (ICDM)*, Nov. 2018, pp. 965–970.

[21] L. Ye and E. Keogh, "Time series shapelets: A new primitive for data mining," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Paris France: ACM, Jun. 2009, pp. 947–956.

[22] J. Grabocka, N. Schilling, M. Wistuba, and L. Schmidt-Thieme, "Learning time-series shapelets," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '14. New York, NY, USA: Association for Computing Machinery, Aug. 2014, pp. 392–401.

[23] B. D. Fulcher and N. S. Jones, "**Hctsa**: A computational framework for automated time-series phenotyping using massive feature extraction," *Cell Systems*, vol. 5, no. 5, pp. 527–531.e3, Nov. 2017.

[24] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "**Catch22**: Canonical time-series characteristics," *Data Mining and Knowledge Discovery*, vol. 33, no. 6, pp. 1821–1852, Nov. 2019.

[25] A. Dempster, F. Petitjean, and G. I. Webb, "**ROCKET**: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, Sep. 2020.

[26] A. Dempster, D. F. Schmidt, and G. I. Webb, "**MiniRocket**: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, ser. KDD '21. New York, NY, USA: Association for Computing Machinery, Aug. 2021, pp. 248–257.

[27] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, "**MultiRocket**: Multiple pooling operators and transformations for fast and effective time series classification," *Data Mining and Knowledge Discovery*, vol. 36, no. 5, pp. 1623–1646, Sep. 2022.

[28] A. Dempster, D. F. Schmidt, and G. I. Webb, "**Hydra**: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, no. 5, pp. 1779–1805, Sep. 2023.

[29] H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: A review," *Data Mining and Knowledge Discovery*, vol. 33, no. 4, pp. 917–963, Jul. 2019.

[30] Z. Cui, W. Chen, and Y. Chen, "Multi-scale convolutional neural networks for time series classification," May 2016.

[31] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *2017 International Joint Conference on Neural Networks (IJCNN)*, May 2017, pp. 1578–1585.

[32] C. Pelletier, G. I. Webb, and F. Petitjean, "Temporal convolutional neural network for the classification of satellite image time series," *Remote Sensing*, vol. 11, no. 5, p. 523, Jan. 2019.

[33] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems*, vol. 27. Curran Associates, Inc., 2014.

[34] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "**InceptionTime**: Finding alexnet for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1936–1962, Nov. 2020.

[35] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "**HIVE-COTE 2.0**: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, no. 11, pp. 3211–3243, Dec. 2021.

[36] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "**TimesNet**: Temporal 2d-variation modeling for general time series analysis," in *The Eleventh International Conference on Learning Representations*, Sep. 2022.

[37] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The great time series classification bake off: A review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, vol. 31, no. 3, pp. 606–660, May 2017.

[38] T.-c. Fu, "A review on time series data mining," *Engineering Applications of Artificial Intelligence*, vol. 24, no. 1, pp. 164–181, Feb. 2011.

[39] C. Rose, B. Guenter, B. Bodenheimer, and M. F. Cohen, "Efficient generation of motion transitions using spacetime constraints," in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '96. New York, NY, USA: Association for Computing Machinery, Aug. 1996, pp. 147–154.

[40] Y. Li, T. Wang, and H.-Y. Shum, "Motion texture: A two-level statistical model for character motion synthesis," *ACM Trans. Graph.*, vol. 21, no. 3, pp. 465–472, Jul. 2002.

[41] R. B. Dannenberg, W. P. Birmingham, G. P. Tzanetakis, C. P. Meek, N. P. Hu, and B. P. Pardo, "The **MUSART** testbed for query-by-humming evaluation," *Comput. Music J.*, vol. 28, no. 2, pp. 34–48, Jun. 2004.

[42] K.-C. Li, M. Yan, and S. Yuan, "A simple statistical model for depicting the **Cdc15**-synchronized yeast cell-cycle regulated gene expression data," *Statistica Sinica*, vol. 12, no. 1, pp. 141–158, 2002.

[43] A. W.-C. Fu, E. Keogh, L. Y. H. Lau, C. A. Ratanamahatana, and R. C.-W. Wong, "Scaling and time warping in time series querying," *The VLDB Journal*, vol. 17, no. 4, pp. 899–921, Jul. 2008.

[44] Y. Shen, Y. Chen, E. Keogh, and H. Jin, "Searching time series with invariance to large amounts of uniform scaling," in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, Apr. 2017, pp. 111–114.

[45] ——, "Accelerating time series searching with large uniform scaling," in *Proceedings of the 2018 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2018, pp. 234–242.

[46] C. C. Aggarwal and C. K. Reddy, Eds., *Data Clustering: Algorithms and Applications*, 1st ed. Boca Raton: Chapman and Hall/CRC, Aug. 2013.

[47] E. Keogh, T. Palpanas, V. B. Zordan, D. Gunopulos, and M. Cardle, "Indexing large human-motion databases," in *Proceedings of the Thirtieth International Conference on Very Large Data Bases - Volume 30*, ser. VLDB '04. Toronto, Canada: VLDB Endowment, Aug. 2004, pp. 780–791.

[48] P. Esling and C. Agon, "Time-series data mining," *ACM Computing Surveys*, vol. 45, no. 1, pp. 1–34, Nov. 2012.

[49] H.-L. Li, S.-W. Liu, and S.-W. Chen, "**PolSAR** ship characterization and robust detection at different grazing angles with polarimetric roll-invariant features," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–18, 2024.

[50] H.-L. Li and S.-W. Chen, "Polyhedral corner reflectors multidomain joint characterization with fully polarimetric radar," *IEEE Transactions on Antennas and Propagation*, vol. 73, no. 12, pp. 10 679–10 693, Dec. 2025.

[51] ——, "General polarimetric correlation pattern: A visualization and characterization tool for target joint-domain scattering mechanisms investigation," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 64, pp. 1–17, 2026.

[52] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, "Addressing big data time series: Mining trillions of time series subsequences under dynamic time warping," *ACM Trans. Knowl. Discov. Data*, vol. 7, no. 3, pp. 10:1–10:31, Sep. 2013.

[53] C. W. Tan, F. Petitjean, and G. I. Webb, "Elastic bands across the path: A new framework and method to lower bound **DTW**," in *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2019, pp. 522–530.

[54] A. Shifaz, C. Pelletier, F. Petitjean, and G. I. Webb, "Elastic similarity and distance measures for multivariate time series," *Knowledge and Information Systems*, vol. 65, no. 6, pp. 2665–2698, Jun. 2023.

[55] S. Salvador and P. Chan, "Toward accurate dynamic time warping in linear time and space," *Intelligent Data Analysis*, vol. 11, no. 5, pp. 561–580, Oct. 2007.

[56] D. Yankov, E. Keogh, J. Medina, B. Chiu, and V. Zordan, "Detecting time series motifs under uniform scaling," in *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '07. New York, NY, USA: Association for Computing Machinery, Aug. 2007, pp. 844–853.

[57] X. Zeng, B. Liang, W. Li, S. Han, and D. Fu, "Similarity measure of time series based on angle-distance penalized metric dynamic time warping," *Scientific Reports*, vol. 15, no. 1, p. 45437, Nov. 2025.

[58] Q. Cai, L. Chen, J. Shao, and L. Chen, "Piecewise dynamic time warping-based subsequence matching in data stream," *IEEE Access*, vol. 13, pp. 119 512–119 522, 2025.

[59] ——, "**AdaPDTW**: An efficient abstract-adaptive piecewise dynamic time warping for time series classification," *IEEE Access*, vol. 13, pp. 84 188–84 201, 2025.

[60] Y. Zhu and D. Shasha, "Warping indexes with envelope transforms for query by humming," in *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '03. New York, NY, USA: Association for Computing Machinery, Jun. 2003, pp. 181–192.

[61] E. Keogh and C. A. Ratanamahatana, "Exact indexing of dynamic time warping," *Knowledge and Information Systems*, vol. 7, no. 3, pp. 358–386, Mar. 2005.

[62] S.-W. Kim, S. Park, and W. Chu, "An index-based approach for similarity search supporting time warping in large sequence databases," in *Proceedings 17th International Conference on Data Engineering*, Apr. 2001, pp. 607–614.

[63] C. W. Tan, M. Herrmann, G. Forestier, G. I. Webb, and F. Petitjean, "Efficient search of the best warping window for dynamic time warping," in *Proceedings of the 2018 SIAM International Conference on Data Mining (SDM)*, ser. Proceedings. Society for Industrial and Applied Mathematics, May 2018, pp. 225–233.

[64] M. Middlehurst, A. Ismail-Fawaz, A. Guillaume, C. Holder, D. Guijo-Rubio, G. Bulatova, L. Tsaprounis, L. Mentel, M. Walter, P. Schäfer, and A. Bagnall, "**Aeon**: A **Python** toolkit for learning from time series," *Journal of Machine Learning Research*, vol. 25, no. 289, pp. 1–10, 2024.

[65] C. A. Ratanamahatana and E. Keogh, "Everything you know about dynamic time warping is wrong," in *3 Rd International Workshop on Mining Temporal and Sequential Data (TDM-04)*. Citeseer, 2004.

[66] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The **UCR** time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, Nov. 2019.

[67] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, "Precision and recall for time series," in *Advances in Neural Information Processing Systems*, vol. 31. Curran Associates, Inc., 2018.

[68] M. Leodolter, C. Plant, and N. Brändle, "**IncDTW**: An **R** package for incremental calculation of dynamic time warping," *Journal of Statistical Software*, vol. 99, pp. 1–23, Sep. 2021.

[69] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, ser. SIGIR '18. New York, NY, USA: Association for Computing Machinery, Jun. 2018, pp. 95–104.

[70] C. Chatfield and H. Xing, *The Analysis of Time Series: An Introduction with **R***. Boca Raton: Chapman and Hall/CRC, 2019.

[71] O. Y. Al-Jarrah, P. D. Yoo, S. Muhaidat, G. K. Karagiannidis, and K. Taha, "Efficient machine learning for big data: A review," *Big Data Research*, vol. 2, no. 3, pp. 87–93, Sep. 2015.

[72] W. Li and K. L. E. Law, "Deep learning models for time series forecasting: A review," *IEEE Access*, vol. 12, pp. 92 306–92 327, 2024.

[73] N. I. Sapankevych and R. Sankar, "Time series prediction using support vector machines: A survey," *IEEE Computational Intelligence Magazine*, vol. 4, no. 2, pp. 24–38, May 2009.

[74] Z. Chen, M. Ma, T. Li, H. Wang, and C. Li, "Long sequence time-series forecasting with deep learning: A survey," *Information Fusion*, vol. 97, p. 101819, Sep. 2023.

[75] J. F. Torres, D. Hadjout, A. Sebaa, F. Martínez-Álvarez, and A. Troncoso, "Deep learning for time series forecasting: A survey," *Big Data*, vol. 9, no. 1, pp. 3–21, Feb. 2021.

[76] D. B. da Silva, D. Schmidt, C. A. da Costa, R. da Rosa Righi, and B. Eskofier, "**DeepSigns**: A predictive model based on deep learning for the early detection of patient health deterioration," *Expert Systems with Applications*, vol. 165, p. 113905, Mar. 2021.

[77] N. Wu, B. Green, X. Ben, and S. O'Banion, "Deep transformer models for time series forecasting: The influenza prevalence case," Jan. 2020.

[78] F. Martínez-Álvarez, G. Asencio-Cortés, J. F. Torres, D. Gutiérrez-Avilés, L. Melgar-García, R. Pérez-Chacón, C. Rubio-Escudero, J. C. Riquelme, and A. Troncoso, "Coronavirus optimization algorithm: A bioinspired metaheuristic based on the **COVID-19** propagation model," *Big Data*, vol. 8, no. 4, pp. 308–322, Aug. 2020.

[79] S. Elsayed, D. Thyssens, A. Rashed, H. S. Jomaa, and L. Schmidt-Thieme, "Do we really need deep learning models for time series forecasting?" Oct. 2021.

[80] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, New Jersey: Wiley, 2015.

[81] G. E. P. Box and D. A. Pierce, "Distribution of residual autocorrelations in autoregressive-integrated moving average time series models," *Journal of the American Statistical Association*, vol. 65, no. 332, pp. 1509–1526, 1970.

[82] G. U. Yule, "On a method of investigating periodicities in disturbed series, with special reference to **Wolfer**'s sunspot numbers," *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 226, pp. 267–298, 1927.

[83] L. Su, D. Li, and D. Qiu, "**BLS-QLSTM**: A novel hybrid quantum neural network for stock index forecasting," *Humanities and Social Sciences Communications*, vol. 12, no. 1, p. 1011, Jul. 2025.

[84] L. Su, L. Gan, C. Pan, and F. Li, "**BLS-AttnTCN**: Attention temporal convolutional fusion networks with broad learning system for multidimensional chaotic time series prediction," *Expert Systems with Applications*, vol. 314, p. 131615, Jun. 2026.

[85] S. Tajmouati, B. E. L. Wahbi, A. Bedoui, A. Abarda, and M. Dakkon, "Applying k-nearest neighbors to time series forecasting: Two new approaches," *Journal of Forecasting*, vol. 43, no. 5, pp. 1559–1574, 2024.

[86] F. Martínez, M. P. Frías, M. D. Pérez, and A. J. Rivera, "A methodology for applying k-nearest neighbor to time series forecasting," *Artificial Intelligence Review*, vol. 52, no. 3, pp. 2019–2037, Oct. 2019.

[87] D. T. Thi, N. T. Phong, and N. D. Bui, "Forecast timeseries based on matrix profile," *Journal of Informatics and Innovative Technologies (JIIT)*, 2021.

[88] P. Srisuradetchai, "A novel interval forecast for k-nearest neighbor time series: A case study of durian export in thailand," *IEEE Access*, vol. 12, pp. 2032–2044, 2024.

[89] E. Cartwright, M. Crane, and H. J. Ruskin, "Financial time series: Market analysis techniques based on matrix profiles," *Engineering Proceedings*, vol. 5, no. 1, p. 45, 2021.

[90] Q. Liu, D. L. X. Fung, L. Lac, and P. Hu, "A novel matrix profile-guided attention **LSTM** model for forecasting **COVID-19** cases in usa," *Frontiers in Public Health*, vol. 9, Oct. 2021.

[91] P. Saha, P. Nath, A. I. Middya, and S. Roy, "Improving temporal predictions through time-series labeling using matrix profile and motifs," *Neural Computing and Applications*, vol. 34, no. 16, pp. 13 169–13 185, Aug. 2022.

[92] R. Sen, H.-F. Yu, and I. S. Dhillon, "Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting," in *Advances in Neural Information Processing Systems*, vol. 32. Curran Associates, Inc., 2019.

[93] A. Mueen and E. Keogh, "Online discovery and maintenance of time series motifs," in *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '10. New York, NY, USA: Association for Computing Machinery, Jul. 2010, pp. 1089–1098.

[94] S. M. Law, "**STUMPY**: A powerful and scalable **Python** library for time series data mining," *Journal of Open Source Software*, vol. 4, no. 39, p. 1504, Jul. 2019.

[95] J. H. Friedman, "Greedy function approximation: A gradient boosting machine." *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.

[96] L. G. Serrano and S. Thrun, *Grokking Machine Learning.* Shelter Island: Manning, 2021.

[97] A. Géron, *Hands-On Machine Learning with **Scikit-Learn** and **PyTorch**: Concepts, Tools, and Techniques to Build Intelligent Systems.* US: O'Reilly Media, 2025.

[98] T. Chen and C. Guestrin, "**XGBoost**: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794.

[99] A. V. Dorogush, V. Ershov, and A. Gulin, "**CatBoost**: Gradient boosting with categorical features support," in *Workshop on ML Systems at NIPS 2017*, 2017.

[100] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "**LightGBM**: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[101] R. J. Williams and D. Zipser, "A learning algorithm for continually running fully recurrent neural networks," *Neural Computation*, vol. 1, no. 2, pp. 270–280, Jun. 1989.

[102] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, ser. Adaptive Computation and Machine Learning. Cambridge, Massachusetts London, England: The MIT Press, 2022.

[103] S. Kolassa and W. Schütz, "Advantages of the mad/mean ratio over the mape," *Foresight: The International Journal of Applied Forecasting*, vol. 6, pp. 40–43, 2007.

[104] C.-C. M. Yeh, N. Kavantzas, and E. Keogh, "**Matrix Profile IV**: Using weakly labeled time series to predict outcomes," *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1802–1812, Aug. 2017.

[105] L. A. Urry, M. L. Cain, S. A. Wasserman, P. V. Minorsky, R. B. Orr, and N. A. Campbell, *Campbell Biology*, twelfth ed. New York, NY: Pearson, 2020.

[106] B. Alberts, *Molecular Biology of the Cell*, 7th ed. New York: W. W. Norton & Company, 2022.

[107] W. W. Cohen, *A Computer Scientist's Guide to Cell Biology: A Travelogue from a Stranger in a Strange Land.* New York, NY: Springer-Verl, 2007.

[108] Y. Lee, K. Jeon, J.-T. Lee, S. Kim, and V. N. Kim, "**MicroRNA** maturation: Stepwise processing and subcellular localization," *The EMBO Journal*, vol. 21, no. 17, pp. 4663–4670, Sep. 2002.

[109] S. Gu, L. Jin, Y. Zhang, Y. Huang, F. Zhang, P. N. Valdmanis, and M. A. Kay, "The loop position of **shRNAs** and **Pre-miRNAs** is critical for the accuracy of **Dicer** processing in vivo," *Cell*, vol. 151, no. 4, pp. 900–911, Nov. 2012.

[110] Y. Feng, X. Zhang, P. Graves, and Y. Zeng, "A comprehensive analysis of precursor **microRNA** cleavage by human **Dicer**," *RNA*, vol. 18, no. 11, pp. 2083–2092, Jan. 2012.

[111] I. J. MacRae, K. Zhou, and J. A. Doudna, "Structural determinants of **RNA** recognition and cleavage by **Dicer**," *Nature Structural & Molecular Biology*, vol. 14, no. 10, pp. 934–940, Oct. 2007.

[112] F. Ahmed, R. Kaundal, and G. P. Raghava, "**PHDcleav**: A **SVM** based method for predicting human **Dicer** cleavage sites using sequence and secondary structure of **miRNA** precursors," *BMC Bioinformatics*, vol. 14, no. 14, p. S9, Oct. 2013.

[113] Y. Bao, M. Hayashida, and T. Akutsu, "**LBSizeCleav**: Improved support vector machine (**SVM**)-based prediction of **Dicer** cleavage sites using loop/bulge length," *BMC Bioinformatics*, vol. 17, no. 1, p. 487, Nov. 2016.

[114] P. Liu, J. Song, C.-Y. Lin, and T. Akutsu, "**ReCGBM**: A gradient boosting-based method for predicting human **Dicer** cleavage sites," *BMC Bioinformatics*, vol. 22, no. 1, p. 63, Feb. 2021.

[115] L. Mu, J. Song, T. Akutsu, and T. Mori, "**DiCleave**: A deep learning model for predicting human **Dicer** cleavage sites," *BMC Bioinformatics*, vol. 25, no. 1, p. 13, Jan. 2024.

[116] L. Mu and T. Akutsu, "**DiCleavePlus**: A transformer-based model to detect human **Dicer** cleavage sites within cleavage patterns," *Genes to Cells*, vol. 31, no. 1, p. e70074, 2026.

[117] S. Griffiths-Jones, H. K. Saini, and S. van Dongen, "**miRBase**: Tools for **microRNA** genomics," *Nucleic Acids Research*, vol. 36, no. suppl_1, pp. D154–D158, Jan. 2008.

[118] T. Xu, N. Su, L. Liu, J. Zhang, H. Wang, W. Zhang, J. Gui, K. Yu, J. Li, and T. D. Le, "**miRBaseConverter**: An **R/Bioconductor** package for converting and retrieving **miRNA** name, accession, sequence and family information in different versions of **miRBase**," *BMC Bioinformatics*, vol. 19, no. 19, p. 514, Dec. 2018.

[119] M. J. Zvelebil, J. O. Baum, and M. Zvelebil, *Understanding Bioinformatics*. New York: Garland Science, 2008.

[120] R. Lorenz, S. H. Bernhart, C. Höner zu Siederdissen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker, "**ViennaRNA Package 2.0**," *Algorithms for Molecular Biology*, vol. 6, no. 1, p. 26, Nov. 2011.

[121] C. C. Aggarwal, *Data Mining: The Textbook*. Cham: Springer International Publishing, 2015.

[122] G. Mendizabal-Ruiz, I. Román-Godínez, S. Torres-Ramos, R. A. Salido-Ruiz, and J. A. Morales, "On **DNA** numerical representations for genomic similarity computation," *PLOS ONE*, vol. 12, no. 3, p. e0173288, Mar. 2017.

[123] D. Anastassiou, "Genomic signal processing," *IEEE Signal Processing Magazine*, vol. 18, no. 4, pp. 8–20, Jul. 2001.

[124] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, "Searching and mining trillions of time series subsequences under dynamic time warping," in *Proceedings of the 18th ACM*

SIGKDD International Conference on Knowledge Discovery and Data Mining, ser. KDD '12. New York, NY, USA: Association for Computing Machinery, Aug. 2012, pp. 262–270.

[125] P. D. Cristea, "Conversion of nucleotides sequences into genomic signals," *Journal of Cellular and Molecular Medicine*, vol. 6, no. 2, pp. 279–303, 2002.

[126] N. Chakravarthy, A. Spanias, L. D. Iasemidis, and K. Tsakalis, "Autoregressive modeling and feature analysis of **DNA** sequences," *EURASIP Journal on Advances in Signal Processing*, vol. 2004, no. 1, pp. 1–16, Dec. 2004.

[127] J. Zhao, X. W. Yang, J. P. Li, and Y. Y. Tang, "**DNA** sequences classification based on wavelet packet analysis," in *Wavelet Analysis and Its Applications*. Springer, Berlin, Heidelberg, 2001, pp. 424–429.

[128] T. Holden, R. Subramaniam, R. Sullivan, E. Cheung, C. Schneider, G. T. Jr, A. Flamholz, D. H. Lieberman, and T. D. Cheung, "**ATCG** nucleotide fluctuation of deinococcus radiodurans radiation genes," in *Instruments, Methods, and Missions for Astrobiology X*, vol. 6694. SPIE, Oct. 2007, pp. 402–411.

[129] A. S. Nair and S. P. Sreenadhan, "A coding measure scheme employing electron-ion interaction pseudopotential (**EIIP**)," *Bioinformation*, vol. 1, no. 6, pp. 197–202, Oct. 2006.

[130] M. Akhtar, J. Epps, and E. Ambikairajah, "On **DNA** numerical representations for period-3 based **Exon** prediction," in *2007 IEEE International Workshop on Genomic Signal Processing and Statistics*, Jun. 2007, pp. 1–4.

[131] R. Zhang and C.-T. Zhang, "**Z Curves**, an intutive tool for visualizing and analyzing the **DNA** sequences," *Journal of Biomolecular Structure and Dynamics*, vol. 11, no. 4, pp. 767–782, Feb. 1994.

[132] J. A. Berger, S. K. Mitra, M. Carli, and A. Neri, "Visualization and analysis of **DNA** sequences using **DNA** walks," *Journal of the Franklin Institute*, vol. 341, no. 1, pp. 37–53, Jan. 2004.

[133] A. P. Ruiz, M. Flynn, J. Large, M. Middlehurst, and A. Bagnall, "The great multivariate time series classification bake off: A review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, vol. 35, no. 2, pp. 401–449, Mar. 2021.

[134] B. W. Matthews, "Comparison of the predicted and observed secondary structure of **T4** phage lysozyme," *Biochimica et Biophysica Acta (BBA) - Protein Structure*, vol. 405, no. 2, pp. 442–451, Oct. 1975.

[135] J. Gorodkin, "Comparing two k-category assignments by a k-category correlation coefficient," *Computational Biology and Chemistry*, vol. 28, no. 5, pp. 367–374, Dec. 2004.